# Source-Native Process Algebra and Initial-Coface Revision: Four-Tier Identity, Exact Evaluation Kernels, and Reversible Incremental Continuation

源生过程代数与初始余面修订：四层同一性、正合求值核与可逆增量续行

Jian Gao (高健)

Independent Researcher · innersummer@hotmail.com · ORCID 0009-0009-5002-1655

**Formal verification and edition note.** Every core construction, positive theorem, and negative-control gate in this paper is verified by the Lean 4 kernel across the full dependency closure of declaration types, proof terms, and inductive or recursive metadata. Throughout, we separate two foundational layers explicitly: a **zero-axiom constructive kernel (T0)** (`#print axioms = []`, relying on zero axioms) governing single-constructor rooted unfolding and universal folding, root-read safety without hidden-null blind spots, fixed-law successor rigidity, unique comparison between faithful process realizations without preset simulation relations, law-free $\leq$; $<$; $\leq$ payment-macro well-foundedness, the one-shot empty-fibre impossibility gate, and categorical initial-coface unique factorization with shared first-write revision under double-empty expressive bankruptcy; and a **standard three-axiom module and completion layer (T1)** (relying solely on propositional extensionality `propext`, classical choice `Classical.choice`, and quotient soundness `Quot.sound`, with zero research axioms and zero unfinished placeholders) governing free $\mathbb{Z}$-module and bilinear lifts over structureless states, exact evaluation kernels, inverse-limit history completions, maximal future-invisible invariant models, double-negation paid-AST homomorphic continuation, primitive-by-primitive cochain replay, bidirectional step–request slice inversion, and cross-domain operator and smooth dynamical instantiations—without back-contaminating the zero-axiom T0 kernel. This work is the restructured Third Edition (v3) of the published first and second editions (originally titled *The State Is Not the History, the Source Is: Identity, Responsibility, and Minimal Revision in Processes*, Second Edition DOI: 10.5281/zenodo.23297175), unifying the full **C1**–**C54** evidence chain. The first-release proof acceptance, second-edition source binding, Appendix D, and Code and Data Availability record the pinned source commits and reproduction entry points.

## Abstract

Static state snapshots and tick-wise numerical equalities cannot anchor identity across an evolving process: two locally valid trajectory segments agreeing at their shared state can still lack any common generating object across the combined history; two implementations can emit identical observation sequences without identifying which event realizes which occurrence; and ledger updates or expressive breakdowns can erase the lineage of unfinished obligations. Working across an explicitly separated **zero-axiom constructive kernel (T0)** and a **free-module completion and continuation layer (T1)**, we develop a **source-native process algebra** organized along four strictly strengthening tiers: *same reading* $<$ *same source object* $<$ *same rooted occurrence* $<$ *same obligation and revision*. A single inductive constructor $\mathrm{occur}(a, [T_1, \ldots, T_k])$ derives nonempty frontiers, associative substitution, parallel and joint cardinality shadows ($+$ and $\times$), and a universally unique fold. The Lean proof of the parallel cardinality formula uses `propext`. Root-read identity $\mathrm{rootRead}(o) = o$ excludes hidden-null witnesses for every downstream consumer. Between any two faithful realizations of a fixed root, the shared occurrence uniquely generates—without pre-supplied simulation relations or auxiliary history/prophecy variables—a pointwise event comparison compatible with emitters and whole-ledger compilation. Whenever an old theory has both empty expression-witness and empty lawful-realization fibres for an obstructed demand (rooted double-empty bankruptcy $\eta$), $\eta$ constructs in zero axioms the **categorical initial coface** $C_\eta = L_+ \sqcup \{\star_\eta\}$, preserving all old facts and theorems without contraction, admitting a unique injective budget-non-increasing factorization into every admissible extension, and sending the adjoined direction onto the exact live row consumed by a typed revision's first whole-ledger write so the new root answers directly at the second tick.

In the T1 module and reversible-continuation layer, arbitrary structureless state sets $S$ and binary operations lift freely into a many-sorted algebraic language satisfying the three-term bilinear update equation. Every rooted tree $T$ and constructor $\kappa$ compiles to a whole-tree program of exact syntax budget $\beta(T)$ equipped with an evaluation-free syntactic left inverse: even when $\kappa \equiv 0$ collapses all semantic outputs to one state point, the program map remains injective and recovers $T$ intact. Over any commutative ring $R$, derivation endpoint differences and module laws span the exact evaluation kernel $\mathrm{range}(\mathrm{rel}_\rho) = \ker(\mathrm{ev}_\rho)$, and every element of the inverse-limit completion $\widehat{C}(s)$ obeys the elementwise evolution law. Environment–expression pairs birth bearer-free computational obligations paid strictly step by step with `Type`-valued completion receipts returning exact distance $d_i = B_i$; after settlement, the updated difference $e \ominus e'$ births the next inquiry, writes back total value $V = p + n$ with fresh decoders, and generates six compatible faces alongside the maximal future-invisible invariant kernel model $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$. Finally, universal semantic-zero congruence identifies $e \ominus e'$ with the trace boundary $\delta_e - \delta_{e'}$ across all future action words and sites to form a writable request in canonical next; ring-free syntactic double negation retainedCode recovers paid ASTs to execute any pre-fixed additive homomorphism $f^{(n+1)}$ across cofinal stops while primitive-by-primitive whole-word replay updates substitution $\tau_1$, relation boundaries, and cochain differentials $d^{0,1}$ in lockstep; executed steps and next-batch signed requests invert bidirectionally via contiguous slices stepInterval and readRequestOrigin ($r = a_i + s$, rejecting omission or duplication of any nonempty group); and sealed histories extend across horizons by

locking the paid prefix and running only the missing tail suffix. Instantiations in Fock arithmetic, Riesz resolvent combs with windowed inverse-residual correction, and a 37-primitive whole-atom smooth dynamical system demonstrate cross-domain habitability.

**Keywords**: source-native process algebra; four-tier process identity; zero-axiom constructive kernel; exact evaluation kernel; syntactic left inverse; categorical initial coface; well-founded debt settlement; reversible incremental continuation; dependent-type formalization

## 摘要

静态状态快照与逐拍数值相等无法锚定演化过程的同一性：两段局部相容的读数在共享交点处取值相同，仍可能不存在任何贯穿全历史的共同生成对象；两套实现可以产出完全一致的观测序列而无法指认哪一事件属于哪次发生；一次模律更新或表达破产更可能抹去未结责任的谱系与归属。本文在显式分离的**零公理构造性内核（T0）**与**自由模完备化代数层（T1）**之上，建立沿“同一读数 $<$ 同一源对象 $<$ 同一次有根发生 $<$ 同一笔责任与修订”逐级闭合的**源生过程代数**。在构造层，单一归纳构造子 $\mathrm{occur}(a,[T_1,\dots,T_k])$ 无需第二构造子或原语算术公理即派生非空前沿、结合推进、并／联基数阴影（$+$ 与 $\times$）及万有唯一折叠。并列基数公式在 Lean 中依赖命题外延性。根读恒等 $\mathrm{rootRead}(o)=o$ 对一切下游消费排除隐藏空证人；同一权威根的任意两个忠实实现无需外部模拟关系或历史／预言辅助变量，即由共同发生零公理唯一导出与发射器及整套编译器相容的逐事件比较态射；当旧理论对某阻滞需求既无表达见证也无合法实现（双空破产 $\eta$）时，$\eta$ 零公理生成保守保持全部旧事实与旧定理的**范畴初始余面** $C_\eta = L_+ \sqcup \{\star_\eta\}$，对任意容许扩充给出三坐标单射且预算不回充的唯一态射分解，并将新增方向精确送入有类型修订新根首次整账写入所消费的新行，在第 2 拍由同一新根直接给出回答。

在 T1 代数与可逆续行层中，无需自带加法的任意原生状态集 $S$ 与二元函数经自由 $\mathbb{Z}$-模与张量泛性质进入多类异质表达式语言，满足双线性三项增量更新律；任意有根树与构造子函数 $\kappa$ 编译为步长恰等于语法预算 $\beta(T)$ 的整树程序，并由纯语法解析器给出不经求值的左逆——即便 $\kappa\equiv 0$ 使所有树求值坍缩为同一零状态点，程序映射仍保持单射并完整还原原树。任意交换环 $R$ 上归约推导端点差与标量模律的线性像正合等于求值核 $\mathrm{range}(\mathrm{rel}_\rho)=\ker(\mathrm{ev}_\rho)$，逆极限完备化载体 $\widehat{C}(s)$ 的全体元素满足逐拍演化等式。原发生登记的环境—表达式对无需借用旧承担者即内生生成执行债务律，逐步严格扣款并由 `Type` 值完成回执返回精确步距 $d_i=B_i$ 与已付前缀；结清后，已付关系在新环境中的差词 $e\ominus e'$ 内生提出预算严格为正的下一问，每次回执按 $V=p+n$ 写回总值并重生解码器，同一材料同步生成代数／组合／拓扑／逻辑／关系／余链六面及依值直和载体上的最大未来不可见不变核模型 $\mathsf{M}_{\mathrm{fut}}=\mathcal{W}/K_{\mathrm{fut}}$。进一步，全环境语义零关系证明残差请求与已付迹边界在一切允许未来词与站点下同属同一 $\mathsf{M}_{\mathrm{fut}}$ 类并直达规范后继可写请求；纯语法双取反提取器 retainedCode 无环标量恢复已付常量语法树以跨共尾执行任意预定加法同态 $f^{(n+1)}$，整词逐原语回放使替换链映射 $\tau_1$、实际边界与余链微分 $d^{0,1}$ 严格锁步；已执行原语步与下一批有号请求建立连续切片 stepInterval 与起源反演 readRequestOrigin 的双向恒等 $r=a_i+s$（漏删或重复任一非空组必改变原生迹），封存运行历史在视野延长时锁死已付前缀、仅从实际未态续跑缺失后缀。算术 Fock 空间、Riesz 预解梳与有限窗口逆残差纠正及 37 原语全原子连续平滑体系给出跨领域实装核验。


**关键词**：源生过程代数；四层过程同一性；零公理构造性内核；正合求值核；整树纯语法左逆；范畴初始余面；良基同债结算；可逆增量续行；依赖类型形式化

## Contents

Abstract 2

1 Introduction: Four-Tier Identity, Counterexample Anchors, and the Two-Kernel Spine 6

2 T0 Zero-Axiom Single-Constructor Unfolding and Native Free Lifts 13

3 Bilinear Three-Term Increments, Replayable Derivations, and Syntax-Only Left Inversion 17

4 The Exact Evaluation Kernel and Inverse-Limit History Completions 21

5 Faithful Process Realizations and Unique Comparison Without Preset Simulation Relations 24

6 The Responsibility Ledger: Lifecycle Conservation, Law-Free Well-Foundedness, and Bearer-Free Step Payment 26

7 Dependency-Tree Residuals, Double-Empty Bankruptcy, and Causal-Audit Feeds 31

8 T0 Zero-Axiom Initial-Coface Revision, Two-Tick Relay, and Automatic Post-Settlement Re-Inquiry 33

9 Complete Execution: Temporal Coding, All-Actor Recovery, Order-Preserving Stock Injection, and the Maximal Future Kernel 41

10 Reversible Incremental Continuation, Cross-Domain Instantiations, and Four-Axis Literature Dialogue 46

11 Conclusion 56

Appendix A Finite-Language Technical Details 58

Appendix B Completion-Carrier Details and Counterexample Proofs 59

Appendix C Fibre Definitions and Derivations from Rooted Failure to First Write 59

Appendix D Formalization Entry Points and Axiom Boundaries 60

Appendix E Quick Reference of Core Terms and Symbols 66

Code and Data Availability 67

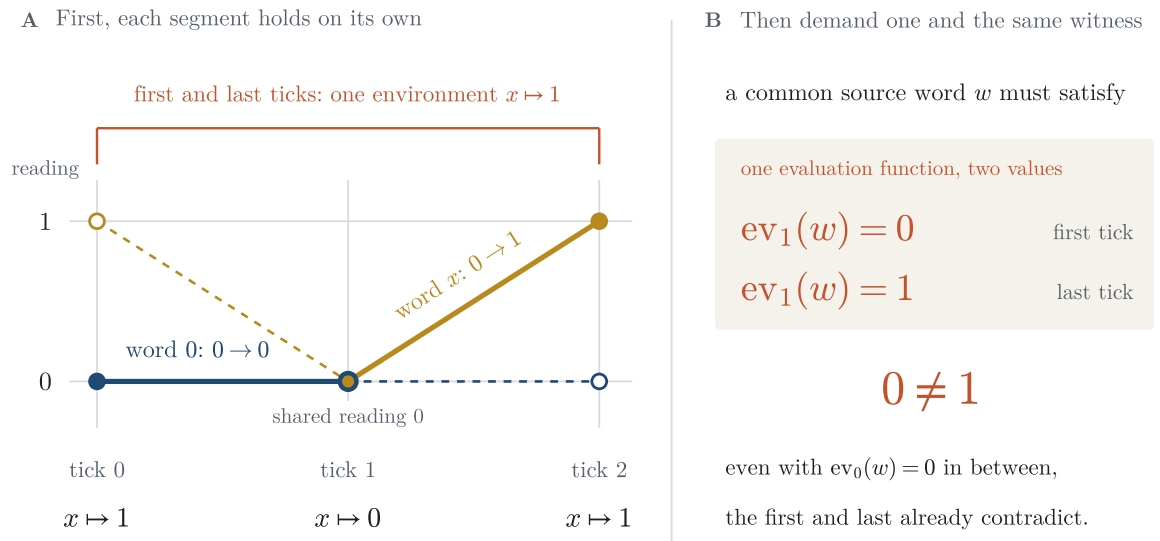
References 68

# 1 Introduction: Four-Tier Identity, Counterexample Anchors, and the Two-Kernel Spine

## 1.1 Why Equal Readings and State Snapshots Cannot Anchor Process Identity

Consider the simplest possible three-tick evolution problem. Let a single variable $x$ pass through the environments $x \mapsto 1$, $x \mapsto 0$, and $x \mapsto 1$. Say that an algebraic word **locally realizes** an update segment if its evaluations before and after the update match the prescribed target readings. The first segment updates the environment from 1 to 0 and asks for readings $0 \to 0$; the constant zero word $w_1 = 0$ realizes it. The second segment updates the environment from 0 back to 1 and asks for readings $0 \to 1$; the variable word $w_2 = x$ realizes it. At the shared middle environment $x \mapsto 0$, the two segments also agree numerically: both evaluate to 0.

Every local check passes and the junction values match. Yet if we ask whether there exists **one common source word** $w$ producing the readings $0, 0, 1$ from start to finish, the answer is no: any such $w$ would have to satisfy $\mathrm{ev}_1(w) = 0$, $\mathrm{ev}_0(w) = 0$, and $\mathrm{ev}_1(w) = 1$ simultaneously, forcing $0 = 1$ under one and the same evaluation map $\mathrm{ev}_1$ at the first and last ticks (Figure 1; Example 4.1). The fallacy lies in treating equal readings at a junction as evidence that both segments belong to the history of one persistent generating object. Equal readings show only that two objects are indistinguishable at that tick; they never make them the same source object.




**Caption**. Figure 1 | Local witnesses versus a common source word (exact numerical figure; Example 4.1, Table 1). The environments are $x \mapsto 1, 0, 1$ in turn. The deep-blue zero word realizes $0 \to 0$ on the first segment; the gold variable word realizes $0 \to 1$ on the second. Solid lines mark adopted local segments; dashed lines with hollow dots supply each word's true readings on the other segment; the vermilion bracket marks the shared environment $x \mapsto 1$ at the first and last ticks, forcing any common source word to satisfy $\mathrm{ev}_1(w) = 0$ and $\mathrm{ev}_1(w) = 1$ simultaneously.

The exact same structural gap recurs across algebraic relations, system implementations, unfinished obligations, and theory revisions:

1. **Cross-tick failure of kernel relations**: the polynomial word $x^2 - x$ evaluates to 0 under $x \mapsto 1$ (lying in the current evaluation kernel) yet jumps to $2 \neq 0$ after updating to

$x \mapsto 2$ (Example 4.2)—treating an equation that holds at one tick as a permanent identity manufactures false equalities at the next.

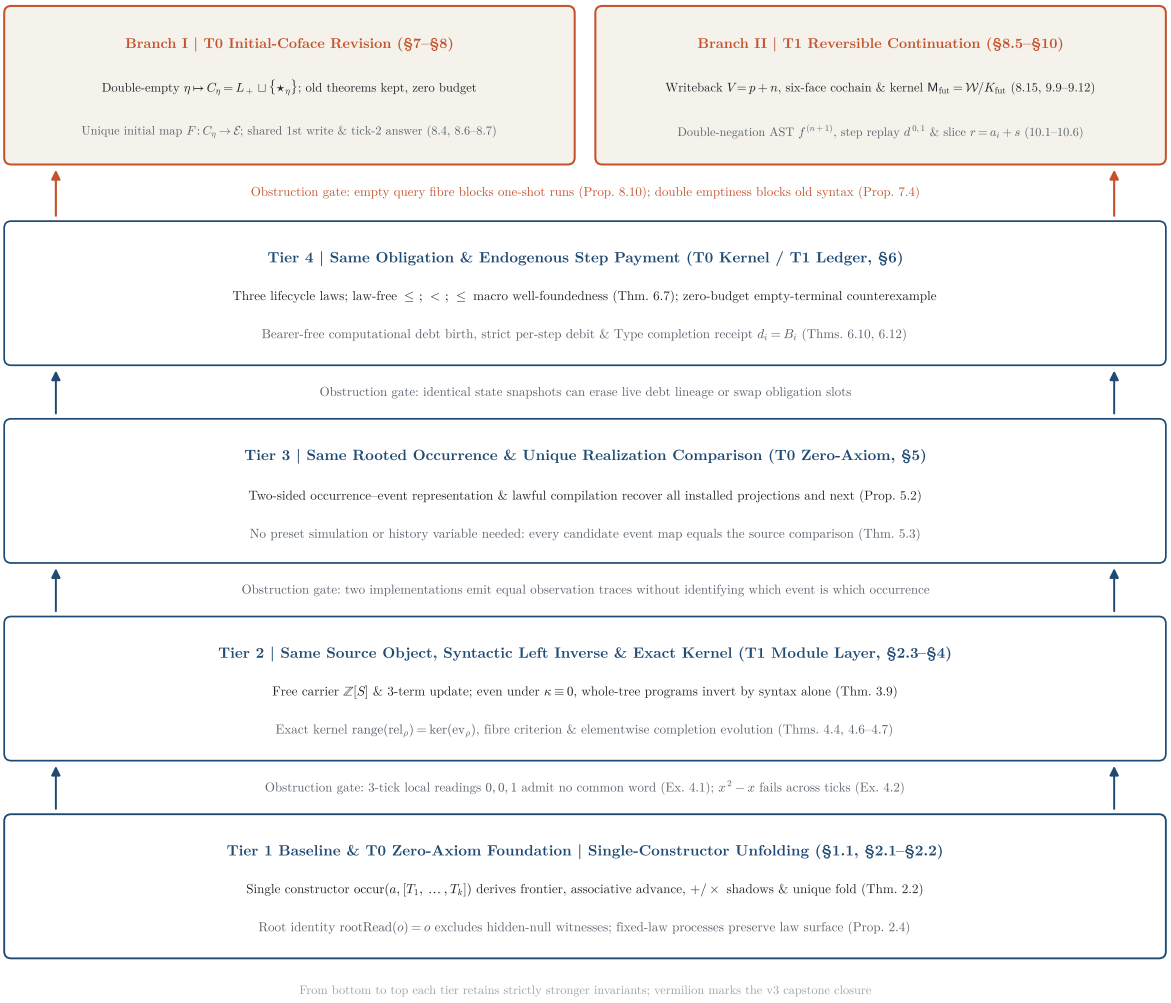
2. **Observationally equivalent traces with unanchored occurrences**: two implementations in different host representations can emit identical value streams without state snapshots ever identifying which implementation event realizes which underlying occurrence, unless external history variables and future-looking prophecy variables are adjoined[7].
3. **Balance-sheet re-labelling that erases live debt lineage**: after a ledger update, comparing aggregate balances alone or permitting zero-cost carry steps can launder an obligation's provenance or let unfinished computational work spin forever without discharging.
4. **Expressive bankruptcy beyond the old vocabulary**: when a new occurrence exposes a demand for which the old theory has neither an expressing syntax term nor a lawful realization witness (double-empty bankruptcy), belief contraction inside a fixed language[18] is powerless—nothing in the old theory is false, yet the expressive structure itself must be enlarged minimally, uniquely, and with full live-ledger accountability.

This paper develops a **source-native process algebra** that organizes these phenomena into **four strictly strengthening tiers of process identity** and closes both expressive-bankruptcy revision and cross-horizon reversible continuation at the top tier (Figure 2). Foundational axioms are split cleanly across two kernels: nonempty frontiers, associative advance, unique folds, root-read safety, realization comparison, macro-step well-foundedness, and categorical initial-coface factorization are proved inside the **zero-axiom constructive kernel T0** (Lean 4 `#print axioms = []`, invoking neither choice, propositional extensionality, nor quotients[17]). The parallel cardinality formula in Theorem 2.2 uses `propext`. Constructions involving free $\mathbb{Z}$-module quotients, exact presentations over arbitrary commutative rings $R$, inverse-limit completions, direct-sum future kernels, and analytic instantiations reside in the **standard three-axiom layer T1** (`propext`, `Classical.choice`, `Quot.sound`), with the exact boundary marked on every result.

The positive theorems, paired counterexamples or negative-control gates, and axiom-kernel assignments across the four tiers are summarized below (numbered tables begin at Table 1):

| Identity tier & axiom kernel | Core positive theorems locked | Paired counterexample / negative-control gate | Section & proof |
|---|---|---|---|
| **T0 foundation & Tier 1 baseline** (`[]`) Single-constructor unfolding | Single constructor $\mathrm{occur}(a, \vec{T})$ derives frontier, associative advance, the joint cardinality shadow & unique fold; root identity $\mathrm{rootRead}(o) = o$ excludes hidden-null witnesses; fixed-law processes preserve law surface | Lower-dimensional projections admit hidden-null blind spots; unrecorded extra parameters cannot fork lawful successors | §2.1–§2.2 (Thm. 2.2, Prop. 2.4) |

| Identity tier & axiom kernel | Core positive theorems locked | Paired counterexample / negative-control gate | Section & proof |
|---|---|---|---|
| **Tier 2: Same source object** (T1) Free lift, syntactic left inverse & exact kernel | Structureless state lift $\mathbb{Z}[S]$ & 3-term update; syntax-only left inverse $\mathrm{read}(\mathrm{prog}_\kappa T) = T$; exact kernel $\mathrm{range}(\mathrm{rel}_\rho) = \ker(\mathrm{ev}_\rho)$; elementwise evolution across $\widehat{C}(s)$ | 3-tick local readings $0, 0, 1$ admit no common word (Ex. 4.1); old relation $x^2 - x$ fails across ticks (Ex. 4.2); constant-zero $\kappa \equiv 0$ collapses all values yet keeps every tree (Thm. 3.9(4)) | §2.3–§4 (Thms. 3.9, 4.4, 4.6–4.7; Apps. A–B) |
| **Tier 3: Same occurrence** (T0 `[]`) Faithful realization & unique comparison | Two-sided occurrence–event representation recovers all installed projections & native next; any two faithful realizations yield a **pointwise unique** event comparison preserving compilation | Identical observation streams fail to fix cross-implementation event identity; multi-point visit fibres lose unconditional uniqueness | §5 (Prop. 5.2, Thm. 5.3) |
| **Tier 4: Same obligation** (T0 kernel / T1 execution ledger) | Three lifecycle laws (no free allocation, no uncertified revival, no silent disappearance); law-free $\leq$; $<$; $\leq$ well-foundedness; bearer-free step debit & `Type` receipt $d_i = B_i$ | Equal-budget transfers spin indefinitely; zero budget with empty terminal fibre makes the master law uninhabited (Note 6.8); zero semantic effect still pays syntax debit | §6 (Props. 6.1, 6.4–6.5; Thms. 6.7, 6.10, 6.12; Supp. S1) |
| **Capstone Branch I: Initial-coface revision** (T0 `[]`) Bankrupt-language ascent | Dependency residuals yield double-empty failure $\eta$; zero-axiom initial coface $C_\eta = L_+ \sqcup \{\star_\eta\}$ preserves all old theorems without old-face leakage, factors **uniquely & injectively**, and enters the new root's 1st write to answer at tick 2 | One-shot empty-fibre interfaces admit no autonomous runtime (Prop. 8.10); double emptiness blocks old syntax & realizations (Prop. 7.4); transfer rows cannot pose as audit carries | §7–§8.4 (Props. 7.4–7.5, 8.9–8.10; Thms. 8.4, 8.6–8.7; App. C) |
| **Capstone Branch II: Reversible continuation** (T1) Homomorphic AST & slice inversion | Total writeback $V = p + n$, six-face cochain & maximal future model $\mathsf{M}_{\mathrm{fut}}$; semantic-zero congruence into next request (C46–C49); retainedCode double-negation method $f^{(n+1)}$ & step cochain replay (C51–C52); stepInterval and readRequestOrigin inversion $r = a_i + s$ & extendTo prefix lock (C53–C54) | Constant $(1, 0)$ zero-effect control verifies non-erasure of old paid values (Supp. S5.2); omitting or duplicating any nonempty step slice strictly alters the native trace (Thm. 10.5 gate) | §8.5–§10.3 (Thms. 8.12, 8.14–8.15, 9.2–9.3, 9.5–9.12, 10.1–10.6; Supp. S2–S6) |




**Caption**. Figure 2 | Four-tier process identity, T0/T1 axiom boundary, and bifurcated capstone architecture (structural diagram). From bottom to top each tier retains strictly stronger invariants: the foundation rests on the zero-axiom ( `[]` ) single-constructor unfolding $\mathrm{occur}(a, [T_1, \ldots, T_k])$ and root-read identity $\mathrm{rootRead}(o) = o$ (§2.1–§2.2, Theorem 2.2, Proposition 2.4); Tier 2 (T1) embeds structureless states into $\mathbb{Z}[S]$ and establishes syntax-only whole-tree left inversion even under $\kappa \equiv 0$, the exact evaluation kernel $\mathrm{range}(\mathrm{rel}_\rho) = \ker(\mathrm{ev}_\rho)$, and elementwise completion evolution (§2.3–§4, Theorems 3.9, 4.4, 4.6–4.7); Tier 3 (T0) yields pointwise unique comparison between faithful realizations via their shared occurrence without preset simulation relations or history variables (§5, Theorem 5.3); Tier 4 proves law-free $\leq$ ; $<$ ; $\leq$ macro well-foundedness (T0), bearer-free per-step debit, and `Type` completion receipts $d_i = B_i$ (§6, Theorems 6.7, 6.10, 6.12); the top capstone bifurcates into Branch I (§7–§8: T0 initial-coface unique factorization $F : C_\eta \to \mathcal{E}$ and shared first-write two-tick revision under double-empty bankruptcy, Theorems 8.4, 8.6–8.7) and Branch II (§8.5–§10: T1 total writeback $V = p + n$, six-face cochain, maximal future kernel $\mathsf{M}_{\mathrm{fut}}$, double-negation homomorphic AST reuse $f^{(n+1)}$, primitive cochain replay $d^{0,1}$, and bidirectional step–request slice inversion $r = a_i + s$, Theorems 8.15, 9.9–9.12, 10.1–10.6). Inter-tier labels state the counterexamples necessitating each tier; vermilion frames mark the third-edition capstone.

### 1.2 A Three-Minute Pen-and-Paper Verification Anchor

Before entering the general formalism, the reader can verify three elementary calculations in three minutes with pen and paper, directly witnessing our three signature mechanisms: **stage-inventory separation and evaluation-free left inversion**, **ring-free double-negation recovery of paid ASTs for homomorphic continuation**, and **shared first-write continuation across an initial coface**.

**Example 1.1 (Three-minute pen-and-paper check).**

**(a) Stage inventories expose the three-tick splice at one tick, and syntactic left inversion survives all-zero collapse.** Along the three-tick environments $1 \to 0 \to 1$ of Figure 1, tick 1 has old environment $\rho_1(x) = 0$ and increment $\varepsilon_1 = 1 - 0 = +1$. The one-tick **stage inventory** (Definition 4.5) records the joint pair $U_1(w) = (\mathrm{ev}_{\rho_1} w,\ \mathrm{eff}_{\rho_1,\varepsilon_1} w)$:

$$U_1(0) = (0,\, 0), \qquad U_1(x) = (0,\, 1) \neq (0,\, 0).$$

The "shared reading 0" at the junction is merely agreement on the old coordinate; the effect coordinate separates the two words immediately (Theorem 4.6). Conversely, take two distinct rooted trees $T_1 = \mathrm{occ}(a; [\,])$ and $T_2 = \mathrm{occ}(a; [\mathrm{occ}(b; [\,])])$ under the constantly zero constructor $\kappa \equiv 0 : \mathrm{Root} \to \mathrm{List}(\mathbb{Z}) \to \mathbb{Z}$. Both evaluate to the identical state point $\mathrm{ev}_\delta(\mathrm{prog}_\kappa T_1) = \mathrm{ev}_\delta(\mathrm{prog}_\kappa T_2) = \delta_0$; yet their syntactic budgets are $\beta(T_1) = 2$ and $\beta(T_2) = 2 + (2 + 0 + 1) = 5$, and the evaluation-free parser read recovers $\mathrm{read}(\mathrm{prog}_\kappa T_1) = T_1 \neq T_2 = \mathrm{read}(\mathrm{prog}_\kappa T_2)$ without ever invoking $\kappa$ (Theorem 3.9(4)). Readings can forget everything; the source program loses not a single node.

**(b) Double negation extracts the paid constant AST without ring scalars to run homomorphisms while charging zero-effect syntax steps.** Evaluate $e = x + x$ in $\rho_0(x) = 3$: left variable binding, right variable binding, and constant addition consume $r(e) = 3$ strict debits $(3 \to 2 \to 1 \to 0)$, settling at the paid constant AST $e' = \mathrm{const}(6)$. If the next environment stays at $\rho_1(x) = 3$ $(\varepsilon = 0)$, settlement births the residual request

$$e \ominus e' = (x + x) + \mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(6)),$$

whose semantic effect is $\mathrm{ev}_{\rho_1}(e \ominus e') = 6 - 6 = 0$, while its syntactic remaining count is $r(e \ominus e') = r(e) + r(e') + 2 = 3 + 0 + 2 = 5 > 0$—vanishing semantic effect never waives the 5-step syntax verification debit (Theorems 8.12(2) and 10.2). Entering the next cofinal packet, the syntactic extractor retainedCode pulls out the right summand $\mathrm{paid} = \mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(6))$ and applies syntactic negation once more (Theorem 10.3):

$$\mathrm{retainedCode} = \mathrm{linear}_{-\mathrm{id}}(\mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(6))), \qquad \mathrm{ev}_{\rho_1}(\mathrm{retainedCode}) = -(-6) = 6.$$

With neither a ring scalar $-1$ nor an external memo table, the paid value $p = 6$ is recovered in place from residual syntax; applying a pre-fixed doubling homomorphism $f(v) = 2v$ evaluates the next packet directly to $(f(6), 0) = (12, 0)$ in born stock.

**(c) Initial-coface attachment and two-tick relay under double-empty bankruptcy.** Suppose an old support carries one live debt row $e_o$ of budget 2, while the old theory's expression-witness and lawful-realization types for obstruction $o$ are both empty ($E_\Theta = \emptyset, I_\Theta(o) = \emptyset$). Rooted failure $\eta$ constructs in zero axioms the initial coface $C_\eta = L_+ \sqcup \{\star_\eta\}$ with $\beta(\mathrm{inl}\, e_o) = 2$ and $\beta(\mathrm{inr}\, \star_\eta) = 0$ (Proposition 8.2). A typed revision on current carrier $\mathbb{N}$ executes its first write $0 \to 1$ via a two-row patch consuming both rows $(2 \to 2,\ 0 \to 0)$, and target compilation $1 \to 2$ produces macro successor 2; at tick 2, visit 2 of the same new root answers $*$ directly without re-revising the question and advances to 3 (Proposition 8.9, Table 4).

### 1.3 Fifteen-Minute Sub-Discipline Verification Channels

Because this paper bridges programming languages and incremental computation, category theory and homological algebra, concurrency and coalgebraic semantics, and dependent type theory, specialists can verify our main theorems along four self-contained fifteen-minute tracks:

- **Programming languages, incremental view maintenance, and reversible computation**: read §2.3 (free carrier $\mathbb{Z}[S]$ and joint binary lift) → §3.1–§3.4 (Proposition 3.3 three-term update, **Theorem 3.9** whole-tree syntactic charging and evaluation-free left inverse under $\kappa \equiv 0$) → §8.5 (**Theorems 8.12 and 8.15** residual re-inquiry and $V = p + n$ total writeback) → **§10.1–§10.3 (Theorems 10.1–10.6, Example 10.7**: double-negation retainedCode homomorphic continuation, stepInterval and readRequestOrigin slice inversion, and extendTo prefix-locked tail continuation; full proofs in Supplement S6, literature comparison in §10.5 Axis 1).
- **Category theory, commutative/homological algebra, and algebraic specification**: read §4.2–§4.3 (**Theorem 4.4** two-certificate exactness $\mathrm{range}(\mathrm{rel}_\rho) = \ker(\mathrm{ev}_\rho)$ over arbitrary commutative rings, Appendix A.3; **Theorems 4.6–4.7** inverse-limit completion evolution, Appendix B.1) → **§8.1–§8.3 (Theorems 8.4 and 8.6** zero-axiom categorical initial coface unique factorization and first-write confluence, Appendices C.1–C.4) → §9.4 and §10.2 (**Theorems 9.11 and 10.4** six-face cochain complex $d_0 d_1 = 0$ and primitive-replay chain-map lockstep; comparison with institutions[19] in §10.5 Axis 4).
- **Process algebra, coalgebraic bisimulation, and formal refinement**: read §2.1–§2.2 (**Theorem 2.2** single-constructor unfolding and root-read safety, **Proposition 2.4** fixed-law successor rigidity) → **§5 (Definition 5.1, Theorem 5.3** unique realization comparison without history or prophecy variables) → §9.2 and §9.4 (**Theorems 9.3, 9.9, and 9.12** all-actor recovery, whole-word recovery prior to order-preserving stock injection, and the maximal future-invisible invariant kernel $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$; comparison with TLA/refinement mappings[6][7] and universal coalgebra[9] in §10.5 Axis 2).
- **Dependent type theory, resource logics, and theorem proving**: read §2.1 (T0 `#print axioms = []` single-constructor and non-univalent isomorphism boundary) → **§6.1–§6.5 (Proposition 6.1** lifecycle conservation laws, **Theorem 6.7** law-free $\leq$; $<$; $\leq$ well-foundedness kernel, **Theorem 6.10** bearer-free execution ledger, **Theorem 6.12** `Type`-valued completion receipt $d_i = B_i$, Supplement S1) → Appendix D (pinned Git commits, declaration signatures, and T0/T1 axiom closures; comparison with Martin-Löf type theory[15], HoTT[16], and resource separation logics[12][13] in §10.5 Axis 3).

### 1.4 Literature Coordinates, Scope Boundaries, and Paper Outline

**Four-axis literature dialogue.** Section 10.5 (with Table 5) conducts a theorem-level dialogue with twenty-four verified primary works across four axes: (1) **Incremental, self-adjusting, bidirectional, and reversible computation**: crediting DBSP[1] for Z-sets $\mathbb{Z}[A]$ and the three-term bilinear increment formula alongside tensor universal properties[5], and contrasting higher-order change theory[2], adaptive functional programming with dynamic dependency graphs and memo tables[22], provenance semirings[3], bidirectional tree lenses get and put[23], and Bennett's compute–copy–uncompute reversible Turing machines[4] with our exact evaluation kernel, all-zero-resilient one-sided syntactic left inverse, memo-free double-negation AST reuse, and uncomputation-free step–request slice inversion; (2) **Classical process calculi, refinement, and coalgebraic observability**: contrasting Milner's CCS[20], Bergstra and Klop's axiomatic process algebra ACP[21], TLA[6], refinement mappings with history/prophecy variables[7], event structures[8], Rutten's universal coalgebra and final-coalgebra bisimulation[9], and Kalman minimal realization[10] to clarify how our source-native process algebra preserves exact branching occurrence trees rather than quotienting by nondeterministic choice or bisimulation, while $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$ realizes the module-theoretic observability quotient with zero prophecy variables; (3) **Resource logics and type-theoretic foundations**: contrasting linear logic[11], separation logic[12], amortised resource separation logic[13], proof-carrying code[14], intuitionistic type theory[15], HoTT univalence[16], and the Lean 4 kernel[17] to explain why receipts inhabit `Type` and why T0 deliberately refuses to equate isomorphic carriers without a shared occurrence; (4) **Theory change, simplicial coface operators, and categorical institutions**: contrasting AGM contraction[18], simplicial coface/codegeneracy operators[24], and Goguen–Burstall institutions[19] (the Satisfaction Condition and conservative extensions) with endogenous initial-coface generation from rooted double-empty bankruptcy.

**Scope and explicit input boundaries.** All processes in this paper are **deterministic single-root processes**: at each temporal visit the fixed root emits a unique canonical occurrence and admits at most one lawful whole-ledger evolution (§5); nondeterministic branching and concurrent conflict lie outside our scope. Throughout, we distinguish objects **endogenously generated** by the source or compiler from **caller-supplied hypotheses**: (i) the same-debt master law for general domain obligations (Theorem 6.7 proves "master law ⇒ finite terminal settlement", whereas §6.5 computational obligations require no external master law); (ii) typed semantic revisions and their semantic anchoring proofs (Theorems 8.6–8.7); and (iii) re-registering a revised new root as a source process via Definition 2.3 to reuse §2.3–§4 (kept as an explicit parameterized interface at the end of §8.4). Likewise, semantic-fibre characterization (Proposition 3.6(3)) and the nonconstructive six-way step classification (Proposition 7.1, marked `noncomputable` in Lean) are structural classifications rather than decision algorithms or complexity bounds.

**Notation and outline.** Four standard notations hold throughout: dependent sums ($\Sigma$-types) are written $\sum_{x:A} B(x)$ with pairs $(x, y)$ or $\langle x, y\rangle$; disjoint unions (coproducts) are written $A \sqcup B$ (or $A \oplus B$) with injections $\mathrm{inl}, \mathrm{inr}$; free $R$-modules are written $\mathrm{Fm}_R(X) = X \to_0 R$ ($\mathbb{Z}[S]$ over $\mathbb{Z}$) with basis elements $\delta_s$; heterogeneous equality across equal type fibres is written $\simeq$. Section 2 develops T0 single-constructor unfolding, source processes, and free lifts; §§3–4 establish the three-term update law, whole-tree syntactic left inversion, the exact evaluation kernel,

and inverse-limit completions; §5 proves unique realization comparison; §6 establishes the three lifecycle laws, law-free macro well-foundedness, bearer-free step payment, and `Type` receipts; §§7–8 establish dependency-residual auditing, T0 initial-coface unique factorization, two-tick revision, and post-settlement re-inquiry with total-value writeback; §9 establishes temporal coding, all-actor recovery, two-inventory admission crossing, six-face complexes, and the maximal future kernel; §10 establishes universal semantic-zero requests, double-negation homomorphic continuation, primitive cochain replay, bidirectional step–request slice inversion, and sealed-prefix tail continuation (C46–C54), followed by three cross-domain instantiations and the four-axis literature dialogue; §11 concludes. Appendices A–C give complete technical derivations, Appendices D–E index formal declarations and terminology, and Supplement S0–S7 gives complete self-contained proofs of §§6.5, 7.2 (end), 8.5, 9, and 10.

## 2 T0 Zero-Axiom Single-Constructor Unfolding and Native Free Lifts

For an answer to belong to a specific run, the foundation must first provide an occurrence structure driven neither by external arithmetic axioms nor by pre-stored future tables, and then lift arbitrary structureless domain states into linear algebraic carriers without merging distinct sources. Subsections 2.1–2.2 establish single-constructor unfolding, root-read safety, and fixed-law process rigidity (C3). The parallel cardinality formula uses `propext` in Lean; §§2.3–2.4 enter the **T1 module layer** to construct free $\mathbb{Z}$-module and bilinear lifts alongside the runtime actual-witness principle (C9).

**Minimal-record definition of world networks and authoritative roots.** A **world relation network** $N$ is a record of the carrier types and dependent type families in the first three rows below; an **authoritative root** on $N$ is a running registration fixing vocabulary, native currents, emitter, and whole-ledger compiler. A **temporal visit** $v$ is one history-bearing visit of the root, and an **occurrence** is the dependent object canonically emitted there. Writing $c$ for a current at the root and $s(c)$ for its support, the last three rows list the occurrence and evolution fields derived from root registration.

| Field owner | Carrier or dependent family | Structural role |
|---|---|---|
| $N$: support and location | Sup; anchors, incidence, lineage, and support readouts | Locates occurrences and records provenance, connections, and carried identity |
| $N$: responsibilities and claims | Responsibility, Claim; $\mathrm{OpenAt}(s, r)$, $\mathrm{HoldsAt}(s, a)$; row claim and natural budget | Opening witnesses make responsibilities live ledger rows; fact witnesses make claims hold |
| $N$: obstructions and dispositions | $\mathrm{Obs}(s)$ with obstruction claim; semantic-change families; $\mathrm{Disposition}(s, k)$ | Records obstructions and dependent receipts for whole-support settlement, transfer, or extension |
| Root: canonical occurrence | $\mathrm{Occ}_W(c)$, $g_c \in \mathrm{Occ}_W(c)$ | Fixes the visit uniquely as its canonically emitted occurrence object |
| Root: lawful evolution | $\mathrm{Evol}_W(c)$, $\mathrm{Lawful}_c(g, e)$ | Identifies the lawful whole-ledger evolution $e$ of occurrence $g$ among candidates |

| Field owner | Carrier or dependent family | Structural role |
|---|---|---|
| Root: whole-ledger compiler | $\mathrm{comp}_c : \mathrm{Occ}_W(c) \to \mathrm{Evol}_W(c)$ | Generates whole-ledger write-back and the next current for subsequent visits |

Here OpenAt, HoldsAt, and dispositions are witness-bearing `Type` families rather than proof-erasing propositions. The **causal world** $W$ derived from an authoritative root simultaneously locks in two anti-magic laws: first, singleton visit occurrence fibres—every $g \in \mathrm{Occ}_W(c)$ satisfies $g = g_c$; second, uniqueness of lawful evolutions—the compiler selects a lawful evolution for every occurrence and every lawful evolution of that occurrence equals it. Both laws are generated constructively by the root compiler rather than assumed from callers (Appendix D.1).

## 2.1 Zero-Axiom Single-Constructor Unfolding and Root-Read Identity (T0)

Prior to introducing rings, modules, or primitive natural-number addition, all hierarchical causality, leaf-frontier advance, parallel/joint composition, and universal folding required by a rooted process derive in zero axioms from **one inductive constructor alone**.

**Definition 2.1 (Single-constructor rooted accounted unfolding).** Let $A$ be any occurrence-label type. **Rooted accounted unfoldings** $\mathcal{U}(A)$ and ordered branch lists are generated by a single mutually inductive constructor:

$$T ::= \mathrm{occur}(a,\ [T_1, \ldots, T_k]) \qquad (a : A,\ k \geq 0).$$

No empty-tree constructor, sibling-merge constructor, or stored completed-future field is admitted. Pure structural recursion over occur derives:

1. **Current root and zero-lookahead observation**: $\mathrm{root}(\mathrm{occur}(a, \vec{T})) := a$ and $\mathrm{observe}(T) := \mathrm{root}(T)$ (reading only the generated root, never unexecuted branches);
2. **Leaf frontier and preorder trace**: $\mathrm{fr}(\mathrm{occur}(a, [\,])) := [a]$, while for $\vec{T} \neq [\,]$, $\mathrm{fr}(\mathrm{occur}(a, \vec{T})) := \mathrm{fr}^*(\vec{T})$ concatenates child frontiers in order; $\mathrm{tr}(\mathrm{occur}(a, \vec{T})) := a :: \mathrm{tr}^*(\vec{T})$ records all occurred nodes in preorder;
3. **One-step frontier advance**: given $\sigma : A \to \mathcal{U}(A)$, $\mathrm{adv}(T, \sigma)$ replaces each leaf $\mathrm{occur}(a, [\,])$ by $\sigma(a)$ while preserving internal roots and advancing branches recursively;
4. **Rooted parallel and joint composition**: for a root label $r : A$ and $U, V \in \mathcal{U}(A)$, define **parallel composition** $U \oplus_r V := \mathrm{occur}(r, [U, V])$ and **joint composition** $U \otimes_r V := \mathrm{occur}(r,\ [\mathrm{adv}(U,\ x \mapsto V)])$ grafting $V$ onto every leaf of $U$; the frontier length $|\mathrm{fr}(T)| \in \mathbb{N}$ is the **cardinal shadow** of $T$;
5. **Universal fold**: for carriers $C, L$ and algebra $\mathcal{A} = (f : A \to L \to C,\ e : L,\ c : C \to L \to L)$, define mutually $\mathrm{fold}_{\mathcal{A}}(\mathrm{occur}(a, \vec{T})) := f(a, \mathrm{fold}^*_{\mathcal{A}}(\vec{T}))$, $\mathrm{fold}^*_{\mathcal{A}}([\,]) := e$, and $\mathrm{fold}^*_{\mathcal{A}}(T :: \vec{T}) := c(\mathrm{fold}_{\mathcal{A}} T, \mathrm{fold}^*_{\mathcal{A}} \vec{T})$.

On the exact temporal root-event type $\mathcal{E}(R)$ of an authoritative ledger root $R$, define **root-coface readout** as the identity $\mathrm{rootRead}(o) := o$. For any observation $\pi : \mathcal{E}(R) \to O$ and

downstream disposition $\chi : \mathcal{E}(R) \to D$, $\pi$ is **consumer-safe** for $\chi$ if $\pi(o_1) = \pi(o_2) \Rightarrow \chi(o_1) = \chi(o_2)$; a **hidden-null witness** is a pair $(o_1, o_2)$ with $\pi(o_1) = \pi(o_2)$ yet $\chi(o_1) \neq \chi(o_2)$.

**Theorem 2.2 (Universal fold, frontier associativity, and root-read identity).** For every type $A$, unfoldings $T, U, V \in \mathcal{U}(A)$, continuations $\sigma, \tau : A \to \mathcal{U}(A)$, and authoritative root $R$:

1. **Nonempty frontier and associative advance**: $\mathrm{fr}(T) \neq [\,]$ (hence $|\mathrm{fr}(T)| \geq 1$), and frontier advance obeys strict associativity and cardinal-shadow homomorphisms

$$\begin{aligned} &\mathrm{adv}(\mathrm{adv}(T, \sigma), \tau) = \mathrm{adv}(T,\ x \mapsto \mathrm{adv}(\sigma(x), \tau)), \\ &\quad |\mathrm{fr}(U \oplus_r V)| = |\mathrm{fr}(U)| + |\mathrm{fr}(V)|, \qquad |\mathrm{fr}(U \otimes_r V)| = |\mathrm{fr}(U)| \cdot |\mathrm{fr}(V)|. \end{aligned} \tag{2.1}$$

   The public Lean proof of the parallel cardinality formula uses `propext`. The other listed declarations have empty axiom lists.
2. **Universal uniqueness of fold**: any pair of maps $(g : \mathcal{U}(A) \to C,\ g^* : \mathrm{List}(\mathcal{U}(A)) \to L)$ satisfying the node and branch recursion equations equals $(\mathrm{fold}_{\mathcal{A}}, \mathrm{fold}^*_{\mathcal{A}})$ pointwise on every input.
3. **Blind-spot-free root readout and installed-projection factorization**: $\mathrm{rootRead}(o) = o$ is consumer-safe for **every** downstream disposition $\chi : \mathcal{E}(R) \to D$, its hidden-null witness type is strictly empty, and every installed subsystem projection $\chi_{\mathrm{sub}}$ on $R$—together with whole-ledger write-back and next current—factors through the unique canonical root evolution at that visit.

Proof. (1) By mutual structural induction on $T$ and branch lists: at $\vec{T} = [\,]$, $\mathrm{fr}(T) = [a] \neq [\,]$; at $\vec{T} = T_1 :: \vec{T}'$, $\mathrm{fr}(T) = \mathrm{fr}(T_1) +\!+ \mathrm{fr}^*(\vec{T}')$ is nonempty because $\mathrm{fr}(T_1)$ is nonempty. For associativity, leaves evaluate both sides to $\mathrm{adv}(\sigma(a), \tau)$, while at nonempty branch lists $\mathrm{adv}(T_1, \sigma)$ remains constructed by occur so the nonempty branch guard is preserved and the list induction hypothesis applies. Parallel cardinality follows from list-append length additivity; joint cardinality follows by induction on $U$, with leaves yielding $|\mathrm{fr}(V)| = 1 \cdot |\mathrm{fr}(V)|$ and branches distributing over sums. (2) Mutual induction rewrites subterms at each constructor without function extensionality. (3) If $\mathrm{rootRead}(o_1) = \mathrm{rootRead}(o_2)$, then $o_1 = o_2$ definitionally so `congrArg` gives $\chi(o_1) = \chi(o_2)$, making any hidden-null witness self-contradictory; projection factorization composes canonical evolution uniqueness $e = e_{\mathrm{can}}$ with projection installation commutation. ■

Theorem 2.2 anchors the constructive starting point of the paper and sets our framework apart from classical concurrency process calculi: whereas Milner's CCS[20] and Bergstra–Klop's ACP[21] axiomatize nondeterministic choice $+$ and sequential product $\cdot$ modulo bisimulation or right distributivity $(x + y) \cdot z = x \cdot z + y \cdot z$ (collapsing trees that branch at different points), our single constructor $\mathrm{occur}(a, \vec{T})$ preserves exact rooted branching geometry while $+, \times$ emerge solely as cardinal shadows of $\oplus_r, \otimes_r$, and hidden-null discrepancies ("off-ledger differences") cannot exist at the unquotiented root occurrence—arising only under dimension-dropping observer projections.

## 2.2 Source Processes and Fixed-Law Successor Rigidity (T0)

A **living current** on $N$ is a dependent triple $c = (V, \mathcal{R}, v)$ comprising a vocabulary $V$, a complete living root $\mathcal{R}$, and one temporal visit $v$ of that root; **erasure** $\mathrm{erase}(c)$ forgets only the pre-emitter continuation wrapper while keeping $V$, the authoritative root, and $v$.

**Definition 2.3 (Source process).** A **source process** $\mathcal{P} = (S, \iota, s_0, \sigma)$ on $N$ consists of:

- an arbitrary state type $S$ with no algebraic structure assumed;
- an injection $\iota : S \hookrightarrow \mathrm{Cur}(N)$ registering every state as a living current;
- an initial state $s_0 \in S$;
- an evidence-bearing one-tick generating successor $\sigma : S \to S$ such that $\mathrm{erase}(\iota(\sigma(s)))$ equals the next authoritative current compiled by the root of $\iota(s)$ at its visit, keeping the living root unchanged on non-terminal branches.

**Proposition 2.4 (Determinacy of fixed-law process successor and on-demand finite histories, T0).** Let $\mathcal{P} = (S, \iota, s_0, \sigma)$ be a source process. Inside the zero-axiom kernel:

1. $\mathrm{erase}(\iota(\sigma(s)))$ equals the generated next authoritative current of the root of $\iota(s)$;
2. the root law surface of $\iota(\sigma(s))$ equals the root law surface of $\iota(s)$;
3. if $\iota(s_1) = \iota(s_2)$, then $\iota(\sigma(s_1)) = \iota(\sigma(s_2))$—no parameter invisible to the complete registered current can silently fork the next world;
4. for any finite observation horizon $n \in \mathbb{N}$ and start $s \in S$, recursion generates a length-$n$ process history; exhausting the horizon ($n = 0$) merely halts observation without manufacturing a world terminal disposition.

Proof. (1) and (3) unfold Definition 2.3 directly, using injectivity $\iota(s_1) = \iota(s_2) \Rightarrow s_1 = s_2$ in (3). (2) applies the law-surface projection to (1) together with the root registration's law-surface preservation identity. (4) recurses structurally on $n \in \mathbb{N}$. ■

Proposition 2.4(2) draws a sharp architectural boundary: a fixed-law process can never smuggle a theory revision inside a local settlement or carrier handover; genuine vocabulary enlargement requires the explicit initial-coface construction of §8.

## 2.3 Free $\mathbb{Z}$-Module Carriers and Joint Lifts of Arbitrary Binary Functions (T1)

A native state may be a database snapshot, board position, molecular configuration, or prover state with no addition of its own. Assigning an independent basis element to every $s \in S$ and taking finitely supported integer combinations yields the free $\mathbb{Z}$-module (Z-set[1]) $\mathbb{Z}[S] := S \to_0 \mathbb{Z}$ with **state points** $\delta_s := 1 \cdot s$. Because $s \mapsto \delta_s$ is injective, distinct states never merge; the native successor $\sigma : S \to S$ extends linearly to $A : \mathbb{Z}[S] \to \mathbb{Z}[S]$ via $A\delta_s = \delta_{\sigma(s)}$.

**Proposition 2.5 (Unique free linear extension of arbitrary native observations).** For any additive abelian group $B$ and any native observation $\mathrm{read} : S \to B$, there exists a unique $\mathbb{Z}$-linear map $\widehat{\mathrm{read}} : \mathbb{Z}[S] \to B$ satisfying $\widehat{\mathrm{read}}(\delta_s) = \mathrm{read}(s)$, and it commutes naturally with the source action:

$$\widehat{\mathrm{read}} \circ A = \widehat{\mathrm{read} \circ \sigma}. \tag{2.2}$$

Proof. By the universal property of free modules: two linear maps agreeing on all basis points $\delta_s$ agree on every finitely supported sum, giving uniqueness, while linear combination gives existence. Both sides of (2.2) evaluate to $\mathrm{read}(\sigma(s))$ on $\delta_s$, requiring no autonomous dynamics on $B$. ■

When an operation consumes two inputs simultaneously—such as multiplication, table join, path concatenation, or operator pairing—their joint pairing must also be preserved. Given arbitrary types $X, Y, Z$ and $f : X \to Y \to Z$:

**Definition 2.6 (Joint free lift).** The **joint map** $\mathrm{joint} : \mathbb{Z}[X] \to \mathbb{Z}[Y] \to \mathbb{Z}[X \times Y]$ is the unique biadditive map with $\mathrm{joint}(\delta_x, \delta_y) = \delta_{(x,y)}$; the **pushforward** $\mathrm{push}_f : \mathbb{Z}[X \times Y] \to \mathbb{Z}[Z]$ is the free linear extension of $(x, y) \mapsto \delta_{f(x,y)}$; the **free bilinear lift** of $f$ is $\mathrm{lift}(f) := \mathrm{push}_f \circ \mathrm{joint}$.

While invoking the universal property of tensor products[5], Definition 2.6 separates **independent pairings** $\mathrm{joint}(u, v)$ from **already-correlated joint inventories** in $\mathbb{Z}[X \times Y]$: for $x \neq x'$ and $y \neq y'$, $\delta_{(x,y)} + \delta_{(x',y')}$ enters $\mathrm{push}_f$ directly without ever factoring as $\mathrm{joint}(u, v)$ (which would force cross terms $(x, y')$ and $(x', y)$). Section 7 operates precisely on such correlated joint inventories.

**Proposition 2.7 (Basis readback and complete three-term increment expansion).** Over the three-sort set $\{\mathrm{left}, \mathrm{right}, \mathrm{result}\}$ with carriers $\mathbb{Z}[X], \mathbb{Z}[Y], \mathbb{Z}[Z]$, basis environment $\rho^\delta$ mapping variables to $\delta_x, \delta_y$, and native expression $e_{f,x,y} := \mathrm{bilinear}_{\mathrm{lift}(f)}(\mathrm{var}\, x, \mathrm{var}\, y)$ (syntax in Definition 3.1):

1. $\mathrm{ev}_{\rho^\delta}(e_{f,x,y}) = \delta_{f(x,y)}$ and $\mathrm{lift}(f)(\delta_x, \delta_y) = \delta_{f(x,y)}$;
2. for every increment environment $\varepsilon = (\varepsilon_x, \varepsilon_y)$, the updated evaluation expands into four exact terms:

$$\mathrm{ev}_{\rho^\delta + \varepsilon}(e_{f,x,y}) = \delta_{f(x,y)} + \mathrm{lift}(f)(\delta_x, \varepsilon_y) + \mathrm{lift}(f)(\varepsilon_x, \delta_y) + \mathrm{lift}(f)(\varepsilon_x, \varepsilon_y). \tag{2.3}$$

Proof. (1) Unfold $\mathrm{push}_f(\mathrm{joint}(\delta_x, \delta_y)) = \mathrm{push}_f(\delta_{(x,y)}) = \delta_{f(x,y)}$. (2) Expand $\mathrm{lift}(f)(\delta_x + \varepsilon_x, \delta_y + \varepsilon_y)$ by biadditivity (specializing Proposition 3.3 below). ■

Crucially, (2.3) holds verbatim for string concatenation, hash lookup, or discrete combinatorics—requiring neither differentiability nor native linearity of $f$.

### 2.4 The Actual-Witness Principle: Runtime States Witness Readings Without Inversion

Call a one-tick transition together with the full registration of its occurrence, ledger, and next current a **tick**. In the one-sorted language with carrier $\mathbb{Z}[S]$ and $s_k := \sigma^k(s_0)$, the source word var and action word $\mathrm{linear}_A(\mathrm{var})$ read at stage $i$ as $(\delta_{s_i}, \delta_{s_{i+1}} - \delta_{s_i})$ and $(\delta_{s_{i+1}}, \delta_{s_{i+2}} - \delta_{s_{i+1}})$. The state point $\delta_{s_{i+1}}$ in the old coordinate of the action word is witnessed directly by the generated runtime state $\sigma^{i+1}(s_0)$; likewise, any native readout $h : S \to B$ (requiring no addition on $B$) is evaluated forward on $\sigma^{i+1}(s_0)$ and paired via a $\Sigma$-type with a proof that its basis image matches the free observation (Appendix A.1). This **actual-witness principle** governs the entire paper: state points and native readouts are always read forward from generated runtime states, never inverted backward from observed combinations.

## 3 Bilinear Three-Term Increments, Replayable Derivations, and Syntax-Only Left Inversion

This section expresses change structurally through an expression's **effect**, supplies an order- and multiplicity-preserving mixed-term decomposition alongside `Type`-valued replayable derivations

(C7), and refines normalization into step-charged execution traces so every rooted tree compiles into an executable whole-tree program with an evaluation-free syntactic left inverse (C23).

### 3.1 Many-Sorted Heterogeneous Expressions and Three-Term Effects

**Definition 3.1 (Heterogeneous algebraic expressions and evaluation).** Let $\mathcal{K}$ be a set of sorts, each $\kappa \in \mathcal{K}$ equipped with an additive abelian group $V(\kappa)$ and variable set $\mathrm{Var}(\kappa)$. **Expressions** $e \in \mathrm{Expr}(\kappa)$ are generated inductively by

$$e ::= \mathrm{var}\, x \mid \mathrm{const}\, c \mid e_1 + e_2 \mid \mathrm{linear}_h(e) \mid \mathrm{bilinear}_\mu(e_1, e_2),$$

where $h : V(\kappa_1) \to V(\kappa)$ is an additive homomorphism and $\mu : V(\kappa_1) \to V(\kappa_2) \to V(\kappa)$ is biadditive. Given an environment $\rho$, evaluation $\mathrm{ev}_\rho : \mathrm{Expr}(\kappa) \to V(\kappa)$ recurses structurally without quotienting or pre-normalizing syntax.

**Definition 3.2 (Structural effect).** Given old environment $\rho$ and increment $\varepsilon$, the **effect** $\mathrm{eff}_{\rho,\varepsilon}(e)$ is defined by $\mathrm{eff}_{\rho,\varepsilon}(\mathrm{var}\, x) = \varepsilon(x)$, $\mathrm{eff}_{\rho,\varepsilon}(\mathrm{const}\, c) = 0$, structural recursion on sums and linear maps, and all three action terms at bilinear nodes[1]:

$$\mathrm{eff}_{\rho,\varepsilon}(\mathrm{bilinear}_\mu(e_1, e_2)) := \mu(\mathrm{ev}_\rho e_1, \mathrm{eff}_{\rho,\varepsilon} e_2) + \mu(\mathrm{eff}_{\rho,\varepsilon} e_1, \mathrm{ev}_\rho e_2) + \mu(\mathrm{eff}_{\rho,\varepsilon} e_1, \mathrm{eff}_{\rho,\varepsilon} e_2). \tag{3.1}$$

**Proposition 3.3 (Update equation).** For every sort $\kappa$, expression $e \in \mathrm{Expr}(\kappa)$, and environments $\rho, \varepsilon$:

$$\mathrm{ev}_{\rho+\varepsilon}(e) = \mathrm{ev}_\rho(e) + \mathrm{eff}_{\rho,\varepsilon}(e). \tag{3.2}$$

Proof. Structural induction on $e$. Variables and constants are immediate; sums regroup four terms by abelian commutativity and associativity; linear nodes use additivity of $h$; bilinear nodes apply the induction hypothesis and expand $\mu(\mathrm{ev}_\rho e_1 + \mathrm{eff}_{\rho,\varepsilon} e_1,\ \mathrm{ev}_\rho e_2 + \mathrm{eff}_{\rho,\varepsilon} e_2)$ into $\mu(\mathrm{ev}_\rho e_1, \mathrm{ev}_\rho e_2)$ plus the exact three terms of (3.1). ■

### 3.2 Order- and Multiplicity-Preserving Mixed-Term Lists

Because a summed effect can vanish through cancellation despite nonzero internal activity, we retain an itemized list preserving syntactic order and multiplicity.

**Definition 3.4 (Ordered mixed-term list).** The **mixed-term list** $\mathrm{mix}_{\rho,\varepsilon}(e) \in \mathrm{List}(V(\kappa))$ maps variables to $[\varepsilon(x)]$, constants to $[\,]$, sums to $\mathrm{mix}(e_1) +\!+ \mathrm{mix}(e_2)$, linear nodes to $h$ mapped across $\mathrm{mix}(e)$, and bilinear nodes to the concatenation of $[y \mapsto \mu(\mathrm{ev}_\rho e_1, y)](\mathrm{mix}(e_2))$, $[x \mapsto \mu(x, \mathrm{ev}_\rho e_2)](\mathrm{mix}(e_1))$, and $[(x, y) \mapsto \mu(x, y)](\mathrm{mix}(e_1) \times_{\mathrm{list}} \mathrm{mix}(e_2))$.

**Proposition 3.5 (Exact summation of mixed-term list).** For every expression $e$ and environments $\rho, \varepsilon$, $\sum \mathrm{mix}_{\rho,\varepsilon}(e) = \mathrm{eff}_{\rho,\varepsilon}(e)$.

Proof. Structural induction on $e$. The first four cases follow from list-sum additivity and homomorphism preservation. At bilinear nodes, biadditivity pulls $\mathrm{ev}_\rho e_1$ out of the first segment,

$\mathrm{ev}_\rho e_2$ out of the second, and list induction on $\mathrm{mix}(e_1)$ equates the Cartesian-product sum to $\mu(\sum \mathrm{mix}(e_1), \sum \mathrm{mix}(e_2))$, matching (3.1). ■

### 3.3 Replayable Derivations in `Type`: Normalization and Semantic Fibres

While effects measure change, **derivations** record the concrete path witnessing equality. Fixing $\rho$, a derivation $d : e \vdash_\rho e'$ is an inductive object in `Type` (rather than proof-irrelevant `Prop`) generated by reflexivity, symmetry, transitivity, variable binding ($\mathrm{var}\, x \vdash_\rho \mathrm{const}(\rho(x))$), three constant-contraction rules (constant sum, linear constant, bilinear constant), and three congruence rules. Inhabiting `Type` allows downstream modules to substitute, splice, and extract step boundaries directly.

**Proposition 3.6 (Normalization and characterization of semantic fibres).** For every environment $\rho$ and expressions $e, e'$ of the same sort:

1. **Soundness**: if $d : e \vdash_\rho e'$, then $\mathrm{ev}_\rho(e) = \mathrm{ev}_\rho(e')$;
2. **Endogenous normalization**: every $e$ recursively generates $\mathrm{norm}_\rho(e) : e \vdash_\rho \mathrm{const}(\mathrm{ev}_\rho(e))$ with no external normalizer or semantic-equality input;
3. **Semantic-fibre characterization**: $(e \vdash_\rho e')$ is inhabited iff $\mathrm{ev}_\rho(e) = \mathrm{ev}_\rho(e')$.

Proof. (1) Induction on $d$; each constant rule holds definitionally. (2) Structural recursion reduces subexpressions via congruence and applies the node's constant rule. (3) "Only if" is (1); "if" composes $\mathrm{norm}_\rho(e)$, constant transport along $\mathrm{ev}_\rho(e) = \mathrm{ev}_\rho(e')$, and $(\mathrm{norm}_\rho(e'))^{-1}$. ■

### 3.4 Whole-Tree Programs: Syntactic Charging and Evaluation-Free Left Inversion

Derivations allow symmetric steps, whereas actual execution charges one irreversible primitive reduction at a time. Define the **remaining primitive count** $r(e) \in \mathbb{N}$ by

$$r(\mathrm{var}\, x) = 1, \quad r(\mathrm{const}\, c) = 0, \quad r(\mathrm{linear}_h e) = r(e) + 1,$$
$$r(e_1 + e_2) = r(e_1) + r(e_2) + 1, \quad r(\mathrm{bilinear}_\mu(e_1, e_2)) = r(e_1) + r(e_2) + 1.$$

An **execution step** applies one primitive reduction (variable binding or one of the three constant contractions) at a syntactic congruence position; finitely many composable steps form an **execution trace**.

**Proposition 3.7 (Step-charged execution traces).** For every environment $\rho$ and expression $e$:

1. every execution step decreases the remaining count by exactly 1 ($r(e'') = r(e') - 1$); constants admit no step, and the step relation is well-founded;
2. left-to-right postorder traversal yields a canonical execution trace $\mathrm{exec}_\rho(e) : e \to \mathrm{const}(\mathrm{ev}_\rho e)$ of exact length $|\mathrm{exec}_\rho(e)| = r(e)$;
3. every execution trace lifts canonically to a derivation in `Type` whose step relation images telescope to $e - \mathrm{const}(\mathrm{ev}_\rho e)$.

Proof. (1) Induction on the step relation: root primitive rules drop 1 to 0, congruence rules drop the active subexpression by 1 while fixing siblings, and pullback of $(\mathbb{N}, <)$ gives well-foundedness. (2)

Structural recursion concatenates left trace, right trace, and root contraction, summing lengths to $r(e)$. (3) Each step is a generating derivation rule and concatenation telescopes endpoint differences. ■

Feeding the single-constructor rooted trees of §2.1 into this step-charged language yields our sharpest separation between readings and source objects.

**Definition 3.8 (Rooted trees and whole-tree programs).** Given root set Root, target carrier $C$, and any **constructor function** $\kappa : \mathrm{Root} \to \mathrm{List}(C) \to C$, write $\mathrm{fold}_\kappa(T) = \kappa(a, [\mathrm{fold}_\kappa T_1, \dots, \mathrm{fold}_\kappa T_k])$ for $T = \mathrm{occ}(a; [T_1, \dots, T_k])$ (abbreviating the single constructor occur as occ). Over sorts $\{\mathrm{root}, \mathrm{result}, \mathrm{children}\}$ with carriers $\mathbb{Z}[\mathrm{Root}], \mathbb{Z}[C], \mathbb{Z}[\mathrm{List}(C)]$ (variables only in root, basis environment $\delta(a) = \delta_a$), define mutually the **whole-tree program** $\mathrm{prog}_\kappa(T)$, **children program** $\mathrm{kids}_\kappa(\vec{T})$, and **syntactic tree budget** $\beta(T)$:

$$\begin{gathered} \mathrm{prog}_\kappa(\mathrm{occ}(a; \vec{T})) := \mathrm{bilinear}_{\mathrm{lift}(\kappa)}(\mathrm{var}\, a,\ \mathrm{kids}_\kappa(\vec{T})), \\ \mathrm{kids}_\kappa([\,]) := \mathrm{const}(\delta_{[\,]}), \qquad \mathrm{kids}_\kappa(T :: \vec{T}) := \mathrm{bilinear}_{\mathrm{lift}(\mathrm{cons})}(\mathrm{prog}_\kappa T,\ \mathrm{kids}_\kappa(\vec{T})), \\ \beta(\mathrm{occ}(a; \vec{T})) := 2 + \beta^*(\vec{T}), \qquad \beta^*([\,]) := 0, \qquad \beta^*(T :: \vec{T}) := \beta(T) + \beta^*(\vec{T}) + 1. \end{gathered} \tag{3.3}$$

**Theorem 3.9 (Whole-tree execution, syntactic charging, and syntax-only left inverse).** For every root set Root, carrier $C$, constructor function $\kappa : \mathrm{Root} \to \mathrm{List}(C) \to C$, and rooted tree $T$:

1. **Exact evaluation**: $\mathrm{ev}_\delta(\mathrm{prog}_\kappa T) = \delta_{\mathrm{fold}_\kappa T}$;
2. **Syntactic charging**: $r(\mathrm{prog}_\kappa T) = \beta(T)$, so $\mathrm{exec}_\delta(\mathrm{prog}_\kappa T)$ has exact length $\beta(T)$ depending solely on tree shape and independent of $\kappa$, with endpoint difference $\mathrm{prog}_\kappa T - \mathrm{const}(\delta_{\mathrm{fold}_\kappa T})$;
3. **Evaluation-free syntactic left inverse**: a partial parser $\mathrm{read} : \mathrm{Expr}(\mathrm{result}) \rightharpoonup \mathcal{U}(\mathrm{Root})$ inspecting syntax and sort tags alone—never invoking $\kappa$ or any evaluator—satisfies $\mathrm{read}(\mathrm{prog}_\kappa T) = T$; hence $\mathrm{prog}_\kappa$ is injective for every $\kappa$;
4. **All-zero collapse control**: taking $C = \mathbb{Z}$ and $\kappa \equiv 0$, **every** rooted tree evaluates to the identical state point $\delta_0$, rendering all semantic outputs indistinguishable; yet $\mathrm{prog}_0$ remains injective and read recovers every original tree intact.

Proof. Mutual structural induction on $T$ and $\vec{T}$. (1) Proposition 2.7(1) gives $\mathrm{lift}(\kappa)(\delta_a, \delta_{\vec{c}}) = \delta_{\kappa(a,\vec{c})}$ and $\mathrm{lift}(\mathrm{cons})(\delta_c, \delta_{\vec{c}}) = \delta_{c::\vec{c}}$, combining with the induction hypothesis. (2) Each tree node contributes 1 variable plus 1 bilinear node (+2), empty lists contribute 0, and each `cons` node contributes +1, matching $\beta, \beta^*$. (3) On sort result, read accepts bilinear nodes whose left sort is root with $\mathrm{var}\, a$ and right sort is children; on children it parses constants as $[\,]$ and bilinear nodes with left sort result as list extensions, never inspecting $\mathrm{lift}(\kappa)$. Mutual induction yields $\mathrm{read}(\mathrm{prog}_\kappa T) = T$, forcing injectivity. (4) Specializes (1) and (3) to $\kappa \equiv 0$. ■

Theorem 3.9(4) mirrors the opening three-tick counterexample in reverse: in Example 4.1 readings agree while source words differ; here every tree evaluates to $\delta_0$, yet the source program retains and returns the complete generative tree.

# 4 The Exact Evaluation Kernel and Inverse-Limit History Completions

Answering multi-tick identity requires solving two algebraic problems: characterizing all valid equality certificates at a single tick (§4.2), and assembling compatible finite prefixes into a completion carrier with a global evolution law (§4.3–§4.4). Throughout this section, $R$ is an arbitrary commutative ring and each $V(\kappa)$ is an $R$-module.

## 4.1 Two Exact Counterexamples: Local Splicing Failure and Cross-Tick Relation Failure

Over a single sort, value module $\mathbb{Z}$, and variable $x$, the evaluations of candidate words $0$, $x$, and $x^2 - x$ across environments are collected in Table 1.

**Example 4.1 (Local witnesses cannot be spliced into a common source history).** Along $x \mapsto 1 \to x \mapsto 0 \to x \mapsto 1$: (1) on $1 \to 0$, $w = 0$ realizes $0 \to 0$; (2) on $0 \to 1$, $w = x$ realizes $0 \to 1$; (3) both evaluate to 0 at the shared environment $x \mapsto 0$; (4) yet no formal source word $w$ satisfies $\mathrm{ev}_1(w) = 0$, $\mathrm{ev}_0(w) = 0$, and $\mathrm{ev}_1(w) = 1$, since the first and third equations force $0 = 1$ in $\mathbb{Z}$ (Appendix B.2).

**Example 4.2 (An old-stage relation fails after an update).** Under $x \mapsto 1$, $\mathrm{ev}_1(x^2 - x) = 1 - 1 = 0$ so $x^2 - x \in \ker(\mathrm{ev}_1)$; after updating to $x \mapsto 2$, $\mathrm{ev}_2(x^2 - x) = 4 - 2 = 2 \neq 0$, whence $\ker(\mathrm{ev}_1) \not\subseteq \ker(\mathrm{ev}_2)$. Under $x \mapsto 1$, $x^2 - x$ and $0$ read identically, separating into 2 and 0 under $x \mapsto 2$.

**Table 1 Readings of candidate words across environments.** Rows list candidate source words; columns list environment assignments.

| Candidate word \ Environment | $x \mapsto 1$ | $x \mapsto 0$ | $x \mapsto 2$ |
|---|---|---|---|
| $0$ | $0^{\otimes,\dagger}$ | $0^{\dagger}$ | $0^{\ddagger}$ |
| $x$ | $1^{\otimes,\dagger}$ | $0^{\dagger}$ | $2$ |
| $x^2 - x$ | $0$ | $0$ | $2^{\ddagger}$ |

Here $\otimes$ marks the contradiction of Example 4.1 requiring a single row to hit both 0 and 1 in column $x \mapsto 1$; † marks the two locally valid segments agreeing at $x \mapsto 0$; and ‡ marks the relation breakdown of Example 4.2 between $x \mapsto 1$ and $x \mapsto 2$.

## 4.2 Two-Certificate Exactness of the Evaluation Kernel

**Definition 4.3 (Formal words and two-family relation indices).** A **formal word** (called a **source word** when witnessing a history) of sort $\kappa$ is a finite $R$-linear combination $\mathrm{Fm}_R(\kappa) := \mathrm{Expr}(\kappa) \to_0 R$, with linear evaluation $\mathrm{ev}_\rho : \mathrm{Fm}_R(\kappa) \to V(\kappa)$. Fixing $\rho$, the **relation-index type** is the disjoint union

$$\mathrm{RelIdx}(\rho, \kappa) := \Big( \sum_{e, e' \in \mathrm{Expr}(\kappa)} (e \vdash_\rho e') \Big) \sqcup \mathrm{SclIdx}(\kappa),$$

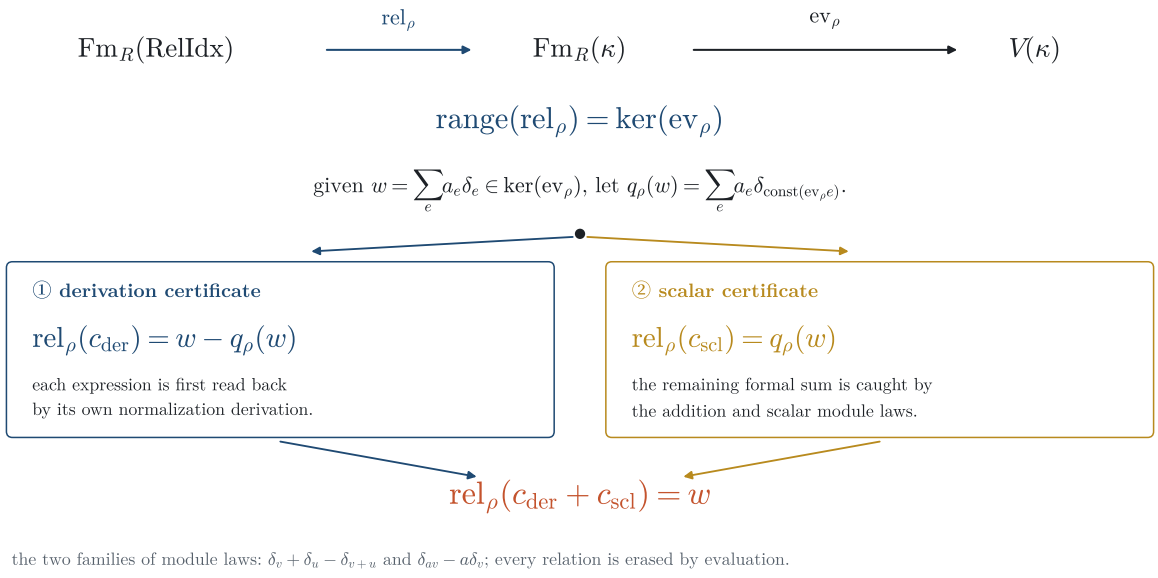
where the left summand (**derivation evidence**) maps $(e, e', d)$ to $\delta_e - \delta_{e'}$ and the right summand (**scalar module-law evidence**) comprises addition relations $(v, v') \mapsto \delta_{\mathrm{const}(v)} +$

$\delta_{\mathrm{const}(v')} - \delta_{\mathrm{const}(v+v')}$ and scalar-action relations $(r, v) \mapsto \delta_{\mathrm{const}(r\cdot v)} - r\,\delta_{\mathrm{const}(v)}$. Extending $R$-linearly defines the **relation map** $\mathrm{rel}_\rho : \mathrm{Fm}_R(\mathrm{RelIdx}(\rho, \kappa)) \to \mathrm{Fm}_R(\kappa)$.

**Theorem 4.4 (Exact evaluation kernel: relation range equals evaluation kernel).** Over any commutative ring $R$, any $R$-module value carriers $V$, any environment $\rho$, and every sort $\kappa$, the image of the relation map equals the evaluation kernel:

$$\mathrm{range}(\mathrm{rel}_\rho) = \ker(\mathrm{ev}_\rho), \qquad \mathrm{Fm}_R(\mathrm{RelIdx}(\rho,\kappa)) \xrightarrow{\mathrm{rel}_\rho} \mathrm{Fm}_R(\kappa) \xrightarrow{\mathrm{ev}_\rho} V(\kappa) \longrightarrow 0 \text{ is exact.} \quad (4.1)$$

Proof. Inclusion $\mathrm{range}(\mathrm{rel}_\rho) \subseteq \ker(\mathrm{ev}_\rho)$ checks generators: derivation endpoints evaluate equally by Proposition 3.6(1), and module-law generators evaluate to 0. Conversely, let $w = \sum_i a_i \delta_{e_i} \in \ker(\mathrm{ev}_\rho)$, put $v_i = \mathrm{ev}_\rho(e_i)$, and form the constant formal sum $q = \sum_i a_i \delta_{\mathrm{const}(v_i)}$ (requiring no nonconstructive choice of derivation witnesses, since Proposition 3.6(2) constructs $\mathrm{norm}_\rho(e_i)$ directly in `Type`; full details in Appendix A.3 and Figure S1). First, combining endogenous normalizations $\mathrm{norm}_\rho(e_i) : e_i \vdash_\rho \mathrm{const}(v_i)$ with weights $a_i$ constructs a **derivation certificate** $c_{\mathrm{der}}$ satisfying $\mathrm{rel}_\rho(c_{\mathrm{der}}) = w - q$. Second, since $\sum_i a_i v_i = \mathrm{ev}_\rho(w) = 0$, $q_0 = \sum_i a_i \delta_{v_i} \in R[V(\kappa)]$ lies in the free evaluation kernel; because $R[V(\kappa)]$ modulo addition and scalar-action relations is $R$-linearly isomorphic to $V(\kappa)$ via $[x] \mapsto \mathrm{eval}(x)$ and $v \mapsto [\delta_v]$, $q_0$ vanishes in the quotient and yields a **scalar certificate** $c_{\mathrm{scl}}$ with $\mathrm{rel}_\rho(c_{\mathrm{scl}}) = q$. Summing gives $\mathrm{rel}_\rho(c_{\mathrm{der}} + c_{\mathrm{scl}}) = (w - q) + q = w$, and $\delta_{\mathrm{const}(v)}$ witnesses surjectivity of $\mathrm{ev}_\rho$. ■




**Caption**. Figure S1 | Two-certificate exact decomposition of the evaluation kernel (proof-structure diagram; Theorem 4.4, Appendix A.3). Let $q_\rho(w)$ denote the constant formal sum after expression-by-expression evaluation. Normalization derivations yield relation image $w - q_\rho(w)$; when $\mathrm{ev}_\rho(w) = 0$, the module-law relations supply a scalar certificate for the remaining $q_\rho(w)$, whose sum (vermilion) has relation image $w$; the forward inclusion evaluates each generating relation to zero.

Notice that every execution trace $t : e \to \mathrm{const}(\mathrm{ev}_\rho e)$ of Proposition 3.7 unpacks the derivation certificate step by step into $\delta_e - \delta_{\mathrm{const}(\mathrm{ev}_\rho e)} \in \ker(\mathrm{ev}_\rho)$; under any increment $\varepsilon$, its evaluation in $\rho + \varepsilon$ equals $\mathrm{eff}_{\rho,\varepsilon}(e)$, serving as the algebraic pivot turning paid receipts into subsequent inquiries in §§6.5, 8.5, and 10.1.

### 4.3 Inverse-Limit Completion Carrier and Elementwise Evolution

Fix a source process $\mathcal{P} = (S, \iota, s_0, \sigma)$, additive material group $M$, readout $\mathrm{mat} : S \to M$, and additive projection $\pi : M \to \mathrm{Env}$. For start $s$ and tick $i \in \mathbb{N}$, set $s_i := \sigma^i(s)$, $\rho_i := \pi(\mathrm{mat}(s_i))$, and $\varepsilon_i := \rho_{i+1} - \rho_i$.

**Definition 4.5 (Stage inventory, prefix tower, and completion carrier).** The **stage inventory** of $w \in \mathrm{Fm}_R(\kappa)$ at tick $i$ is the joint pair

$$U_i^s(w) := \left(\mathrm{ev}_{\rho_i}(w),\ \mathrm{eff}_{\rho_i,\varepsilon_i}(w)\right) \in V(\kappa) \times V(\kappa). \tag{4.2}$$

For horizon $b \in \mathbb{N}$, the length-$(b+1)$ **prefix reading** is $\mathrm{pre}_b^s(w) := (U_0^s(w), \dots, U_b^s(w))$, with **stage quotient** $Q_b(s) := \mathrm{Fm}_R(\kappa)/\ker(\mathrm{pre}_b^s)$. Dropping the last tick $U_{b+1}^s$ defines fixed-start **prefix restrictions** $r_b^s : Q_{b+1}(s) \to Q_b(s)$, whose inverse limit is the **history completion carrier** $\widehat{C}(s) := \varprojlim_b Q_b(s)$ with projections $\mathrm{pr}_b^s : \widehat{C}(s) \to Q_b(s)$ and canonical map $w \mapsto [w]_s$. Conversely, dropping the first tick $U_0^s$ defines **start-advance maps** $d_b^s : Q_{b+1}(s) \to Q_b(\sigma(s))$; because $r_b^{\sigma(s)} \circ d_{b+1}^s = d_b^s \circ r_{b+1}^s$, they induce a unique $R$-linear **completion successor** $T : \widehat{C}(s) \to \widehat{C}(\sigma(s))$ satisfying $\mathrm{pr}_b^{\sigma(s)} \circ T = d_b^s \circ \mathrm{pr}_{b+1}^s$ and $T([w]_s) = [w]_{\sigma(s)}$ (Figure 3).

**Theorem 4.6 (Tick-wise relation characterization of completion fibres).** Two source words $u, v \in \mathrm{Fm}_R(\kappa)$ map to the same completion element ($[u]_s = [v]_s$) iff at **every tick** $i \in \mathbb{N}$ the pair-lifted word $\widetilde{u - v}$ (Appendix A.2) lies in that tick's paired relation range:

$$[u]_s = [v]_s \quad \Longleftrightarrow \quad \forall i \in \mathbb{N},\ \ \widetilde{u - v} \in \mathrm{range}\big(\mathrm{rel}_{(\rho_i,\varepsilon_i)}\big). \tag{4.3}$$

Proof. Equality in the inverse limit holds iff projections agree in every $Q_b(s)$, i.e., $U_i^s(u - v) = (0, 0)$ for all $i \geq 0$. Since $\mathrm{ev}_{(\rho_i,\varepsilon_i)}(\widetilde{w}) = U_i^s(w)$ (Appendix A.2), this is equivalent to $\widetilde{u - v} \in \ker(\mathrm{ev}_{(\rho_i,\varepsilon_i)})$, to which Theorem 4.4 applies at each tick, yielding (4.3). ■

**Theorem 4.7 (Elementwise evolution equation on the completion carrier).** For **every** element $z \in \widehat{C}(s)$ (not merely canonical images of single source words) and every tick $i$ of every finite stage:

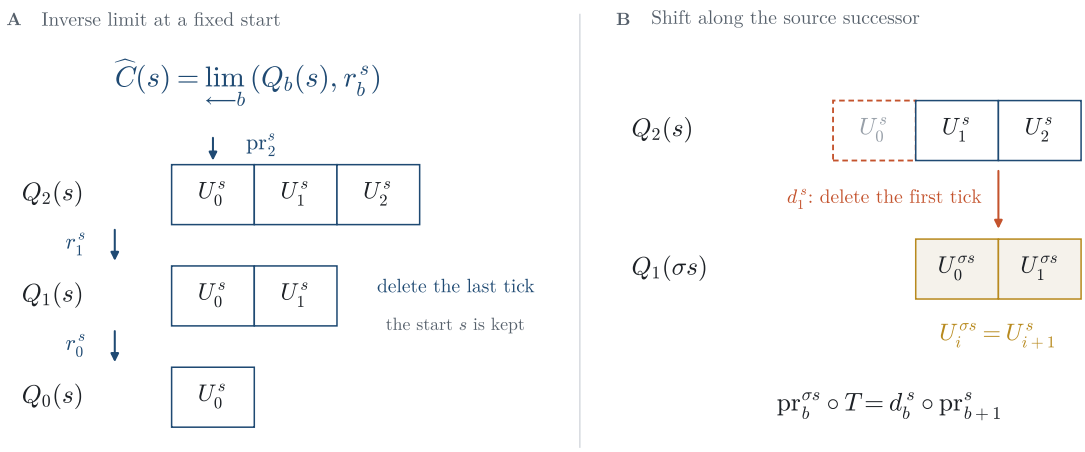
$$\mathrm{old}_i(T(z)) = \mathrm{old}_i(z) + \mathrm{eff}_i(z), \tag{4.4}$$

where $\mathrm{old}_i, \mathrm{eff}_i$ extract the first and second coordinates of tick $i$.

Proof. On any source word $w$, Proposition 3.3 with $\rho_i + \varepsilon_i = \rho_{i+1}$ gives $\mathrm{ev}_{\rho_{i+1}}(w) = \mathrm{ev}_{\rho_i}(w) + \mathrm{eff}_{\rho_i,\varepsilon_i}(w)$. Fixing tick $i$, surjectivity of $\mathrm{Fm}_R(\kappa) \twoheadrightarrow Q_{i+1}(s)$ provides a finite-stage representative $w_{(i+1)}$ for $\mathrm{pr}_{i+1}^s(z)$ (never choosing a single word across all infinite stages); reading $\mathrm{pr}_i^{\sigma(s)}(Tz) = d_i^s(\mathrm{pr}_{i+1}^s(z))$ evaluates $\mathrm{old}_i(Tz)$ as $\mathrm{ev}_{\rho_{i+1}}(w_{(i+1)})$, yielding (4.4) (Appendix B.1). ■

### 4.4 Prefix Extensions Preserving the Common Source Witness

**Proposition 4.8 (Prefix witness extension keeping the common source word).** Fixing a starting depth and horizon $b$, if source word $w$ witnesses $p = \mathrm{pre}_b^s(w)$, then the **same word** $w$ directly supplies the $(b+1)$-horizon extension $p^+ = \mathrm{pre}_{b+1}^s(w)$: dropping the new last tick restricts




**Caption**. Figure 3 | Prefix truncation and start advance (structural diagram; Definition 4.5, Theorems 4.6–4.7). A: fixed-start restriction $r_b^s : Q_{b+1}(s) \to Q_b(s)$ deletes the last tick, and the completion carrier $\widehat{C}(s)$ is the inverse limit along these restrictions. B: start advance $d_b^s : Q_{b+1}(s) \to Q_b(\sigma(s))$ deletes the first tick (vermilion dashed cell) and advances the start to induce the linear successor $T$; the two maps act on opposite ends of finite windows, and the evolution equation of Theorem 4.7 holds for every $z \in \widehat{C}(s)$ without assuming a single representing source word.

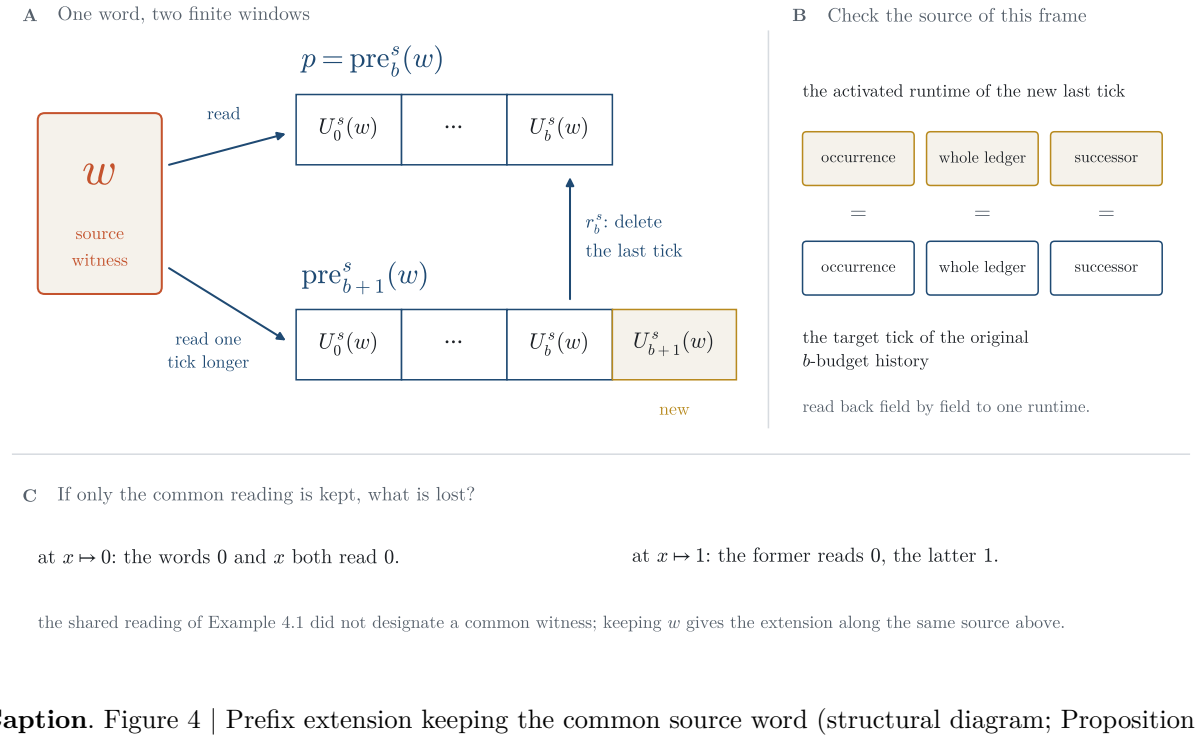
back to $p$, and the activated runtime at the new last tick equals the target runtime of the $b$-horizon history, with zero witness selection from bare readings. ■

Proof. Evaluating one further tick with the same $w$ restricts back to $p$ definitionally, and its last-tick runtime is $\sigma^{b+1}(s)$, verified field by field with payload, particle-wave readout, whole-ledger write-back, and next current in the Fock runtime (Appendix D.1). This settles the opening counterexample completely: local junction readings never substitute for a shared source witness (Figure 4).

## 5 Faithful Process Realizations and Unique Comparison Without Preset Simulation Relations

While a common source word anchors history identity inside one carrier, two implementations using different host languages, event encodings, or compilation strategies must also identify which event realizes which underlying occurrence (Tier 3 identity). Classical refinement mappings[7] map directly between implementation state spaces, generally requiring auxiliary history variables and future-looking prophecy variables. Working inside the **T0 zero-axiom kernel** (`#print axioms = []`), we take the opposite route: each implementation establishes a two-sided faithful representation with the causal world $W$ of the fixed root, and—without direct communication or auxiliary variables—the shared occurrence uniquely determines the event comparison and preserves whole-ledger compilation (C10).

An **execution process** $P$ assigns to every current $c$ an event carrier $\mathrm{Evt}_P(c)$, an emitted event $\mathrm{emit}_P(c) \in \mathrm{Evt}_P(c)$, and a compiler $\mathrm{compile}_P$ returning candidate whole evolutions. Comparison




**Caption**. Figure 4 | Prefix extension keeping the common source word (structural diagram; Proposition 4.8). Given $p = \mathrm{pre}^s_b(w)$ and source witness $w$, reading $\mathrm{pre}^s_{b+1}(w)$ with the same $w$ extends the prefix so that dropping the new last tick recovers $p$, and the activated runtime at the new last tick equals the target tick of the original history. Vermilion marks the preserved witness $w$; the lower panel recalls Example 4.1, where equal middle readings never designated a shared witness.

is anchored to the causal world $W$ of §2, where each visit has a unique canonical occurrence $g_c \in \mathrm{Occ}_W(c)$ and obeys uniqueness of lawful evolutions.

**Definition 5.1 (Faithful process realization).** An execution process $P$ **faithfully realizes** $W$ if it supplies three zero-axiom constructive components:

- **Two-sided occurrence–event representation**: at each current $c$, maps $\varphi_c : \mathrm{Occ}_W(c) \rightleftarrows \mathrm{Evt}_P(c) : \varphi_c^{-1}$ satisfying $\varphi_c^{-1} \circ \varphi_c = \mathrm{id}$ and $\varphi_c \circ \varphi_c^{-1} = \mathrm{id}$ without choice or quotients;
- **Emitter compatibility**: $\varphi_c(g_c) = \mathrm{emit}_P(c)$ at every current $c$;
- **Compilation legality**: for every $g \in \mathrm{Occ}_W(c)$, $\mathrm{compile}_P(\varphi_c(g))$ belongs to $\mathrm{Lawful}_c(g, \cdot)$.

**Proposition 5.2 (Event-by-event complete recovery, T0).** Let $P$ faithfully realize $W$, let $v$ be any temporal visit, and let $e \in \mathrm{Evt}_P(v)$ be **any** event in that visit's fibre (not assumed in advance to be the emitted event). Inside the zero-axiom kernel:

1. **Same occurrence**: $\varphi_v^{-1}(e) = g_v$ equals pointwise the canonical occurrence generated by the root at $v$;
2. **Event equals emitted event**: $e = \mathrm{emit}_P(v)$, collapsing the single-visit event fibre to the emitted event (while event families across distinct visits may remain arbitrarily rich);
3. **All installed projections agree**: for **every** projection $\chi$ in the source projection law of $W$, evaluating $\chi$ at $\varphi_v^{-1}(e)$ equals heterogeneously the root's own projection at $v$;
4. **Native successor agrees**: the next current compiled from $e$ by $P$ equals the next current generated by the root at $v$.

Proof. Since $\varphi_v^{-1}(e) \in \mathrm{Occ}_W(v)$, singleton visit-occurrence uniqueness gives $\varphi_v^{-1}(e) = g_v$,

proving (1). Applying $\varphi_v$ and using the right inverse plus emitter compatibility gives $e = \varphi_v(\varphi_v^{-1}(e)) = \varphi_v(g_v) = \mathrm{emit}_P(v)$, proving (2). For (3), recovering the exact occurrence $g_v$ first and projecting afterwards transports $\chi$ along $\varphi_v^{-1}(e) = g_v$ and composes with Theorem 2.2(3) across all installed projections. Item (4) reads off the fixed next current $\mathrm{next}(g_v)$ of $W$. ■

**Theorem 5.3 (Unique comparison without preset simulation relations, T0).** Let $P_1, P_2$ both faithfully realize the same causal world $W$. Inside the zero-axiom kernel:

1. **Source-generated comparison and whole-compiler preservation**: the composite $\Phi_{12,c} := \varphi_{2,c} \circ \varphi_{1,c}^{-1} : \mathrm{Evt}_{P_1}(c) \rightleftarrows \mathrm{Evt}_{P_2}(c)$ is a two-sided event representation compatible with both emitters and with **the entire compiler**: for every $e \in \mathrm{Evt}_{P_1}(c)$, $\mathrm{compile}_{P_2}(\Phi_{12,c}(e)) = \mathrm{compile}_{P_1}(e)$, so the whole-ledger evolutions generated by $P_1$ and $P_2$ coincide at every current;
2. **Unconditional pointwise uniqueness**: **every** candidate event map $\Psi_c : \mathrm{Evt}_{P_1}(c) \to \mathrm{Evt}_{P_2}(c)$ (**with no emitter-compatibility, homomorphism, or commuting-diagram hypothesis attached**) satisfies $\Psi_c(e) = \Phi_{12,c}(e)$ pointwise at every current $c$ and event $e$.

Proof. (1) Two-sided inversion and emitter compatibility compose $\varphi_1, \varphi_2$ directly. For any $e \in \mathrm{Evt}_{P_1}(c)$, setting $g = \varphi_{1,c}^{-1}(e)$ places both $\mathrm{compile}_{P_1}(e) = \mathrm{compile}_{P_1}(\varphi_{1,c}(g))$ and $\mathrm{compile}_{P_2}(\Phi_{12,c}(e)) = \mathrm{compile}_{P_2}(\varphi_{2,c}(g))$ inside $\mathrm{Lawful}_c(g, \cdot)$, where uniqueness of lawful evolutions forces equality. (2) Both $\Psi_c(e)$ and $\Phi_{12,c}(e)$ inhabit the target fibre $\mathrm{Evt}_{P_2}(c)$, so Proposition 5.2(2) equates both to $\mathrm{emit}_{P_2}(c)$. ■

Singleton occurrence fibres enforce unconditional pointwise event uniqueness, while uniqueness of lawful evolutions enforces whole-compiler agreement. The unified-field construction invokes Theorem 5.3 directly to align configuration admission, matter readouts, action equations, and macro successors across representations (§10.4).

## 6 The Responsibility Ledger: Lifecycle Conservation, Law-Free Well-Foundedness, and Bearer-Free Step Payment

An unanswered demand entering a run leaves a live **obligation (responsibility)** in the process's own ledger. The **whole live ledger** at support $s$ is the witness-bearing dependent sum

$$L_s := \sum_{r:\mathrm{Responsibility}} \mathrm{OpenAt}(s, r),$$

from each row of which a claim and natural-number progress budget $\beta$ are read. A lawful whole-ledger evolution assigns an origin to every target row and a destination to every source row under the write rule of a single occurrence; a finite patch lists only rows **generated at this step**, keeping unchanged old rows as **same-source identity remainders**. We establish in turn the three lifecycle conservation laws (§6.1, C1), debt activation and event-indexed compilation (§6.2–§6.3, C12), law-free macro well-foundedness and master-law finite settlement (§6.4, C2), and bearer-free computational obligation birth, strict per-step debit, and `Type`-valued completion receipts (§6.5, C22, C39).

### 6.1 Three Lifecycle Laws: Conservation Across the Edge Grammar

Fix a responsibility vocabulary $\mathcal{V}$. States of a **native responsibility process** are lists of slots, each either **live** or a **terminal archive**. Legal edges between states take exactly three forms: (1) **admission**, appending one live slot at the end; (2) **boundary crossing**, applying a boundary event to a live slot whose classifier outputs one of four dispositions—carry, holder release, inherited terminal, or final-obligation terminal; and (3) **reopening**, applying a reopening event to a terminal archive to make it live again.

**Proposition 6.1 (Three responsibility lifecycle laws and order-preserving history).** For every vocabulary $\mathcal{V}$, native responsibility process, legal edge $s \to t$, and slot $k$:

1. **No free generation**: a freshly allocated live slot ($s$ has no slot $k$ and $t_k$ is live) arises solely from an admission edge;
2. **Reopening is the sole revival path**: a newly live slot ($s_k$ non-live and $t_k$ live) arises solely from admission or reopening—boundary crossing never revives an archived slot;
3. **No silent disappearance**: disappearance of a live slot ($s_k$ live and $t_k$ non-live) arises solely from the final-obligation terminal branch at slot $k$; carry, holder release, and inherited terminal all leave a live successor in the same lineage slot;
4. **Order-preserving history**: the target-state history of any finite run equals the source history concatenated with the edge events in order; folding from the empty lifecycle yields the exact generating event list.

Proof. Inspect how each edge modifies the slot list: admission appends at the end while preserving old slots; boundary crossing and reopening update only index $k$. Among the four boundary-crossing branches, carry, holder release, and inherited terminal all write live payloads, leaving final-obligation terminal as the sole source of non-live slots. Item (4) follows by induction on run length. ■

### 6.2 Debt Activation Laws: Extended Worlds and Two-Sided Readback

**Definition 6.2 (Debt activation law).** A **debt activation law** $\Lambda$ supplies seven dependent components (optional families default to empty): (1) **debt identity**—state set $D$, fixed label $\mathrm{id}_0$, and fixed claim $\mathrm{cl}_0$; (2) **budget** $\beta : D \to \mathbb{N}$; (3) **strict payment steps** $\mathrm{Step}(d, d') :$ Type carrying $\beta(d') < \beta(d)$; (4) **same-debt transport** with $\beta(d') \leq \beta(d)$; (5) **local settlement witnesses** Settle, inhabited only where $\beta(d) = 0$; (6) **whole-support phase-terminal events**, each containing a local settlement; and (7) **state obstructions** $\mathrm{Obs} : D \to$ Type.

**Definition 6.3 (Extended world network).** Given $\Lambda$, $N[\Lambda]$ sets supports to $\mathrm{Sup} \times \mathrm{Option}(D)$, responsibilities to $\mathrm{Responsibility} \sqcup \mathrm{DebtId}$, and claims to $\mathrm{Claim} \sqcup \mathrm{DebtClaim}$; the new debt fibre is empty over $\langle s, \mathrm{none} \rangle$ and opens over $\langle s, \mathrm{some}\, d \rangle$ uniquely at label $\mathrm{id}_0$ with claim $\mathrm{inr}\, \mathrm{cl}_0$ and budget $\beta(d)$, extending dispositions across payment, phase termination, and obstruction.

**Proposition 6.4 (Two-sided readback of the extended world ledger, T0).** Inside the zero-axiom kernel, $N[\Lambda]$ satisfies: (1) over $\langle s, \mathrm{none} \rangle$, open responsibilities correspond constructively and two-sidedly to $L_s$, with empty new-debt fibre; (2) over $\langle s, \mathrm{some}\, d \rangle$, open responsibilities correspond two-sidedly to "$L_s \sqcup$ one unique debt row" of label $\mathrm{id}_0$ and budget $\beta(d)$; (3) embedded

old rows preserve claims and budgets definitionally; (4) payment steps map to budget-decreasing active transfers, transports never increase budgets, settlement witnesses exist only at $\beta(d) = 0$, and $\beta(d) = 0$ alone never manufactures a phase-terminal event; (5) a finite inventory of $L_s$ extends to "old index $\sqcup$ debt row" (proof in Appendix C.5). ■

### 6.3 Event-Indexed Whole-Ledger Compilation and the Target-6 Instance

An **occurrence-indexed source** attached to the canonical occurrence at $c$ emits gen and reads directly from it the activation law $\Lambda$, state $d$, and **generated disposition**—a strict payment step, phase-terminal event, or obstruction. Obstructions pass through a fixed **audit protocol** (terminology table in Appendix E) pairing demand generation with row-disposition compilation.

**Proposition 6.5 (Event-indexed whole-ledger compilation, T0).** Given the audit protocol on $N$ and an active debt row in the live ledger of $N[\Lambda]$, compilation dispatches on the generated event's disposition without caller choice: a payment step compiles to a whole-ledger evolution to the target debt state; a phase-terminal event compiles to whole-support terminal evolution; an obstruction compiles to a theory audit of that exact active debt row with disposition "theory audit required" (Appendix C.5). ■

**Table 2 Debt states, budgets, and dual-interface dispositions in the target-6 prime-pair partition instance.** The strict payment step and target phase termination arise from distinct interfaces (declarations in Appendix D.1).

| Debt state | Budget $\beta$ | Disposition branch and source | Compiled output | Next budget |
|---|---|---|---|---|
| Open (active) | 1 | Strict payment step: native partition $2+4 \to 3+3$ (unique step) | Complete whole-ledger evolution (old rows carried) | 0 |
| Paid (target) | 0 | Settlement from target phase-terminal family (not a second emission) | Whole-support terminal-ledger evolution | — (terminal) |

### 6.4 Law-Free $\leq; <; \leq$ Macro Well-Foundedness and Master-Law Finite Settlement

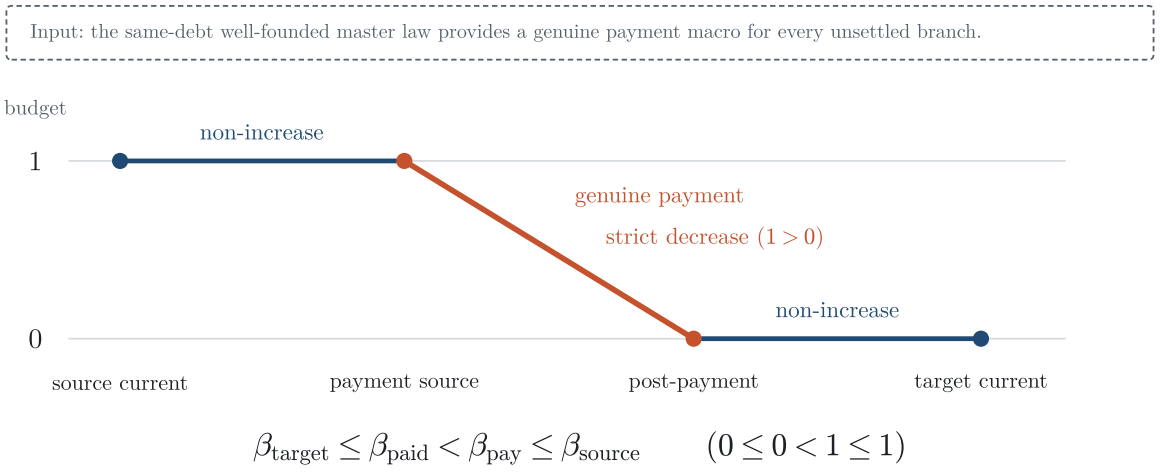
Without strict debit, equal-budget transfers can defer an obligation forever. Fixing $\mathcal{P}$ and initial debt $\omega$, define: (1) a **same-debt current** $(s, r,$ lineage and claim equations); (2) a **same-debt step** supplying the unique same-debt target row in $L_{\sigma(s)}$ with $\beta(r') \leq \beta(r)$; and (3) a **payment macro** $m = (h_-, p, h_+)$ sandwiching one **genuine strict payment step** $p$ ($\beta_{\text{paid}} < \beta_{\text{pay}}$) between two finite (possibly empty) budget-non-increasing same-debt chains $h_-, h_+$.

**Definition 6.6 (Same-debt well-founded master law).** For $\mathcal{P}$ and $\omega$, a **same-debt well-founded master law** (**master law**) is an emitter $\ell$ returning at every same-debt current $x$ either local terminal settlement (proving the next ledger has no same-debt row) or a payment macro $m = (h_-, p, h_+)$ obeying the sandwiched budget inequalities (Table 3, Figure S2):

$$\beta(m.\text{target}) \;\leq\; \beta_{\text{paid}} \;<\; \beta_{\text{pay}} \;\leq\; \beta(x).$$

**Table 3 Sandwiched budget inequalities across the three stages of a payment macro (Definition 6.6).** Only the middle payment stage decreases strictly; the right column instantiates Table 2 with identity outer chains.

| Composite stage | General budget law | Fixed-instance values |
|---|---|---|
| Target budget → post-payment budget | Non-increase ($\leq$) | $0 \leq 0$ |
| Post-payment budget → payment-source budget | Strict genuine payment ($<$) | $0 < 1$ |
| Payment-source budget → macro-source budget | Non-increase ($\leq$) | $1 \leq 1$ |




**Caption**. Figure S2 | Three-stage sandwiched budget descent of a payment macro (exact numerical figure; Definition 6.6, Theorem 6.7, Table 3). Along the macro boundary order budgets are $1, 1, 0, 0$: the middle stage is the strict debit $1 \to 0$ of Table 2 surrounded by identity boundaries. In general $\beta_{\text{target}} \leq \beta_{\text{paid}} < \beta_{\text{pay}} \leq \beta_{\text{source}}$; both outer stages allow non-increasing transport while only the middle stage (vermilion) must decrease strictly, and the dashed frame marks the external master-law input required for full terminal discharge.

**Theorem 6.7 (Law-free macro well-foundedness and master-law finite terminal settlement, T0).** Inside the zero-axiom kernel (`#print axioms = []`):

1. **Law-free well-foundedness kernel**: with **no master law supplied**, the continuation relation of same-debt payment macros is constructively well-founded; no payment macro loops back to the same current, and no zero-budget current admits a payment macro;
2. **Master-law finite terminal settlement**: if in addition a master law $\ell$ of Definition 6.6 is supplied, constructive well-founded recursion generates for **every** same-debt current $x$ a finite settlement history, a determined endpoint with no same-debt continuation, and a terminal receipt for the tracked row.

Proof. (1) Chaining the three stages of $m : x \rightsquigarrow y$ gives $\beta(y) \leq \beta_{\text{paid}} < \beta_{\text{pay}} \leq \beta(x)$, hence $\beta(y) < \beta(x)$. Thus macro continuation is a subrelation of the pullback of $(\mathbb{N}, <)$ along $\beta$, constructively well-founded by `InvImage.wf` and `Nat.lt_wfRel` in zero axioms; self-loops force

$\beta(x) < \beta(x)$ and $\beta(x) = 0$ forces $\beta(y) < 0$, both impossible. (2) Recursing along (1), $\ell(x) = \mathrm{inl}\, s$ halts with $(x, s, \epsilon_x)$, while $\ell(x) = \mathrm{inr}\, m$ recurses on $m.\mathrm{target}$ ($\beta(m.\mathrm{target}) < \beta(x)$), prepends $(h_- \cdot p) \cdot h_+$, and extracts the tracked row's whole-ledger terminal receipt at the endpoint. ■

**Note 6.8 (The master law as domain input and the zero-budget empty-terminal counterexample).** Theorem 6.7(2) proves "master law ⇒ finite terminal settlement"; the master law itself is a domain input. Without it, a state with $\beta(x) = 0$ and an empty local-terminal fibre admits neither a payment macro nor a settlement witness, rendering the master-law type uninhabited—demonstrating that numerical budget zero never automatically fabricates a terminal discharge receipt.

## 6.5 Bearer-Free Computational Obligation Birth, Step Payment, and `Type` Completion Receipts

Whereas general domain obligations require a master law, **computational obligations** require no prior bearer, master law, or future request table: an environment–expression pair $(\rho, e)$ registered at an occurrence is already a complete debt activation law.

**Definition 6.9 (Execution debt law).** For any $(\rho, e)$, the **execution debt law** $\Lambda_{\rho,e}$ instantiates Definition 6.2 with debt states $(e', t)$ ($t : e \to e'$ an execution trace under $\rho$), singleton label, claim $e$, budget $\beta(e', t) := r(e')$, strict payment step $(e', t) \to (e'', t \cdot s)$ along $s : e' \to e''$ (budget decrease by Proposition 3.7(1)), local settlement iff $e'$ is a constant ($\beta = 0$), identity transport solely at settlement, and empty obstructions.

**Theorem 6.10 (Bearer-free computational obligation birth, strict per-step payment, and relation receipt).** Over any authoritative root on $N$, origin current $c$, and reader supplying $(\rho, e)$ at $c$'s occurrence with $B := r(e)$—and with **no old bearer row, pre-registered inquiry, or future request table**—an endogenous construction on $N[\Lambda_{\rho,e}]$ produces a source, whole-ledger compiler, authoritative root, and living process such that (complete proof in Supplement S1):

1. **Birth and identity remainder**: currents are debt states of $\Lambda_{\rho,e}$ starting at $(e, \mathrm{id})$; the finite patch contains solely the new mathematical row while **every old row** on the origin support is preserved intact as a same-source identity remainder (without assuming finiteness of the old ledger), keeping origin support, law surface, and all old projections;
2. **Strict per-step debit and no refill**: at causal visit $k$, the tracked row has budget $B - k$ (truncated natural subtraction); for $k < B$, the executor's first reduction step forms a genuine payment macro with empty outer chains, strictly decreasing the budget without refill;
3. **Exact-step settlement and evaluation soundness**: at $k = B$, budget reaches 0, local settlement holds, the paid trace has length $B$, and the completed constant equals $\mathrm{ev}_\rho(e)$, while subsequent visits preserve the paid state and distinguish visit depths;
4. **Relation receipt and updated effect**: the paid trace has relation image $\delta_e - \delta_{\mathrm{const}(\mathrm{ev}_\rho e)} \in \ker(\mathrm{ev}_\rho)$, evaluating under $\rho + \varepsilon$ to $\mathrm{eff}_{\rho,\varepsilon}(e)$ for every increment $\varepsilon$. ■

**Proposition 6.11 (Whole-tree inquiry: value, provenance, and cost in one answer).** Installing Theorem 6.10 on an inquiry root whose projection reads a rooted tree $T(o)$ via $o \mapsto (\delta, \mathrm{prog}_\kappa T(o))$ delivers in a single answer the fold value $\delta_{\mathrm{fold}_\kappa T}$, the paid trace of exact length

$\beta(T)$, and syntax-only recovery of $T$ from both query and result expressions via Theorem 3.9(3). ■

To compose computations across recurring epochs (§8.5), downstream modules extract exact settlement endpoints and paid prefixes directly in `Type` (C39):

**Theorem 6.12 (`Type`-valued completion receipts and exact paid prefixes).** Fixing initial inquiry frame $\Phi_0$ and source-program configuration $\mathcal{P}$ with recursive frames $\Phi_i$, emitted actions $a_i$ (inl settlement, inr payment), and remaining counts $B_i := r(\Phi_i.\text{state.expr})$, strong recursion on $B_i$ constructs from **every actual start** $i \in \mathbb{N}$ the `Type`-valued receipt (complete proof in Supplement S1.3)

$$\mathsf{Receipt}_i := \sum_{d_i:\mathbb{N}} \Big\{ s_i : \mathsf{Settle}(\Phi_{i+d_i}.\text{state}) \;\Big|\; d_i \le B_i \;\wedge\; a_{i+d_i} = \text{inl}\, s_i \Big\}, \tag{6.1}$$

satisfying: (1) **exact distance**: $d_i = B_i$, and every $0 \le h < B_i$ emits an actual payment $a_{i+h}$ with budget $B_i - h$, retaining all intermediate states and compilations; (2) **endpoint and environment preservation**: $\Phi_{i+B_i}$ retains the registered input and paid trace with state.expr $=$ const$(\text{ev}_\rho e)$ (keeping the environment fixed whenever the reader's compatibility law holds); (3) **direct birth compilation**: settlement compilation at $i + B_i$ invokes obligation admission whose canonical successor in the same run **is** nextBorn$(\Phi_{i+B_i}, \mathcal{P})$; and (4) **cofinal birth cover**: iterating $u_0 = i,\ v_j = u_j + B_{u_j},\ u_{j+1} = v_j + 1$ yields strictly increasing cofinal birth stops $(v_j \ge i + j)$ whose segments $[u_j, v_j]$ reside inside the single recursive frame run. ■

# 7 Dependency-Tree Residuals, Double-Empty Bankruptcy, and Causal-Audit Feeds

When a transition leaves an irreducible discrepancy in the old representation, that discrepancy must be classified and anchored to the current occurrence as a **rooted expressive failure** before entering initial-coface revision (§8); conversely, when dependency trees align, the same classification hands over a step-charged whole-tree program package and feeds the next visit automatically (C11, C25–C27).

Fix a complete root registration, a complete complex inner-product space $\mathcal{H}$ (inner-product geometry is unused in §7's classification and audit proofs, serving as the shared recognition carrier for §9 and physics instantiations), an installed joint recognition rec, and a temporal visit $v$, determining a **step** step$(v)$ whose whole-ledger write-back and next current equal the root's generated records.

## 7.1 Six-Branch Master Disposition and Same-Support Free Residual Extension

**Proposition 7.1 (Six-branch master disposition of a dependent step).** Every step is exhaustively classified by a total six-branch inductive function settle carrying exact coordinates: (1) **terminal settlement**; (2) **generator residual** between target and source histories; (3) **relation residual** under generator compatibility; (4) **pairing-shape residual** between source and target

unfolding trees; (5) **fully generated** passive carry with aligned tree rooted at the pairing sequence; and (6) **joint residual** at a determined node of the alignment trace. ■

Proof. Nested dispatch composes terminal-successor inspection, generator/relation comparison with position-wise unfolding-tree zipping (Theorem 2.2), and aligned-tree folding, reading exact coordinates from each sub-decision's dependent output. ■

**Definition 7.2 (Same-support free residual extension).** Given a sealed **rooted passive residual** $r$ (carrying focus support, residual family $\mathcal{R}$, and target occurrence), the **same-support free extension** $N[\mathcal{R}]$ keeps supports, anchors, incidence, and lineage as in $N$, sets responsibilities and claims to Responsibility$\sqcup\mathcal{R}$ and Claim$\sqcup\mathcal{R}$ (opening new rows solely at $r$'s focus support), sets obstructions to Obs$\sqcup(\mathcal{R}\times[\text{support} = \text{focus}])$, and adds the new opening receipt under law-surface extension.

Here a **theory state** $\Theta$ comprises expression fibres $E_\Theta(s)$, denotation $d_\Theta : E_\Theta(s) \to$ Claim, two-sided theorem–fact representations, and lawful-realization fibres $I_\Theta(o)$. Expressing an obstruction $o$ requires a witness in $\sum_{a:E_\Theta(s)}[d_\Theta(a) = \text{claim}(o)]$; **double-empty expressive failure** means both this expression-witness type and $I_\Theta(o)$ are empty.

**Proposition 7.3 (Initiality of new-claim elimination and conservative old-ledger readback, T0).** Inside the zero-axiom kernel, the claim eliminator of $N[\mathcal{R}]$ is pointwise initial (any pair of maps $f$ on old claims and $g$ on $\mathcal{R}$ extends uniquely to $[f, g]$), and all old open rows, world facts, and dispositions survive unchanged via constructive two-sided representations or definitional equality.

**Proposition 7.4 (Dependency residuals yield exact double-empty expressive failures, T0).** Transporting $\Theta$ conservatively to $\widetilde{\Theta}$ on $N[\mathcal{R}]$ (keeping $E_\Theta(s)$, setting $d_{\widetilde{\Theta}}(a) := \text{inl}\, d_\Theta(a)$, and setting new-side realizations to $\emptyset$) makes every new residual coordinate an exact double-empty failure in zero axioms—even when $\Theta$ is complete root semantics on $N$.

Proof (Propositions 7.3 and 7.4). Coproduct elimination $[f, g]$ fixes values uniquely on $\text{inl}, \text{inr}$ without altering old opening witnesses. In Proposition 7.4, any old expression $a$ denoting $\text{inr}\, r$ would satisfy $\text{inl}\, d_\Theta(a) = \text{inr}\, r$, refuted by constructor disjointness (`nomatch`), while $I_{\widetilde{\Theta}}(\text{inr}(r, h)) = \emptyset$ holds definitionally. The failure reflects missing vocabulary dimensions, not logical inconsistency of $\Theta$. ■

## 7.2 Causal Audit Transitions and Whole-Tree Successor Feeds

**Proposition 7.5 (Causal audit transition: old rows as identity remainders and unique new-row authority).** Given a selected residual $r$, fixed audit protocol on $N$, and exact temporal causal root event, an endogenous **causal audit transition** is generated (Appendix C.6): (1) step disposition equals settle's output and occurrences match; (2) the target audit payload stores both source and residual-target occurrences; (3) whole-ledger write-backs and next currents agree across root event, step, and root; (4) **every embedded old row passes row-by-row identity verification** as "not newly generated"; (5) **the new residual row alone acquires live temporal causal authority**; and (6) that row directly assembles the **rooted double-empty failure** of Definition 8.1 and the **initial coface** of §8. ■

On the two aligned branches (fully generated and joint residual), the source hands over step-charged whole-tree packages directly into execution:

**Proposition 7.6 (Whole-tree source-program feed of a dependent step).** Under the fixed data of §7, settle produces a type-matched **feed**: terminal returns the root's whole-ledger evolution; the three residual branches return the residual; the two aligned branches return a **whole-tree program package**—aligned tree $T$, paid state and value $\delta_{\tau(T)}$ under $\mathrm{prog}_{\kappa}T$, origin material, and all $\beta(T)+1$ stages at exact syntax cost $\beta(T)$. ■

**Theorem 7.7 (Registered-input activation and native successor feed).** Adjoining the root's registered inquiry input $\iota$ at visit $v$ (field-by-field preservation contracts and complete proof in Supplement S2.2):

1. **Registered activation and whole-tree inversion**: on both aligned branches, a new inquiry state $s_{\iota}$ compiles $\iota$'s query to an answer whose syntax-only left inverse recovers $T$ via Theorem 3.9(3) with value $\delta_{\tau(T)}$ (preserving all prior query compilations);
2. **Length-$\beta(T)$ charged runtime**: the inquiry runtime from $s_{\iota}$ executes a strict debit history of exact length $\beta(T)$ to $\delta_{\tau(T)}$, keeping whole-ledger write-backs at every count and offset;
3. **Caller-free successor feed**: from living root, visit $v$, recognition rec, and audit protocol alone (requiring no pre-completed prior-question state), the non-terminal successor generates $v^{+}$ and the next feed/runtime while equating the successor-target history with the canonical history at $v^{+}$. ■

## 8 T0 Zero-Axiom Initial-Coface Revision, Two-Tick Relay, and Automatic Post-Settlement Re-Inquiry

We now address the deepest transition in process identity: when an old theory has neither an expression nor a lawful realization for an obstructed demand (double-empty bankruptcy), how can the process enlarge its expressive space without contracting a single old theorem or losing a single live obligation, and then answer the demand? Subsections 8.1–8.4 prove inside the **T0 zero-axiom kernel** (`#print axioms = []`) that any rooted double-empty failure $\eta$ endogenously generates the **categorical initial coface**, factoring uniquely, injectively, and without budget refill through every admissible extension and sending the new direction onto the exact live row consumed by a typed revision's first whole-ledger write so the new root answers directly at tick 2 (C4–C6, C13). Subsection 8.5 returns to the sufficient-vocabulary branch, showing how settled receipts birth the next question, execute same-occurrence boundary differences under independent syntax budgets, and write back complete totals $V = p + n$ (C24, C35–C38, C42).

The **fixed minimal instance** running through §§8.1–8.4 makes every field concrete: the old world has supports $\{\mathrm{false}, \mathrm{true}\}$, singleton anchors/incidence/lineage/responsibilities/claims, one open row of budget 2 per support, and one old world fact; the old root has unit current payload $()$, canonical occurrence $\langle \mathrm{true}, () \rangle$, and micro-step write $() \mapsto ()$ advancing visit depth by 1. The old theory $\Theta$ has empty expression and realization fibres, singleton query $()$ bound to the budget-2 row, and obstruction $o = ()$ at true. By payload alone every tick is $()$; anchored to root, visit, and obstructed row, the absence acquires an exact address.

### 8.1 Rooted Double-Empty Failure and Zero-Leakage Initial Cofaces (T0)

**Definition 8.1 (Rooted double-empty expressive failure).** Fixing old root, visit $v$, theory $\Theta$, obstruction $o$ at $s$, and double-empty proof $f$, $f$ is **rooted** (denoted $\eta$) when accompanied by six dependent witnesses: (1) **theory ownership** ($\Theta$ is the old root's law surface); (2) **occurrence projection** (the emitted occurrence projects to $f$); (3) **obstruction audit** (compiled to "theory audit required"); (4) **row causal authority** on the demanded open row $e$; (5) **whole-ledger compatibility** between audit disposition and root compilation (excluding in the fixed instance any "transfer" row posing as an identity-preserving audit carry); and (6) **non-terminal causal successor** supplying target support $s_+$, target ledger $L_+$, and target row $e_+ \in L_+$.

**Proposition 8.2 (Local initial coface and zero-budget attached direction, T0).** Inside the zero-axiom kernel, every rooted failure $\eta$ generates a single-constructor direction type $D_\eta = \{\star_\eta\}$ and local initial-coface carrier

$$C_\eta := L_+ \sqcup D_\eta, \qquad \beta(\mathrm{inl}\, e) := \beta(e), \qquad \beta(\mathrm{inr}\, \star_\eta) := 0. \tag{8.1}$$

Old rows $L_+$ embed via inl with constructive retraction mapping $\mathrm{inl}\, e \mapsto e$ and $\mathrm{inr}\, \star_\eta \mapsto e_+$ (strictly left-inverting inl without choice); $\mathrm{inr}\, \star_\eta$ has no old preimage and carries the unique attaching direction bounded by $\eta$ (proved directly by single-constructor induction).

Proof. Disjointness of inl and inr refutes any old preimage in zero axioms (`PEmpty` elimination); the retraction evaluates $\mathrm{inr}\, \star_\eta$ at the deterministic $e_+$ carried inside $\eta$; budgets preserve $\beta(e)$ on the left and assign 0 on the right. ■

The term **initial coface** reflects the exact cosimplicial geometry[24]: in the simplicial category $\Delta$, an injective coface operator $\delta_i^n : [n-1] \hookrightarrow [n]$ embeds a lower-dimensional vertex set into a simplex with one adjoined vertex, paired with a surjective codegeneracy retraction $\sigma_i^n : [n] \twoheadrightarrow [n-1]$ collapsing the new vertex onto an existing vertex so that $\sigma_i^n \circ \delta_i^n = \mathrm{id}$. In (8.1), $\mathrm{inl} : L_+ \hookrightarrow L_+ \sqcup \{\star_\eta\}$ is precisely the vertex-omitting coface injection missing $\star_\eta$, while the constructive retraction sending $\mathrm{inr}\, \star_\eta \mapsto e_+ \in L_+$ is its paired codegeneracy map; Theorem 8.4 proves that this triple $(\mathrm{inl}, \mathrm{retraction}, \star_\eta)$ is strictly initial across the category of all admissible coface revisions.

**Proposition 8.3 (Whole-network initial coface, zero old-face leakage, and survival of old theorems, T0).** Inside the zero-axiom kernel, $\eta$ generates an extended world network $N_\eta$ taking disjoint unions with $D_\eta$ on supports, incidence, responsibilities, and claims while keeping anchors and lineage (piecewise fibres in Appendix C.1), satisfying:

1. **Conservative preservation with zero old-face leakage**: on every embedded old support $\mathrm{inl}\, s$, open ledgers, facts, and dispositions correspond constructively and two-sidedly to $N$ at $s$ (the latter two definitionally), while new responsibilities and new claims on old supports are strictly empty ($\mathrm{Open}'(\mathrm{inl}\, s, \mathrm{inr}\, \star_\eta) = \emptyset$, $\mathrm{Holds}'(\mathrm{inl}\, s, \mathrm{inr}\, \star_\eta) = \emptyset$);
2. **Coface inheritance and sealed dispositions**: at the new coface support $\mathrm{inr}\, \star_\eta$, all open rows and facts of $s_+$ survive alongside one new open row $e_{\mathrm{obs}} = \langle \mathrm{inr}\, \star_\eta, \mathrm{same}(\star_\eta) \rangle$ of claim $\mathrm{inr}\, \star_\eta$ and budget 0, where $\mathrm{inr}\, \star_\eta$ holds with unique semantic-change constructor $\mathrm{reveal}_\eta$ from $\mathrm{inl}\, \mathrm{claim}(o)$, and dispositions of $s_+$ are wrapped two-sidedly in the $\eta$-indexed envelope $\mathrm{Seal}_{\eta,k}$ so unsealed old or sibling receipts cannot impersonate the new support;

3. **Full survival of old theorems**: every proof object $p : \mathrm{Theorem}_{\Theta}(a)$ of the old theory maps via $\Theta$'s theorem–fact presentation and left fact embedding to the exact same world fact on the old face of $N_\eta$, with zero AGM contraction[18]. ■

### 8.2 Categorical Initiality: Unique Faithful Factorization Across Three Coordinates (T0)

**Theorem 8.4 (T0 zero-axiom categorical initiality and unique faithful factorization of the initial coface).** Fix $\eta$. An **admissible coface revision** $\mathcal{E}$ supplies constructive retractions $(j_{\mathrm{led}}, p_{\mathrm{led}})$, $(j_{\mathrm{inc}}, p_{\mathrm{inc}})$, and $(j_{\mathrm{clm}}, p_{\mathrm{clm}})$ from $L_+$, Inc, and Claim together with one new point $(a_{\mathrm{led}}, a_{\mathrm{inc}}, a_{\mathrm{clm}})$ having no old preimage on each coordinate, such that inherited budgets never increase ($\beta_{\mathcal{E}}(j_{\mathrm{led}}(e)) \le \beta(e)$) and $\beta_{\mathcal{E}}(a_{\mathrm{led}}) = 0$. Inside the zero-axiom kernel (`#print axioms = []`):

1. **Admissibility of the initial coface**: $N_\eta$ at $\mathrm{inr}\,\star_\eta$ is itself a canonical admissible coface revision;
2. **Existence of an injective, budget-non-increasing initial morphism**: to **every** admissible coface revision $\mathcal{E}$ there exists a canonical faithful morphism $F : C_\eta \to \mathcal{E}$ whose three coordinate maps
$$F_{\mathrm{led}}(\mathrm{inl}\, e) = j_{\mathrm{led}}(e), \quad F_{\mathrm{led}}(\mathrm{inr}\,\star_\eta) = a_{\mathrm{led}} \tag{8.2}$$
(and analogously $F_{\mathrm{inc}}, F_{\mathrm{clm}}$) are **all injective**, preserve old embeddings and new points, and satisfy $\beta_{\mathcal{E}}(F_{\mathrm{led}}(x)) \le \beta(x)$ on every ledger coordinate;
3. **Strict unique factorization**: $F$ is strictly unique in the category of structure-preserving faithful coface morphisms, and any candidate coordinate map respecting old embeddings and the new point equals $F$ pointwise.

Proof. (1) Proposition 8.3 supplies the retractions, empty old-preimage proofs, $\beta(e) \le \beta(e)$, and $\beta(\mathrm{inr}\,\star_\eta) = 0$. (2) For injectivity of (8.2) (identical across all three coordinates), assume $F_{\mathrm{led}}(u) = F_{\mathrm{led}}(v)$ and split by constructors: on $(\mathrm{inl}\, e_1, \mathrm{inl}\, e_2)$, applying $p_{\mathrm{led}}$ gives $e_1 = p_{\mathrm{led}}(j_{\mathrm{led}}(e_1)) = p_{\mathrm{led}}(j_{\mathrm{led}}(e_2)) = e_2$; on $(\mathrm{inl}\, e, \mathrm{inr}\,\star_\eta)$ or its symmetry, $j_{\mathrm{led}}(e) = a_{\mathrm{led}}$ contradicts $\mathcal{E}$'s empty old-preimage proof; on $(\mathrm{inr}\, d_1, \mathrm{inr}\, d_2)$, the singleton constructor of $D_\eta$ forces $d_1 = d_2 = \star_\eta$. Budget non-refill holds on $\mathrm{inl}\, e$ by $\mathcal{E}$'s hypothesis and on $\mathrm{inr}\,\star_\eta$ by $0 \le 0$. (3) Any structure-preserving coordinate map has its values on $\mathrm{inl}\, e$ and $\mathrm{inr}\,\star_\eta$ fixed by (8.2), giving pointwise uniqueness, while the field-free canonical morphism token admits only one constructor and holds by reflexivity `rfl` (Appendix C.1). ■

Theorem 8.4 makes "minimal revision" mathematically exact: **a downstream revision may carry richer dynamics, but it can never bypass the unique direction generated by the initial coface**.

### 8.3 Typed Semantic Revisions, Semantic Anchoring, and the First Write (T0)

**Definition 8.5 (Typed semantic revision).** For fixed $\eta$, a **typed semantic revision** $\Xi$ supplies seven running components with compatibility proofs: (1) new world, vocabulary, semantic translation, and living root; (2) retraction of the new initial support to $s_+$; (3) budget-non-increasing retraction embedding of $L_+$ into the new initial ledger preserving translated claims; (4) two-sided occurrence representation between old successor and new initial visit preserving

canonical emission; (5) a zero-budget new row $e_{\text{new}}$ whose row, claim, and support incidence have no old preimages; (6) proof that $e_{\text{new}}$'s claim holds at the new initial support with semantic change from the old obstruction claim; and (7) **first whole-ledger write**: non-terminal initial compilation of the new root generating both all inherited old rows and $e_{\text{new}}$ in its finite patch.

**Theorem 8.6 (Unique factorization of the attached direction into the first whole-ledger write, T0).** Inside the zero-axiom kernel, every typed semantic revision $\Xi$ induces an admissible coface revision of Theorem 8.4 whose unique faithful morphism $F_\Xi$ maps the initial coface's attached direction to $F_\Xi(\mathrm{inr}\,\star_\eta) = e_{\text{new}}$—the exact row consumed by $\Xi$'s first whole-ledger write. ■

Proof. Conditions (3) and (5) of Definition 8.5 instantiate an admissible coface revision, so Theorem 8.4 yields the unique $F_\Xi$ with $F_\Xi(\mathrm{inr}\,\star_\eta) = e_{\text{new}}$, which condition (7) places directly into the generated patch of the first write. ■

**Theorem 8.7 (Semantic anchoring and two-row answer–next recovery, T0).** Suppose $\Xi$ carries either **exact semantic anchoring** (one world equality plus seven heterogeneous equalities identifying $\Xi$'s world, root law surface, translation, initial support, old-ledger retraction, new row, new fact, and semantic change with those of $N_\eta$, Appendix C.2) or **split-refined semantic anchoring** (requiring root semantics on a richer world whose first compilation genuinely splits the migrated failure row into two distinct strictly lower-budget child rows with no dormant off-ledger rows). Inside the zero-axiom kernel, advancing the initial causal authorities of both the migrated old failure row and $e_{\text{new}}$ to their first-write destinations and invoking **the new root's own** answer–next compiler recovers both answer–next tokens alongside the unique factorization of Theorem 8.6, with zero external target-occurrence or next-current inputs (Appendix C.3). ■

**Proposition 8.8 (Associativity of relative expressive capacities across successive revisions, T0).** When a second typed revision is rooted at a visit of the first revised root, relative expressive capacities on incidence and claims admit injective embeddings with direction family $D_1 \sqcup D_2$, and three composable capacities satisfy $k_{(12)3} \circ \alpha = k_{1(23)}$ under canonical coproduct rebracketing $\alpha$ (Appendix C.4). ■

## 8.4 The Fixed Two-Tick Revision Instance and the One-Shot Impossibility Gate

**Proposition 8.9 (Initial-coface revision and second-tick direct answer in the fixed two-tick instance).** For the fixed minimal instance and the exactly anchored natural-number new root (carrier $\mathbb{N}$, write $n \mapsto n+1$, two-row patch $\{0,1\}$), tick 1 of the canonical inquiry runtime takes the expression-revision branch (terminology table in Appendix E), outputting public answer $\star_\eta$ whose occurrence reads back the first write $0 \to 1$ consuming both $e_o$ (budget 2) and $e_n$ (budget 0); target compilation $1 \to 2$ produces the shared macro successor 2 on the three-point support $\mathrm{Bool} \sqcup D_\eta$ (provably distinct from the two-point old micro-successor); and at tick 2, visit 2 of the same new root enters the direct-answer branch, returning the singleton answer $*$ (certifying original query $()$, current occurrence, and $e_n$) while advancing to 3 (Table 4, Figure 5).

**Table 4 Complete trajectory of the fixed two-tick initial-coface revision instance.** Here $\star_\eta$ is the unique attached direction of $\eta$; $e_o = \langle \mathrm{inl}(), () \rangle$ and $e_n = \langle \mathrm{inr}\, \star_\eta, \mathrm{same}(\star_\eta) \rangle$; $0, 1, 2, 3$ are natural-number currents of the new root.

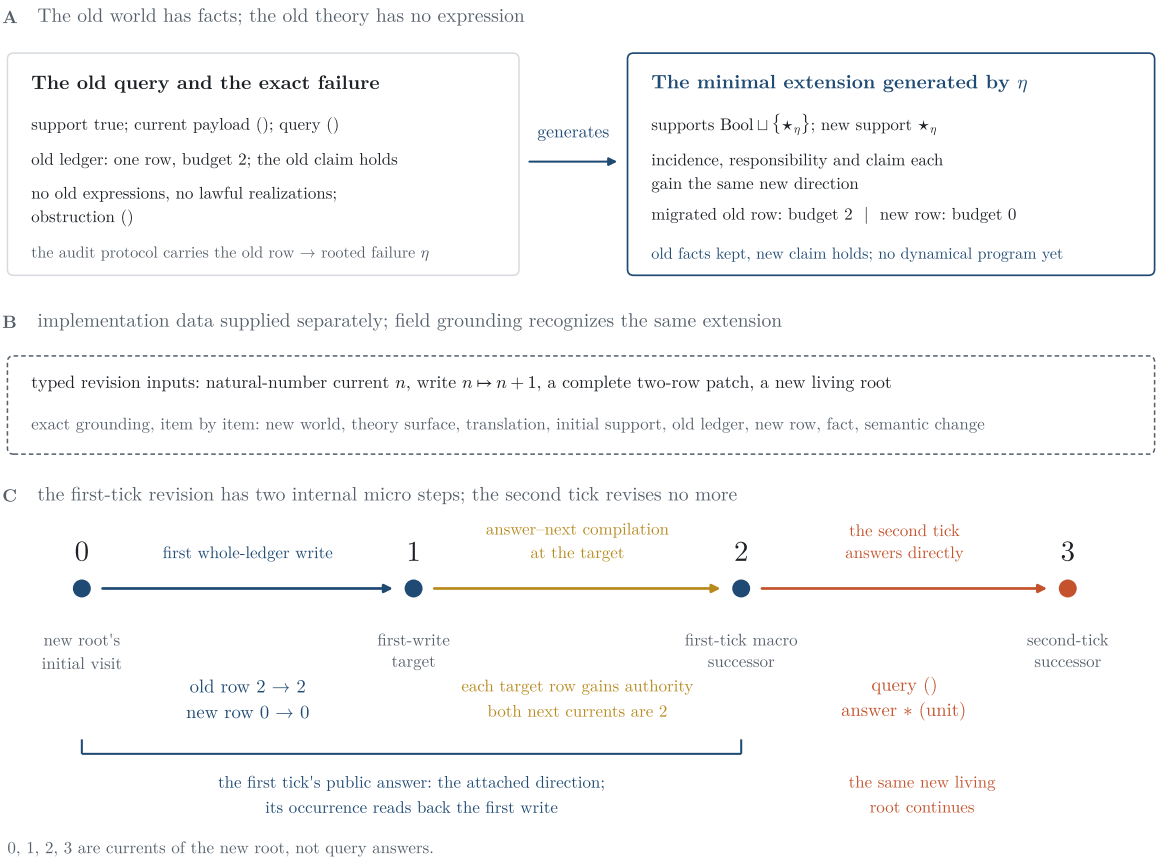
| Stage | Current, support, and live ledger | Object or state transition | Input vs. endogenous output |
|---|---|---|---|
| Old initial inquiry | Old visit 0, payload $()$; support true; row budget 2 | Query $()$, occurrence $\langle \mathrm{true}, () \rangle$; empty old syntax & realizations | Old world, double-empty theory, and query are instance data |
| Tick-1 bankruptcy | Audit carries old row to old successor, payload $()$ | Obstruction $o = ()$; empty-fibre eliminations; audit matches carry | $\eta$ emitted endogenously by the installed query compiler |
| Initial coface | Support $\mathrm{Bool} \sqcup D_\eta$; focus $\star_\eta$; rows $e_o : 2,\ e_n : 0$ | Incidence, responsibility & claim gain $\star_\eta$; old facts survive; $\mathrm{reveal}_\eta$ | Generated in zero axioms from $\eta$; no $\mathbb{N}$ dynamics yet |
| Revision & anchoring | New root current 0, occurrence $\langle \star_\eta, () \rangle$ | Carrier $\mathbb{N}$, write $n \mapsto n + 1$, two-row patch; root semantics of $N_\eta$ | Dynamics from typed revision; exact anchoring recognizes $N_\eta$ |
| First write | Current $0 \to 1$, support $\star_\eta$ | Non-increasing transfers $e_o : 2 \to 2,\ e_n : 0 \to 0$; patch $\{0, 1\}$ | Consumes both rows and generates target causal authorities |
| Tick-1 target compile | Consumes both target rows at visit 1; both yield next 2 | Public answer $\star_\eta$; macro successor is new-root current 2 | Invokes new root's compiler, fusing $0 \to 1$ and $1 \to 2$ |
| Tick-2 direct answer | Same new root, visit 2; original query $()$ | Answer face reads $*$ certifying query, occurrence & $e_n$; $2 \to 3$ | Answers directly without re-revising; continues recursively |

Proof. Four verified links join the two ticks: (i) the old inquiry compiler emits the revision branch on double emptiness, matching tick 1 of the canonical runtime pointwise; (ii) patch indices $\{0, 1\}$ match inl / inr bijectively without budget refill; (iii) after $0 \to 1$, Theorem 8.7 evaluates the new root's row compiler on both rows to 2, and cardinality separation between three-point and two-point supports distinguishes the macro successor from the old micro-successor; (iv) visit 2 compiles directly to the answered constructor $*$ and advances $2 \to 3$. ■

**Proposition 8.10 (A one-shot empty-fibre interface admits no autonomous inquiry runtime, T0).** Inside the zero-axiom kernel (`#print axioms = []`), no interface whose post-answer query fibre is empty admits an autonomous inquiry runtime.

Proof. Any autonomous runtime's total activation law would have to produce a next query activation at the exact successor of the first answer, inhabiting an empty query type—contradiction. ■

Once a revised new living root is registered as a source process via Definition 2.3, the free carrier $\mathbb{Z}[S]$, exact evaluation kernel, and completion carrier $\widehat{C}(s)$ of §2.3–§4 apply to it immediately (kept as an explicit parameterized reuse interface, Appendix D).




**Caption**. Figure 5 | From rooted double-empty failure to two ticks of answers from the same new root (exact trajectory; Table 4, Proposition 8.9). Double-empty failure $\eta$ generates the zero-axiom initial coface and attached direction $\star_\eta$; the typed revision supplies the natural-number dynamical root, joined via exact semantic anchoring. The first public answer is $\star_\eta$, whose occurrence reads back the new root's first write: $0 \to 1$ consumes both rows of budgets 2 and 0, and target compilation $1 \to 2$ yields the shared macro successor 2; at tick 2, visit 2 of the same new root directly reads the singleton answer $*$ (vermilion) and advances to 3; the dashed frame marks caller-supplied revision inputs.

## 8.5 Post-Settlement Continuation: Residual Re-Inquiry, Boundary Differences, and Total-Value Writeback

Proposition 8.10 shows negatively that a runtime halts if no next question exists; we now close the positive loop with zero external request tables by evaluating a settled computation's paid relation receipt in the updated environment (C24).

An **inquiry frame** $\Phi$ comprises an inquiry state, a native program reading the root compiler, a registered input $(\rho, e)$, an actual-environment reader $E$, and causal depth $k$, with paid state $(e', t)$ in $\Lambda_{\rho,e}$. Let $\rho'$ denote $E$'s reading at $\Phi$'s occurrence and $\varepsilon := \rho' - \rho$.

**Definition 8.11 (Residual request).** The **residual request** of paid state $(e', t)$ in updated environment $\rho' = \rho + \varepsilon$ is the input registered at $\rho'$:

$$e \ominus e' := e + \mathrm{linear}_{-\mathrm{id}}(e'), \tag{8.3}$$

whose obligation bearer is the frame's current responsibility row.

**Theorem 8.12 (Actual action generates the next question and drives the recursive frame runtime).** For every initial inquiry frame $\Phi_0$ (complete proof in Supplement S2.1, Figure 6):

1. **Request value equals updated effect**: $\mathrm{ev}_{\rho'}(e \ominus e') = \mathrm{ev}_{\rho'}(e) - \mathrm{ev}_{\rho'}(e')$, vanishing at $\rho$ and equalling under $\rho'$ the effect $\mathrm{eff}_{\rho,\varepsilon}(e) - \mathrm{eff}_{\rho,\varepsilon}(e')$ of the paid trace certificate $\delta_e - \delta_{e'}$ (vanishing iff its canonical coimage residual under $\mathrm{ev}_{\rho'}$ vanishes); at settlement $e' = \mathrm{const}(\mathrm{ev}_\rho e)$, its value is $\mathrm{eff}_{\rho,\varepsilon}(e)$;
2. **Forced obligation birth**: since $r(e \ominus e') = r(e) + r(e') + 2 \geq 2 > 0$, the request is never initially settled; its first step is an actual reduction compiling to **obligation admission** rather than an old-debt payment;
3. **Table-free recursive frame runtime**: a reduction step advances depth $(r(e), k+1) \to (r(e), k+2)$, while settlement births a depth-0 frame on $e \ominus e'$ of rank $(r(e) + r(e') + 2, 1)$; strict lexicographic rank growth makes erasure injective and aligns frame-by-frame with the canonical inquiry runtime over singleton query fibres;
4. **Strict separation of payment and birth**: restricting to the fixed-root debt process, only reduction steps emit same-debt payments, zero budget admits no payment macro, and settlement births a distinct obligation row. ■

**Example 8.13 (Next questions of the three-tick candidate words).** In Table 1, register each word at $x \mapsto 1$, run to settlement, and shift the environment: (1) 0 settles in 0 steps to $\mathrm{const}(0)$; shifting to $x \mapsto 0$ births $0 \ominus \mathrm{const}(0)$ of value 0 and syntax budget $0 + 0 + 2 = 2$; (2) $x$ settles in 1 step to $\mathrm{const}(1)$; shifting to $x \mapsto 0$ births $x \ominus \mathrm{const}(1)$ of value $-1$ and budget 3; (3) $x^2 - x$ settles at $x \mapsto 1$ to $\mathrm{const}(0)$; shifting to $x \mapsto 2$ births a request of value 2, turning Example 4.2's relation breakdown into a positive-budget inquiry. Registered at the junction $x \mapsto 0$, 0 and $x$ both answer 0 now, yet upon returning to $x \mapsto 1$ ask next questions of values 0 and 1: the old coordinate is the current answer, and the effect coordinate is the next question. ■

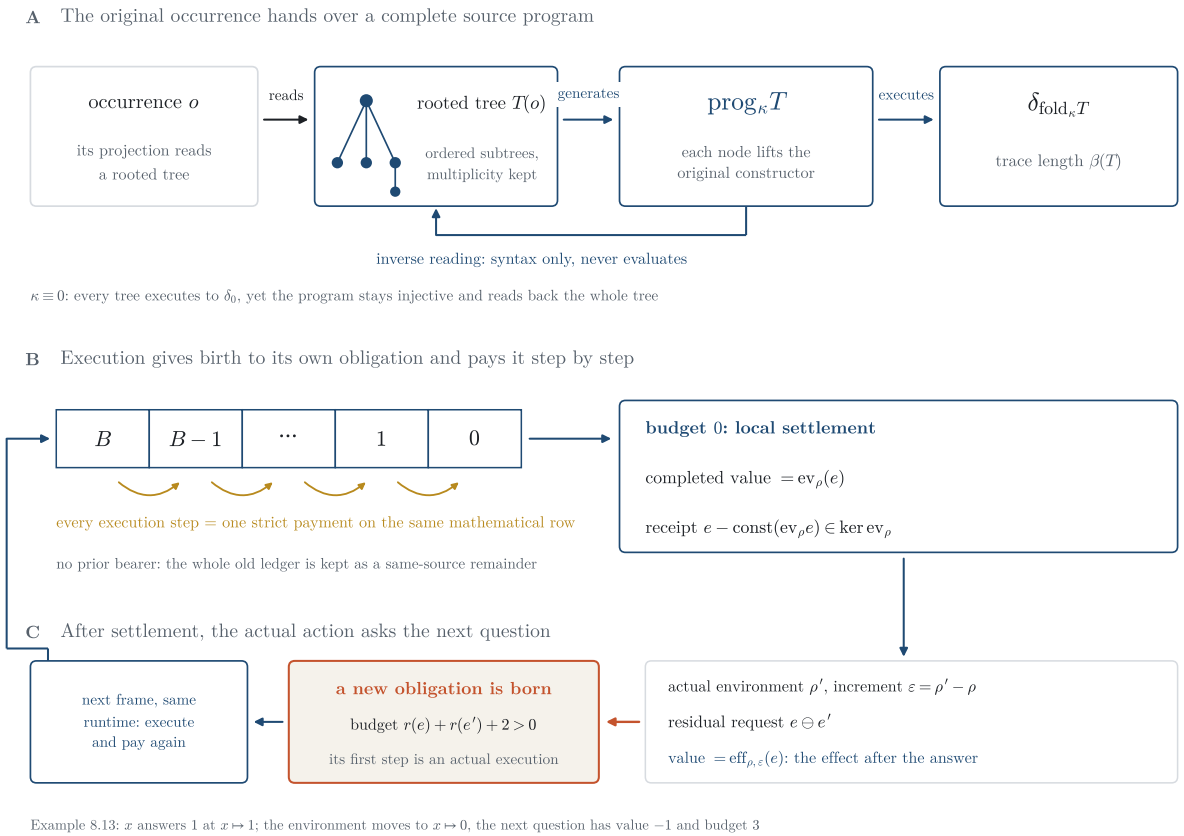
Moreover, subtracting two complete paid boundaries at the same occurrence generates an independently charged difference query with joint-stock feedback (C35–C38), while each completion receipt writes back old paid value plus native effect as a complete updated total (C42):

**Theorem 8.14 (Independent syntax-charged execution and joint-stock feedback of a same-occurrence boundary difference).** At an exact successor occurrence with two paid relation boundaries $b_\Psi, b_\mu$ sharing environment $\rho^+$, native pair write $\mu = (\Psi, \sigma)$, and after-scope readings $Q^+(b_\Psi) = r - c(\Psi, 0)$ and $Q^+(b_\mu) = r - c(\mu)$ around common root point $r$, expanding $d := b_\Psi - b_\mu$ in support-list order defines $\mathrm{coeff}(d)$ (complete proof in Supplement S2.3):

1. **Root-point cancellation and exact difference**: $d$ stays in the relation closure with $Q^+(d) = c(0, \sigma)$, evaluating in $\rho^+$ to $(0, \sigma)$;
2. **Independent syntactic budget**: $\mathrm{coeff}(d)$ has exact remaining count

$$c(d) := \sum_{(e_i, a_i) \in \mathrm{suppList}(d)} \Big( |a_i|(r(e_i) + 1) + \mathbf{1}_{a_i < 0} + 1 \Big), \tag{8.4}$$

executed by Theorem 6.10 along a length-$c(d)$ trace with $c(d) + 1$ states without borrowing either parent query's fee certificate;




**Caption**. Figure 6 | Endogenous chain from original occurrence to next question (structural diagram; Theorems 3.9, 6.10, 8.12, Example 8.13). An occurrence projection reads rooted tree $T$; its whole-tree program executes in $\beta(T)$ steps while a syntax-only parser recovers $T$ without evaluation. Every execution step strictly debits the born mathematical row $B, B-1, \ldots, 0$; at settlement the completed value equals the original evaluation and the paid trace yields a relation receipt. After environment change, the residual request has value equal to the effect and positive budget $r(e) + r(e') + 2 > 0$; its first step births a new obligation (vermilion) returning to the same inquiry runtime, with no dashed external-input frame anywhere along the chain.

3. **Cross-admission feedback and environment square**: for source-family query $q$, paid endpoint $q'$, and action-variable substitution binding $\theta$ (substitution notation in §9.3), answered compilation births residual $h = q \ominus q'$ whose bound feedback $h + h[\theta]$ (budget $\geq 3$, evaluating to the trace-boundary effect $\mathrm{eff}_{\rho,\rho\circ\theta-\rho}(\delta_q - \delta_{q'})$) and paid exposure enter joint stock; finite incoming configuration proves $\mathrm{env}(\mathrm{actionReader}(o)) = \mathrm{env}(\mathrm{queryReader}(o))$ at every occurrence while keeping receiver budget $B_{\mathrm{res}} - 1 \geq 1$ (with $B_{\mathrm{res}} = r(e) + r(e') + 2 \geq 2$) after step 1. ■

**Theorem 8.15 (Each completion receipt writes back the total value and regenerates its decoder).** Given action material $M = (\rho, \varepsilon, e, \mathrm{State}, \mathrm{Trace}, \mathrm{owner})$ settled at $\mathrm{const}(p)$ ($p = \mathrm{ev}_\rho e$), residual request $e \ominus \mathrm{const}(p)$ at $\rho + \varepsilon$, completed `Type`-receipt value $n$, and decoder write, setting birth-environment readout on the settled branch to $\mathrm{write}(p+n)$ yields (complete proof and constant zero-effect control in Supplement S5.2):

$$n = \mathrm{ev}_{\rho+\varepsilon}(e) - p, \qquad V := p + n = \mathrm{ev}_{\rho+\varepsilon}(e), \qquad \rho_{\mathrm{Born}} = \mathrm{write}(V), \tag{8.5}$$

advancing only mathematical states on paid branches while regenerating a fresh decoder from

$p^+$ at each subsequent cofinal packet to write $p^+ + n^+$. Even when $n = 0$ while $p \neq 0$, Born writes write$(p)$, never erasing previously paid work. ■

# 9 Complete Execution: Temporal Coding, All-Actor Recovery, Order-Preserving Stock Injection, and the Maximal Future Kernel

While §8.5 tracked a single word across settlement, a multi-stage run also carries temporal visit histories, source and target dependency histories, pairing-tree operators, and two-level inventories. This section proves that one joint program folds complete time and common histories alongside effects (§9.1, C28), all pairing-position actors jointly recover the all-word model and future readouts (§9.2, C29), universal source-program admission carries two-level typed inventories with word–state covariance (§9.3, C30–C32), and the default run closes whole-word recovery, strictly order-preserving trace-position injection into born stock, cofinal intermediate-tick coverage, six-face cochain complexes, and the **maximal future-invisible invariant kernel model** (§9.4, C33–C34, C40, C43–C45).

An important structural division governs the rest of the paper: **§§9.1–9.2** retain §7's Hilbert space $\mathcal{H}$ (with living root, visit $v$, joint recognition rec, audit protocol, and non-terminal step$(v)$ carrying $v^+$, $\mathcal{H}_s, \mathcal{H}_t$, and source/target pairing unfolding trees) to capture normed multi-node measurements and fibre-lifting criteria; **from §9.3 onward through the whole of §10, all inner-product and metric-completeness hypotheses are dropped**, returning to arbitrary additive abelian groups where universal admission, order-preserving stock injection, six-face cochains, $\mathsf{M}_{\mathrm{fut}}$, and reversible continuation are driven purely by free modules and syntax—verified by hand on our running integer expression in **Example 9.13** at the end of §9.4.

## 9.1 One Joint Program Folds Complete Time, Common History, and the Action-Difference Lifting Criterion

**Definition 9.1 (Temporal codes, occurrence receipts, and pairing exposures).** (1) **Temporal code**: visits are finitely reachable, sealed cofinal, or post-cofinal, each encoded inductively; a **temporal code** $c$ pairs the current with its history code under mutually inverse enc and dec; (2) **Receipt and temporal action**: rcpt$(c)$ pairs $c$ with the origin material of $c$'s canonical occurrence, nx$(c)$ encodes the successor visit (none at terminals), and act$(c) :=$ (rcpt$(c)$, nx$(c)$); (3) **Exposure and actors**: at each pairing position, recognition installs $(M, A, U)$—a $\mathbb{Z}$-linear measurement $M : \mathrm{Car} \to \mathcal{H}$, equivariant action $A$, and linear isometry $U$ on $\mathcal{H}$; **every node position** of the source and target unfolding trees is an **actor** (repeated payloads counted separately by position).

**Theorem 9.2 (One joint program: complete time, common history, and the action-difference lifting criterion).** Let step$(v)$ lie in an aligned branch of Proposition 7.1 with tree $T$ from Proposition 7.6 (complete proof in Supplement S3.1):

1. **Two-sided temporal coding**: $\mathrm{dec} \circ \mathrm{enc} = \mathrm{id}$, $\mathrm{enc} \circ \mathrm{dec} = \mathrm{id}$, act is injective, and $\mathrm{enc}(v^+) = \mathrm{nx}(\mathrm{enc}\, v)$;
2. **Joint whole-tree charging and syntactic left inverse**: tagging each node of $T$ with $\mathrm{enc}(v)$ to form $T_J$, five-tuple constructor $\kappa_J$ folds $\tau(T)$, the receipt tree, and both sides' history/evaluator/exposure data; under $(\delta, \delta' - \delta)$, $\mathrm{prog}_{\kappa_J} T_J$ pays a strict trace of exact length $\beta(T_J) = \beta(T)$, syntax-inverts back to $T_J$ and $v$ at every count, and characterizes old-fibre preimages via vanishing lifting residual;
3. **Common history closure**: $\mathcal{H}_s$ and $\mathcal{H}_t$ generate a union common history with exact zero completion fibres on both sides, inducing an isometric measured evolution on $\ell^2(\mathcal{H} \times \mathcal{H})$ over $\mathrm{Src} \times \mathrm{Tgt}$;
4. **Dual-residual lifting criterion**: for node action outputs $a_s, a_t$, history transition $\iota$, and difference $d := a_t - \iota(a_s)$, there exists $r \in \mathrm{Fib}_{\partial_s}(a_s)$ with $\iota(r) = a_t$ **iff** both the cokernel image residual $[d] \in \mathrm{coker}\, \iota$ and observation residual $\partial_t a_t - \partial_s a_s \in \mathcal{H}$ vanish; a fixed 25-step program computes $(d, \partial d, [d])$ with strict per-step debit;
5. **Canonical direct successor**: the target occurrence of step$(v)$ equals the source occurrence of step$(v^+)$, equating target and next-source generator closures, relation closures, and full exposure trees. ■

## 9.2 All Actors: All-Word Model, Ordered Effect History, and Joint Future Recovery

Inspecting tree roots alone misses internal cancellations; assembling all source actors, target actors, and a simultaneous root letter into alphabet $L$ with combined readout rd recovers the full observable orbit structure (analogous to Kalman minimal realization[10] and coalgebraic final quotients[9], without field or finite-dimension hypotheses).

**Theorem 9.3 (All-word model and joint future recovery across all pairing-position actors).** Under the above data (complete proof in Supplement S3.2):

1. **All-word observable quotient**: setting $K := \{x : \forall w \in L^*,\ \mathrm{rd}(A_w x) = 0\}$, every $A_\ell$ descends to $\mathrm{Mod} := \mathrm{Car}/K$, two elements agree iff all word readouts agree, and Mod is linearly isomorphic to the word-readout completion carrier;
2. **Cancellation-proof ordered effect history**: for $w = \ell_1 \cdots \ell_n$ and $x$ ($x_k = A_{\ell_k} \cdots A_{\ell_1} x$, $\mathcal{U}_w = U_{\ell_n} \cdots U_{\ell_1}$), the total covariance residual telescopes into transported step residuals:

$$\mathcal{U}_w M(x) - M(A_w x) = \sum_{k=1}^{n} U_{\ell_n} \cdots U_{\ell_{k+1}} (U_{\ell_k} M(x_{k-1}) - M(x_k)),$$

   stored in a length-$n$ history retaining nonzero intermediate terms even when endpoint values cancel;
3. **Node $\ell^2$ isometry and three-face double inverse**: position-indexed measurement $\mathrm{meas} : \mathrm{Mod} \to \ell^2(\mathrm{positions}; \mathcal{H})$ satisfies $\|\mathrm{meas}(x)\|^2 = \sum_p \|M_p(x)\|^2$ with isometric letter evolutions; combining coarse and all future $\mathcal{H}$ readers is injective on Mod, the integral, coherent cofinal,

and coarse faces are jointly injective on the complete orbit carrier, and any query word $q$ executes in $|q| + 3$ strict steps without refill. ■

## 9.3 Universal Source-Program Admission, Two-Level Stock Crossing, and Word–State Covariance

**Definition 9.4 (Source-program configuration).** A **source-program configuration** $\mathcal{S}$ on an inquiry frame supplies a low-level variable family $L$, an optional projection, and a reader returning pair-valued inputs $(\rho, e)$ at each occurrence (with optional decoders).

**Theorem 9.5 (Universal source-program admission: paid queries generate the next obligation birth).** For any frame $\Phi$ and $\mathcal{S}$, with answered state $s$, occurrence $o$, and next query environment $\rho'$ (prior-projection and cursor-shift contracts in Supplement S4.1): (1) $o$'s material assembles on $\rho' = \rho + (\rho' - \rho)$ the residual request evaluating to the effect, whose positive budget $r(e) + r(e') + 2 > 0$ forces compilation to **obligation admission** at the same root and visit (preserving the first whole-ledger write); (2) the born frame launches Theorem 8.12's well-founded no-refill runtime with updated total environment (advancing macro history from tick $k$ to $k + 2$) while embedding both old stock and paid exposure into born stock. ■

**Theorem 9.6 (Two-level typed inventories cross admission value-preservingly with event-wise inverse readback).** With physical input $(\rho_P, e_P)$, low-level input $(\rho_L, e_L)$, and constant binding $\sigma(x) := \mathrm{const}(\rho_P(x))$ (Supplement S4.2): (1) $\mathrm{ev}_{\rho_L}(e[\sigma]) = \mathrm{ev}_{\rho_P}(e)$ and substituted paid traces have length $r(e_P[\sigma])$; (2) registered material crosses admission heterogeneously via projection-type equality; (3) $\sigma$-substituted physical stock joins low-level stock with injective coordinate maps under both math and birth steps; (4) the born-stock word family's canonical coimage residual vanishes iff its result value vanishes. ■

**Theorem 9.7 (Bilinear word–state covariance and iterated prefix charging).** For state $s$ with $\rho_s, \varepsilon_s = \rho_{\sigma(s)} - \rho_s$ and orbit binding $\sigma$ reading $\rho_{\sigma(s)}$ and $\varepsilon_{\sigma(s)}$ (Supplement S4.3): (1) free extension defines the $\mathbb{Z}$-bilinear pairing $\langle w, \delta_s \rangle := (\mathrm{ev}_{\rho_s} w,\ \mathrm{eff}_{\rho_s, \varepsilon_s} w)$; (2) $\langle \sigma^* w, u \rangle = \langle w, Au \rangle$ holds across the full carrier, inducing coimage time advance $\mathrm{act}^n[e] = [e[\sigma]^n]$; (3) prefixes $0, \ldots, n$ have individual trace sum $\Sigma_n = \sum_{i \le n} r(e[\sigma]^i)$ and vector-batch budget $\Sigma_n + 2(n+1)$, with finite snapshots recovered from call frames. ■

## 9.4 Default Run, Order-Preserving Stock Injection, Six-Face Complexes, and the Maximal Future Kernel

**Theorem 9.8 (The default run executes the source-generated request together with old and next subtrees).** From Theorem 9.2's joint root and reader (Supplement S5.1): (1) at each count, the default runtime executes that occurrence's source-generated request in $r(\text{input})$ steps; (2) one main expression combines query, old-code subtree, and next-code subtree via three nodes to evaluate $(\mathrm{ev}(\text{query}),\ \mathrm{ch}(c_{\text{old}}) + \mathrm{ch}(c_{\text{next}}))$ at exact syntax budget $r(\text{query}) + c_{\text{old}} + c_{\text{next}} + 3$; (3) subtree environments read directly from the updated born environment. ■

Projecting to frames before recovering formal words would collapse distinct ticks sharing equal readings; we therefore **recover the complete formal word first and restrict to frames**

**afterwards**, injecting every paid trace slot strictly monotonically into born stock so equal-valued intermediate events at distinct steps never merge:

**Theorem 9.9 (Whole-word recovery prior to frame readout and strictly order-preserving trace-position injection into born stock, C40).** For sealed runtime $\mathcal{R}$, full carrier $\mathcal{F} = \mathbb{Z}[\mathcal{R}.\mathrm{State}]$, complete-model quotient $q$, recovery rec, and linear frame reader $O : \mathcal{F} \to \mathbb{Z}[\mathrm{Frame}]$ (Supplement S4.4):

1. **Lossless whole-word recovery and injective frame advance**: $\mathrm{rec}(q(w)) = w$ for every $w \in \mathcal{F}$, and at tick $i$ under model action $A$,

$$O\,\mathrm{rec}(q(\delta_{s_i})) = \delta_{\Phi_i}, \qquad O\,\mathrm{rec}(Aq(\delta_{s_i})) = \delta_{\Phi_{i+1}}, \tag{9.1}$$

   recovering current, paid state, raw input, trace, decoder, and both inventories with model points injective in $i$;
2. **Strictly order-preserving trace-position injection**: with old and physical trace lengths $L_{\mathrm{old}}, L_{\mathrm{phys}}$, slot $j$ of the query's trace exposure embeds into born stock $H_B$ via $\iota(j) = j + L_{\mathrm{old}} + L_{\mathrm{phys}} + 3$ (offset 3 from three side-by-side seed seams) with $H_B.\mathrm{trace}[\iota(j)] = H_Q.\mathrm{trace}[j]$, keeping equal-valued events at distinct positions separate (while initial-visit and new-current readers remain distinct along their respective current identities, Supplement S4.4). ■

**Theorem 9.10 (Cofinal admission-packet stops and complete intermediate paid-tick segments in the same runtime, C43).** Computing the `Type` distance $d(t)$ to the next admission packet defines stops (Supplement S5.3, Figure 7A)

$$t_0 := d(0), \qquad \ell_j := 1 + d(t_j + 1), \qquad t_{j+1} := t_j + \ell_j, \qquad A_{\mathcal{R}}^{\ell_j} W_{t_j} = W_{t_{j+1}}, \tag{9.2}$$

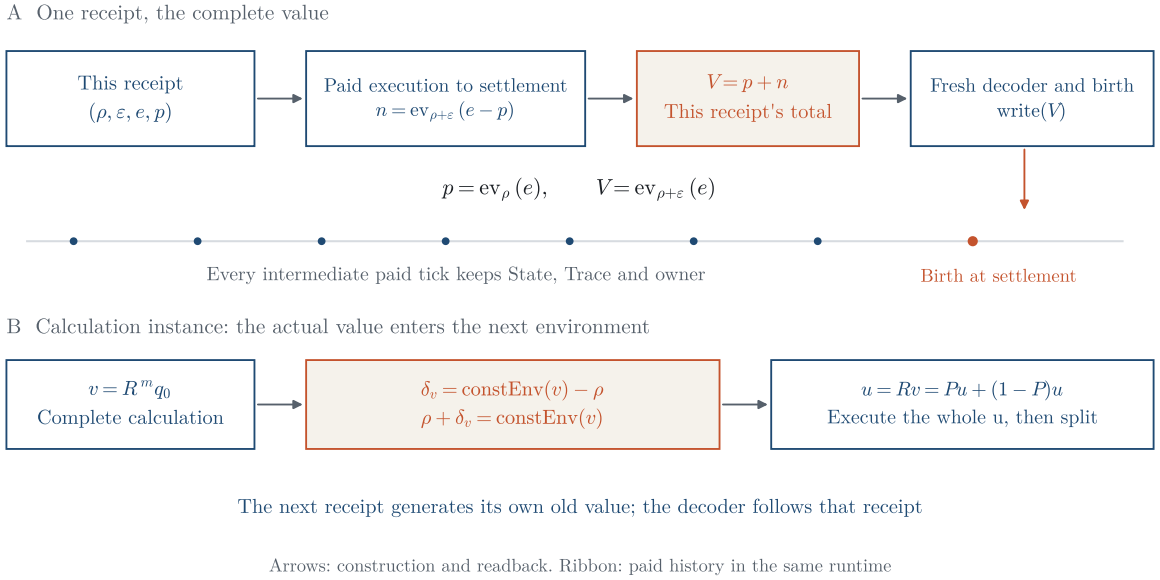
which are strictly increasing and cofinal ($t_j \geq j$), cover every intermediate paid tick $t_j, \ldots, t_j + \ell_j$ in the same runtime, and advance the complete field word $W_t$ across packet grades with zero external packet tables. ■

**Theorem 9.11 (Six faces of one actual paid boundary and retention across birth, C44).** At packet $j$ with paid material $M_j$, raw $e_j$, paid endpoint $e'_j$, trace $T_j$, prefix history $H_j$, and boundary $b_j := \delta_{e_j} - \delta_{e'_j}$, one $M_j$ simultaneously generates the **algebraic**, **combinatorial**, **topological**, **logical**, **relation** $\sigma_j = (b_j, b_j \in H_j.\mathrm{relationClosure})$, and **cochain** faces (Supplement S3.3):

$$C_1 \xrightarrow{d_1} C_0 \xrightarrow{d_0} \widehat{C_0}, \qquad d_1(\sigma_j) = b_j, \qquad d_0(d_1(\sigma_j)) = 0, \tag{9.3}$$

whose updated effect equals the residual request value, whose whole-word coimage recovery satisfies $\mathrm{rec}(\mathrm{can}(w)) = w$, and whose $(b_j, \sigma_j, T_j)$ survive across birth distinct from the next packet's newly normalized $\sigma_B$. ■

**Theorem 9.12 (Original whole-word action on the direct-sum carrier and the maximal future-invisible invariant kernel model, C45).** Over packet formal-word modules $\mathcal{W}_j$ and finitely supported direct sum $\mathcal{W} := \bigoplus_{j \in \mathbb{N}} \mathcal{W}_j$, pair lifting and packet transport define $S_j : \mathcal{W}_j \to \mathcal{W}_{j+1}$ and $S(\sum_j \iota_j(w_j)) := \sum_j \iota_{j+1}(S_j w_j)$, with site readouts $D_j(w) := d_j(w_j) =$




**Caption**. Figure 7 | One receipt recovers the complete total; the next receipt regenerates old value and decoder (structural diagram; Theorems 8.15, 9.10, §10.4). A: present material supplies $p = \mathrm{ev}_{\rho}(e)$; at settlement its native residual value $n = \mathrm{ev}_{\rho+\varepsilon}(e - \mathrm{const}(p))$ combines into $V = p + n$, written by a fresh decoder, while only the settled branch triggers birth and the ribbon retains all intermediate paid ticks in the same runtime. B: the complete comb result $v = R^m q_0$ generates $\delta_v = \mathrm{constEnv}(v) - \rho$ for the next environment, where the full operator $R$ executes $u = Rv$ before splitting into $Pu$ and $(1 - P)u$; vermilion marks total-value assembly and value-driven environment update.

$(\mathrm{ev}_{\rho_j} w_j, \mathrm{eff}_{\rho_j,\varepsilon_j} w_j)$ and allowed future words $a \in \{*\}^{<\omega}$ ($S_{[]} = \mathrm{id}$, $S_{a*} = S \circ S_a$). Setting (Supplement S5.4)

$$K_{\mathrm{fut}} := \bigcap_{a \in \{*\}^{<\omega}} \bigcap_{j \in \mathbb{N}} \ker(D_j \circ S_a), \qquad \mathsf{M}_{\mathrm{fut}} := \mathcal{W}/K_{\mathrm{fut}}, \tag{9.4}$$

we have: (1) **receiver square**: $\mathrm{ev}_{\rho^{\mathrm{rec}}_{j+1}}(S_j w) = d_j(w)$ without assuming $\rho_{j+1} = \rho_j$, preserving stock membership; (2) **all-future fibre criterion**: $[u] = [v]$ in $\mathsf{M}_{\mathrm{fut}}$ iff all future-word readouts agree at every site; and (3) **maximality**: $K_{\mathrm{fut}}$ is the **largest submodule** of $\mathcal{W}$ invisible at all sites now and invariant under $S$, projecting back to current joint completion at $a = [\,]$ compatibly with $\bar{S}$. ■

**Example 9.13 (Concrete two-packet check of order-preserving stock addresses, cofinal stops, and the maximal future kernel).** Return to Example 1.1(b) over $W = \mathbb{Z}$ with $e_0 = x + x$, initial environment $\rho_0(x) = 3$, next-packet environment $\rho_1(x) = 5$ ($\varepsilon_0 = +2$), and empty initial stocks ($L_{\mathrm{old}} = L_{\mathrm{phys}} = 0$):

1. **Order-preserving stock slots (Theorem 9.9)**: packet 0 executes $r(e_0) = 3$ primitive steps ($j = 0, 1, 2$), mapped by $\iota(j) = j + 0 + 0 + 3 = j + 3$ to born-stock slots $\iota(0) = 3$, $\iota(1) = 4$, $\iota(2) = 5$; even in the degenerate case $\rho_0(x) = 0$ where the first two steps have identical endpoint value 0, the two variable-binding events occupy distinct slots $3 \neq 4$ rather than merging, recovered only because $\mathrm{rec}(q(w)) = w$ precedes frame projection.
2. **Cofinal stop recursion (Theorem 9.10)**: packet 0 settles at distance $t_0 = d(0) = r(x+x) = 3$ (ticks $0, 1, 2$ debit $3 \to 2 \to 1 \to 0$; tick 3 settles at $e'_0 = \mathrm{const}(6)$ and births $e_1 = e_0 \ominus \mathrm{const}(6)$

at tick 4); tick 4 has remaining distance $d(4) = r(e_0 \ominus \mathrm{const}(6)) = 3 + 0 + 2 = 5$, giving span $\ell_0 = 1 + d(4) = 6$ and second stop $t_1 = t_0 + \ell_0 = 3 + 6 = 9$, with $A^6_{\mathcal{X}} W_3 = W_9$ covering every intermediate tick $3, 4, \ldots, 9$ inside one runtime.

3. **Six-face boundary vs. maximal future kernel (Theorems 9.11–9.12)**: at stop 0, boundary $b_0 = \delta_{x+x} - \delta_{\mathrm{const}(6)}$ obeys $d_0(d_1(\sigma_0)) = 0$ since $\mathrm{ev}_{\rho_0}(b_0) = 6 - 6 = 0$, yet its paired site reading is $d_0(b_0) = (\mathrm{ev}_{\rho_0} b_0,\ \mathrm{eff}_{\rho_0,+2} b_0) = (0, 4) \neq (0, 0)$, so $\iota_0(b_0) \notin K_{\mathrm{fut}}$ (future shift immediately exposes its nonzero effect); conversely, the nonzero formal difference $z_0 := \delta_{(x+x)\ominus\mathrm{const}(6)} - b_0 \in \mathcal{W}_0$ satisfies $\mathrm{ev}_\rho(z_0) = (2\rho(x) - 6) - (2\rho(x) - 6) = 0$ and $\mathrm{eff}_{\rho,\varepsilon}(z_0) = 2\varepsilon(x) - 2\varepsilon(x) = 0$ for **all** $(\rho, \varepsilon)$, so $D_j(S_a(\iota_0(z_0))) = (0, 0)$ across every $a \in \{*\}^{<\omega}$ and site $j$, exhibiting a concrete nonzero element $\iota_0(z_0) \in K_{\mathrm{fut}}$ generalized in Theorem 10.1 below. ■

## 10 Reversible Incremental Continuation, Cross-Domain Instantiations, and Four-Axis Literature Dialogue

Theorem 9.12 established the cross-packet action $S$ and maximal future model $\mathsf{M}_{\mathrm{fut}}$; turning this algebra into a **lossless, reusable, and bidirectionally invertible incremental kernel** closes three final problems:

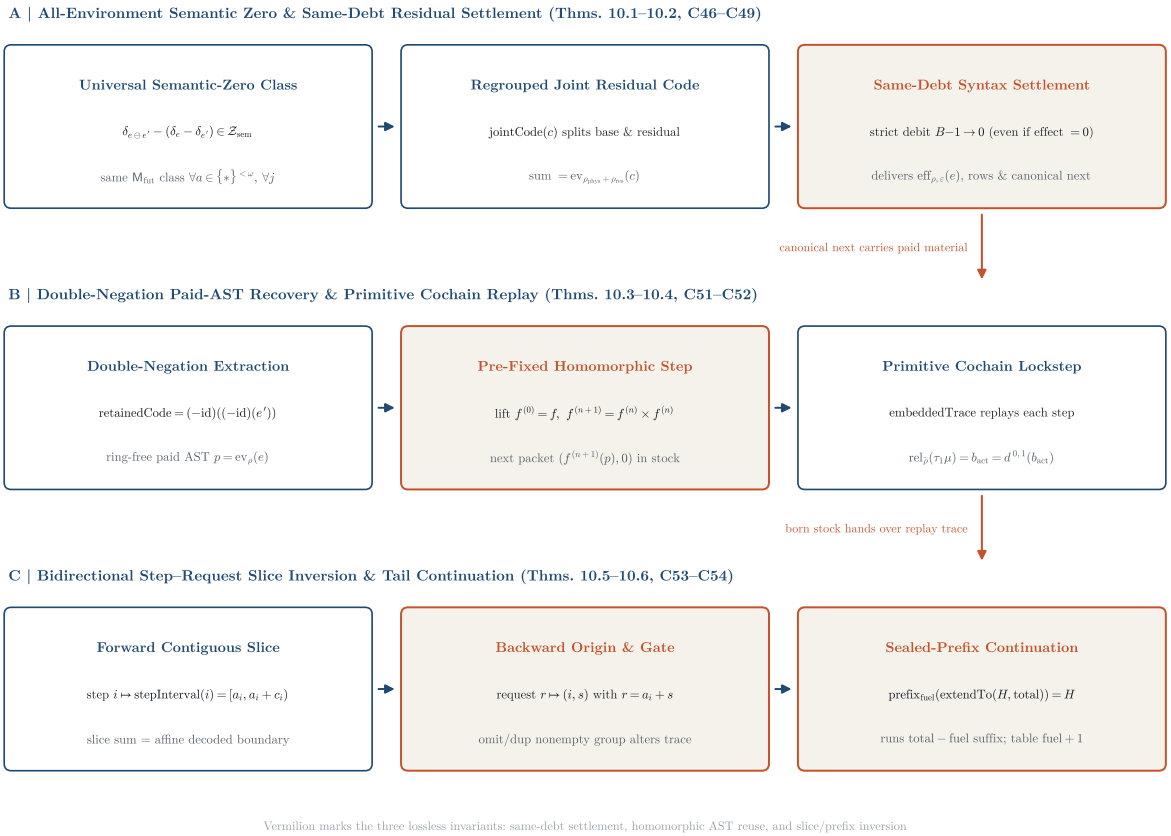
1. Does the executable residual request $e \ominus e'$ represent the **exact same equivalence class** in $\mathsf{M}_{\mathrm{fut}}$ as the algebraic trace boundary $\delta_e - \delta_{e'}$ across all future action words and sites, transporting directly to a writable request in canonical next while regrouping full retained residuals and settling same-debt syntax credits (§10.1, C46–C49)?
2. Once a paid constant AST is wrapped inside two-layer residual syntax, can it be extracted in place **without ring scalars or memo tables** to execute any pre-fixed additive homomorphism $f$ across cofinal stops, while primitive-by-primitive replay updates relation boundaries and cochain differentials in lockstep (§10.2, C51–C52)?
3. Do executed primitive steps and next-batch signed requests admit **gap-free bidirectional slice inversion** (rejecting omission or duplication of any nonempty step group), and does extending a stored history from fuel to total $\geq$ fuel lock the entire paid prefix while running **only the missing tail suffix** (§10.3, C53–C54)?

Subsections 10.1–10.3 establish all three affirmatively (Figure 8; complete proofs in Supplement S6) and trace a concrete numerical run in Example 10.7; §10.4 presents cross-domain instantiations (C41, C50, and CPS1); and §10.5 conducts our four-axis literature dialogue.

### 10.1 Universal Semantic-Zero Congruence, Regrouped Residual Evaluation, and Same-Debt Settlement (C46–C49)

Define the **all-environment semantic-zero submodule** of formal words vanishing under every environment assignment:

$$\mathcal{Z}_{\mathrm{sem}}(\kappa) := \{ w \in \mathrm{Fm}_{\mathbb{Z}}(\kappa) \mid \forall \rho \in \mathrm{Env},\ \mathrm{ev}_\rho(w) = 0 \}.$$




**Caption**. Figure 8 | Reversible incremental continuation: universal semantic-zero congruence, double-negation paid-AST reuse, and bidirectional step–request slice inversion (structural diagram; Theorems 10.1–10.6, C46–C49, C51–C54). A: the formal difference $\delta_{e\ominus e'} - (\delta_e - \delta_{e'})$ lies in the all-environment semantic-zero submodule $\mathcal{Z}_{\mathrm{sem}}$, sharing one $\mathsf{M}_{\mathrm{fut}}$ equivalence class across all future action words $a \in \{*\}^{<\omega}$ and sites $j$; regrouped jointCode$(c)$ evaluates base and residual inputs jointly while strict syntax debit $B{-}1 \to 0$ (charged even when the semantic effect vanishes) delivers $\mathrm{eff}_{\rho,\varepsilon}(e)$, whole-ledger row destinations, and canonical next. B: retainedCode applies ring-free double negation $(-\mathrm{id})((-\mathrm{id})(e'))$ to recover paid AST $p = \mathrm{ev}_\rho(e)$, driving graded homomorphism $f^{(n+1)}$ to evaluate $(f^{(n+1)}(p), 0)$ into born stock at the next cofinal stop while embeddedTrace keeps substitution $\tau_1$, actual boundary $b_{\mathrm{act}}$, and cochain differential $d^{0,1}$ in lockstep. C: forward stepInterval maps step $i$ to contiguous slice $[a_i, a_i + c_i)$, backward readRequestOrigin inverts any request $r$ via $r = a_i + s$ (omitting or duplicating any nonempty group alters the native trace), and extendTo locks the paid prefix while running only the total − fuel suffix (single-pass timeline length fuel + 1).

Unlike a single-environment kernel $\ker(\mathrm{ev}_{\rho_0})$ (Example 4.2 showed $x^2-x \in \ker(\mathrm{ev}_1)\setminus\ker(\mathrm{ev}_2)$), membership in $\mathcal{Z}_{\mathrm{sem}}$ survives every environment shift and every iterated pair lift.

**Theorem 10.1 (Universal semantic-zero congruence in the maximal future model and writable request in canonical next, C46).** For any graded source-program family, initial inquiry frame, program configuration, language equality, and cofinal stop $j$ with raw expression $e_j$ and paid endpoint $e'_j$ (complete proof in Supplement S6.1):

1. **Universal semantic-zero closure and future-model equivalence**: the formal difference between the residual request generator and the trace boundary lies in $\mathcal{Z}_{\mathrm{sem}}$ and remains closed under update inventories, iterated pair lifts $\widetilde{(\cdot)}$, and packet/language transports:

$$\begin{aligned} &\delta_{e_j\ominus e'_j} - (\delta_{e_j} - \delta_{e'_j}) \in \mathcal{Z}_{\mathrm{sem}} \\ \Longrightarrow \quad &\forall a \in \{*\}^{<\omega},\ \forall m \in \mathbb{N},\ \ D_m\Big(S_a(\iota_j(\delta_{e_j\ominus e'_j}) - \iota_j(\delta_{e_j} - \delta_{e'_j}))\Big) = (0,0), \end{aligned} \tag{10.1}$$

   identifying the residual request and trace boundary as the **exact same equivalence class** in $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$;
2. **Writable request in canonical next and dual row destinations**: Theorem 6.12's selected receipt drives settled birth compilation so that actual cofinal stop $j+1$'s current equals the settlement target's canonical next current; transporting the full paid material (environment, AST, state, trace, and bearer) along that identity generates a writable native request in packet $j+1$ in the same $\mathsf{M}_{\mathrm{fut}}$ class while supplying both old and born whole-ledger row destinations from the single write.

Proof idea. In every $\rho$, $\mathrm{ev}_\rho(\delta_{e\ominus e'}) = \mathrm{ev}_\rho(e) - \mathrm{ev}_\rho(e') = \mathrm{ev}_\rho(\delta_e - \delta_{e'})$. Whenever $w \in \mathcal{Z}_{\mathrm{sem}}$, Proposition 3.3 gives $0 = \mathrm{ev}_{\rho+\varepsilon}(w) = 0 + \mathrm{eff}_{\rho,\varepsilon}(w)$, so $U_{\rho,\varepsilon}(w) = (0,0)$ for all $(\rho,\varepsilon)$; decomposing any paired environment into $(\rho,\varepsilon)$ yields $\widetilde{w} \in \mathcal{Z}_{\mathrm{sem}}$. Induction on $|a|$ keeps $S_a$ inside site-wise $\mathcal{Z}_{\mathrm{sem}} \subseteq \ker D_m$, proving membership in $K_{\mathrm{fut}}$ (Supplement S6.1). ■

**Theorem 10.2 (Cross-cofinal code regeneration, regrouped residual evaluation, and strict same-debt settlement, C47–C49).** Given a paired-carrier source code $c \in \mathrm{Expr}(V \times V)$ and graded source-program family (complete proof in Supplement S6.2):

1. **Regrouped evaluation on full retained residuals (C47)**: setting $\mathrm{regroup}((u_{11},u_{12}),(u_{21},u_{22})) := ((u_{11}+u_{21},0),(u_{12}+u_{22},0))$ and $\mathrm{jointCode}(c) := (\widetilde{c})[x \mapsto \mathrm{linear}_{\mathrm{regroup}}(\mathrm{var}\, x)]$, evaluating on any two-layer environment $\Theta$ with embedded base input $\rho_{\mathrm{phys}}(x) := (\pi_1(\Theta(x)_1 + \Theta(x)_2), 0)$ and retained residual input $\rho_{\mathrm{res}}(x) := (\pi_2(\Theta(x)_1 + \Theta(x)_2), 0)$ yields (written back recursively across cofinal packets via Theorem 8.15 while preserving query and stock maps, Supplement S6.2)

$$\mathrm{ev}_\Theta(\mathrm{jointCode}(c)) = (\mathrm{ev}_{\rho_{\mathrm{phys}}}(c),\ \mathrm{eff}_{\rho_{\mathrm{phys}},\rho_{\mathrm{res}}}(c)), \qquad \pi_1 + \pi_2 = \mathrm{ev}_{\rho_{\mathrm{phys}}+\rho_{\mathrm{res}}}(c); \tag{10.2}$$

2. **Shared-provenance raw input and linear receiver continuation (C48)**: complete material and the action-reader raw input Raw share one projection restriction; $\rho_{\mathrm{Raw}}$ and decoded $\rho_{\mathrm{Born}}$ generate $\delta = \rho_{\mathrm{Born}} - \rho_{\mathrm{Raw}}$ whose pair-lift coordinates sum to $\mathrm{ev}_{\rho_{\mathrm{Born}}}(e_{\mathrm{Raw}})$, while linear homomorphic receivers re-read the common source at each occurrence to output initial result plus actual effect;

3. **Strict same-debt syntax debit and completed answer delivery (C49)**: after step 1 at admission, remaining syntax credit $B - 1 \geq 1$ executes strict same-debt debits without refill across every paid branch (paying full syntactic cost even when $\mathrm{eff}_{\rho,\varepsilon}(e) = 0$), reaching the completed current with value $\mathrm{eff}_{\rho,\varepsilon}(e)$ and compiling to deliver the completed answer, receipt, old/born row destinations, and canonical next. ■

## 10.2 Ring-Free Double-Negation Paid-AST Recovery, Homomorphic Continuation, and Primitive Cochain Replay (C51–C52)

Over general additive abelian groups $W_s$ (requiring no ring unit 1 or scalar multiplication), a settled endpoint $\mathrm{const}(p)$ enters the next frame wrapped inside $e \ominus \mathrm{const}(p) = \mathrm{add}(e, \mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(p)))$ (or with an incoming summand $\mathrm{add}(\mathrm{add}(e, \mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(p))), g)$). Theorem 10.3 extracts the paid AST in place via purely syntactic double negation to run any pre-fixed additive homomorphism $f : W_s \to W_s$ across every cofinal stop.

**Theorem 10.3 (Ring-free double-negation paid-AST extraction and cross-cofinal homomorphic continuation, C51).** For any additive group family $W$, pre-fixed additive homomorphism $f : W_s \to W_s$, and graded pair groups $V_0 = W$, $V_{n+1} = V_n \times V_n$ with $f^{(0)} := f$, $f^{(n+1)} := f^{(n)} \times f^{(n)}$, define on level-$n$ frame $\Phi$ the syntactic extractor

$$\mathrm{retainedCode}_n(\Phi) := \begin{cases} \mathrm{linear}_{-\mathrm{id}}(\mathrm{paid}), & \text{if input is } \mathrm{add}(\mathrm{add}(_, \mathrm{paid}), _) \text{ or } \mathrm{add}(_, \mathrm{paid}), \\ e_{\mathrm{src}}, & \text{otherwise } e_{\mathrm{src}}. \end{cases} \tag{10.3}$$

Executing $\mathrm{linear}_{f^{(n+1)}}(\widetilde{\mathrm{retainedCode}}_n(\Phi))$ in zero-increment paired environment $(\rho_\Phi, 0)$ satisfies at every cofinal packet $P_n$ (Supplement S6.3):

1. **Ring-free recovery of the paid AST**: registered input stays invariant across Theorem 6.12's paid prefix; once packet $P_n$'s paired action settles at $p \in V_{n+1}$, crossing birth yields $\mathrm{paid} = \mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(p))$ so $\mathrm{retainedCode}_{n+1}$ equals heterogeneously $\mathrm{linear}_{-\mathrm{id}}(\mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(p)))$, evaluating in every environment to $-(-p) = p$ (on a level-0 frame, direct method input sends $p_0 \in V_0$ to $f^{(1)}(p_0, 0) = (f(p_0), 0) \in V_1$);
2. **Cross-cofinal homomorphic action and full trace stock**: packet $P_{n+1}$ executes $\mathrm{linear}_{f^{(n+2)}}(\widetilde{\mathrm{retainedCode}}_{n+1})$ on its level-$(n+1)$ frame to evaluate to $(f^{(n+1)}(p),\ 0) \in V_{n+2}$, writes its entire query trace into born stock writtenStock via the stock-preservation map, installs four faces, dual readback, and cochains prior to emission, and extends across any finite segment of the sealed field runtime. ■

Beyond final values, **every intermediate primitive step** of a source expression replays natively into the carrier environment, driving relation boundaries, cochain differentials, and coimage actions in exact lockstep (C52):

**Theorem 10.4 (Whole-word primitive-by-primitive native replay and lockstep stock–boundary–cochain differential update, C52).** Given source process $\mathcal{P}$ and readEnv : $S \to \mathrm{Env}$ freely lifted to observer environment $\rho_w^{\mathrm{obs}}$ (recovering $\mathrm{readEnv}(s_t)$ at $w = \delta_{s_t}$), with context embedding embed and orbit binding $\sigma$ (Supplement S6.4):

1. **Step-wise native embedding and exact substitution cost**: embeddedStep and embeddedTrace translate each primitive step $s : e_1 \to e_2$ into a native trace whose substituted zero-effect lift pairedForward has exact length $r(\mathrm{embed}(e)[\sigma])$;
2. **Born-stock write and next-query retention**: at every count $k$ of the sealed runtime, replay exposure and paid trace enter birthStock and persist inside the next born frame's complete written query trace;
3. **Four-way lockstep across chain map, boundary, cochain differential, and coimage**: for any integral source relation word $\mu_k$, substitution chain map $\tau_1$, actual boundary $b_{\mathrm{act},k} \in X^0$ in the next relation closure, natural degree $0 \to 1$ cochain differential $d^{0,1} : X^0 \hookrightarrow X^1$ (submodule inclusion into the generator closure), and completion projection $\pi_{\mathrm{comp}}$ on $X^1$, the underlying words and quotient classes satisfy

$$\begin{aligned} &\mathrm{rel}_{\tilde{\rho}_k}(\tau_1(\mu_k)) = b_{\mathrm{act},k} = d^{0,1}(b_{\mathrm{act},k}), \qquad \pi_{\mathrm{comp}}(d^{0,1}(b_{\mathrm{act},k})) = 0, \\ &\mathrm{act}_{\mathrm{coim},k}([\mathrm{rel}(\mu_k)]_{k+1}) = [b_{\mathrm{act},k}]_k, \end{aligned} \tag{10.4}$$

   while projecting the rooted unfolding recovers rootRead identically. ■

## 10.3 Bidirectional Step–Request Slice Inversion and Sealed-Prefix Tail Continuation (C53–C54)

Checking batch sums alone when handing signed requests to a next stage cannot detect omitted cancelling pairs or duplicated slices. Theorem 10.5 establishes **gap-free bidirectional slice inversion** between executed primitive steps and next-batch signed requests (achieving exact addressability via monotone prefix offsets without Bennett's reverse uncomputation[4]), while Theorem 10.6 locks paid prefixes when extending execution horizons.

**Theorem 10.5 (Bidirectional slice inversion between executed primitive steps and next-batch signed requests with anti-tamper gates, C53).** For a graded source-program family, admission packet $P$, and birth execution $E$ whose $M$ paid-step relation words expand under $S_P$ into next-batch signed request list $\mathcal{B}$ of length $N = |\mathcal{B}|$ (Supplement S6.5):

1. **Forward contiguous step slice** (stepInterval): every executed step $i \in \{0, \dots, M-1\}$ determines a contiguous interval $[a_i,\, a_i + c_i) \subseteq \{0, \dots, N\}$, where $a_i$ is the expanded request length of the first $i$ steps and $c_i$ is the signed support length of $S_P w_i$; the slice $\mathcal{B}[a_i : a_i + c_i)$ equals step $i$'s own request list, whose request words $\mathrm{word}(q) := a(\delta_e - \delta_{\mathrm{const}(\mathrm{ev}_{\rho_{\mathrm{dec}}} e)})$ sum under stock-decoded environment $\rho_{\mathrm{dec}}$ to the affine boundary $\mathrm{affine}_{\rho_{\mathrm{dec}}}(S_P w_i) := S_P w_i - q_{\rho_{\mathrm{dec}}}(S_P w_i)$ (with $q_\rho$ the constant formal sum of Theorem 4.4);
2. **Backward unique origin inversion** (readRequestOrigin): conversely, every request ordinal $r \in \{0, \dots, N-1\}$ constructively recovers without external lookup tables its unique originating step $i \in \{0, \dots, M-1\}$, local support slot $s \in \{0, \dots, c_i - 1\}$, signed coefficient, and prefix split

obeying

$$r = a_i + s, \qquad \bigsqcup_{i=0}^{M-1} \mathcal{B}[a_i : a_i + c_i) = \mathcal{B}, \qquad \sum_{i=0}^{M-1} \sum_{q \in \mathcal{B}[a_i : a_i + c_i)} \mathrm{word}(q) = \sum_{q \in \mathcal{B}} \mathrm{word}(q); \quad (10.5)$$

3. **Full-batch splice and anti-tamper negative-control gate**: concatenating $\mathcal{B}[a_i : a_i + c_i)$ in step order (retaining empty slices in place) recovers $\mathcal{B}$, executedTrace($\mathcal{B}$), and the joint cochain boundary; for any step with a nonempty slice ($c_i > 0$), **omitting** or **duplicating** that slice strictly alters the native executed trace. ■

**Theorem 10.6 (Sealed-prefix rigidity, tail-only incremental continuation, and single-pass timeline table, C54).**

1. **Prefix-locked tail continuation**: for any deterministic inquiry runtime $\mathcal{R}$, start $s_0$, and stored history $H \in \mathsf{HistoryAt}(\mathcal{R}, \mathrm{fuel}, s_0)$, stored lookup equals active lookup at all $i < \mathrm{fuel}$ ($H.\mathrm{storedOccurrenceAt}(i) = H.\mathrm{occurrenceAt}(i)$); extending to any horizon total $\geq$ fuel via extendTo reuses all stored activations without recomputation, **executes solely the missing** total $-$ fuel **suffix from the reached tail state**, and obeys (Supplement S6.6)

$$\mathrm{prefix}_{\mathrm{fuel}}(\mathrm{extendTo}(H, \mathrm{total})) = H; \quad (10.6)$$

2. **Single-pass timeline table and stage-inventory recovery**: on any living-root micro-history $H_{\mathrm{mic}} \in \mathsf{SourceNativeLivingRootHistoryAt}(\mathcal{P}, \mathrm{fuel}, s_0)$ with projection $\pi : S \to M$, storedTimeline pre-allocates capacity fuel $+ 1$ and projects each stored state once to build $T \in M^{\mathrm{fuel}+1}$ whose adjacent pairs $(T[i], T[i+1])$ recover the exact stage inventory $U_i^{s_0}$ of Theorem 4.6 at every $i < \mathrm{fuel}$. ■

We verify the entire chain of Theorems 10.1–10.6 on a concrete arithmetic run.

**Example 10.7 (End-to-end hand check of reversible continuation: double-negation homomorphism, slice inversion, and tail continuation).** Over $W = \mathbb{Z}$, take $e = \mathrm{var}\, x + \mathrm{var}\, x$, $\rho_0(x) = 3$, and tripling homomorphism $f(v) = 3v$.

1. **Packet-0 execution and semantic-zero congruence (Theorems 10.1–10.2)**: $e$ reduces in $M = 3$ steps—step 0 left bind $\delta_{x+x} \to \delta_{\mathrm{const}(3)+x}$, step 1 right bind $\to \delta_{\mathrm{const}(3)+\mathrm{const}(3)}$, step 2 constant sum $\to \delta_{\mathrm{const}(6)}$—settling at $e' = \mathrm{const}(6)$ ($p = 6$). Shifting to $\rho_1(x) = 5$ ($\varepsilon = +2$), both $e \ominus \mathrm{const}(6)$ and $\delta_{x+x} - \delta_{\mathrm{const}(6)}$ evaluate to $2\rho(x) - 6$ in **every** environment $\rho$, so their difference lies in $\mathcal{Z}_{\mathrm{sem}}$ and shares one $\mathsf{M}_{\mathrm{fut}}$ class; $e \ominus e'$ pays 5 strict syntax steps to compute $n = 10 - 6 = 4$, and Theorem 8.15 writes $V = 6 + 4 = 10 = \mathrm{ev}_{\rho_1}(e)$.
2. **Double-negation homomorphic continuation (Theorem 10.3)**: on the level-0 frame, $\mathrm{retainedCode}_0$ extracts $\mathrm{linear}_{-\mathrm{id}}(\mathrm{linear}_{-\mathrm{id}}(\mathrm{const}(6)))$ evaluating to $p_0 = -(-6) = 6 \in V_0$; zero-increment pair lifting to $(6, 0) \in V_1$ under $f^{(1)} = 3\mathrm{id} \times 3\mathrm{id}$ computes $(f(6), 0) = (18, 0) \in V_1$ into born stock (advancing via $f^{(2)}$ across the next packet).
3. **Bidirectional slice inversion (Theorem 10.5)**: each of the $M = 3$ step differences has $c_i = 2$ signed support items (e.g., $+1 \cdot (x + x)$ and $-1 \cdot (\mathrm{const}(3) + x)$ at step 0), giving batch length $N = 6$ and slices $[0, 2), [2, 4), [4, 6)$. Request ordinal $r = 5$ inverts via readRequestOrigin(5) uniquely to step $i = 2$, slot $s = 1$ ($5 = 4 + 1$, corresponding

to $-1 \cdot \text{const}(6)$ when listing step endpoints in reduction order); summing the three slices telescopes to $\delta_{x+x} - \delta_{\text{const}(6)}$, whereas omitting or duplicating any length-2 slice breaks trace equality.

4. **Sealed-prefix tail continuation (Theorem 10.6)**: from a stored fuel $= 2$ history $H_2$ ending at $\rho_2$, extendTo$(H_2, 5)$ locks ticks $0, 1$, executes only $5 - 2 = 3$ suffix ticks from $\rho_2$, and fills a single-pass table of $5 + 1 = 6$ cells whose adjacent differences $T[i+1] - T[i]$ yield all 5 stage inventories. ■

## 10.4 Three Cross-Domain Instantiations (C41, C50, and Arithmetic, Physical, and Molecular Systems)

**Group 1: Fock native histories, whole-tree charging, and stored timelines in canonical arithmetic.** A particle–wave Fock runtime lifts native field words into $\mathbb{Z}[S]$, discharges the target-6 partition debt by invoking Proposition 3.6, Theorems 4.4 and 4.6–4.7, Proposition 4.8, and Proposition 6.5 (Table 2), executes and syntax-inverts unit-history trees via Theorem 3.9, drives integral cochains and homomorphic methods via whole-word primitive replay (Theorem 10.4), and feeds fixed-target additive micro-actor stages from a length-(fuel + 1) single-pass storedTimeline with provably distinct adjacent bind cells $T[0] \neq T[1]$ (Theorem 10.6).

**Group 2: Riesz resolvent combs, four-trace finite windows, and inverse-residual correction (C41, C50, C53).** Given a right-half-plane nontrivial zero observation ($\operatorname{Re}\zeta > 1/2$, exact $\Xi$-order $m$, $z = \zeta/2$ with $\operatorname{Re} z > 1/4$, and comb $q_0$), $e_k = \text{linear}_R^k(x)$ executes in $m+1$ steps prior to emission to produce $v = R^m q_0$ (C41, Figure 7B), generating $\delta_v = \text{constEnv}(v) - \rho$ and writing $v + (R^m v - v) = R^m v$ via Theorem 8.15. On paired input $(v, 0)$, the 3-step physical program **computes the full ambient $u = R_z v$ before splitting orthogonally into** $(p, r) = (Pu, (1-P)u)$, yielding for $1/4 < \operatorname{Re} w < 1/2$ the complete Mellin pricing identity (Supplement S5.5):

$$H_w(p) + 2M_w(r) = -\frac{H_w(v)}{z + w - 1/2}.$$

In finite $P_a$-window operator actions (C50), Möbius source $\nu$ and both means execute cutoff pullbacks, reciprocal weights, and Tate poles as literal ASTs storing forward, inverse, second-mean, and cochain **four execution traces** in born stock; reading the first inverse residual then executes $\nu -$ residual under cutoff substitution to adjoin the correction trace **while retaining all four prior stock traces**, outputting source window price minus residual window price while keeping cutoff radius distinct from discrete evolution step indices. Grouped Γ-requests directly instantiate Theorem 10.5's stepInterval whole-ledger write and anti-omission/duplication gates.

**Group 3: Unified-field faithful realization and the 37-primitive whole-atom smooth CPS1 system.** In *From a Single Occurrence Source to a Continuous–Quantum Unified Field*, registering the self-sourced spin-pair root as a causal world invokes Proposition 5.2 and Theorem 5.3 to recover configuration admission, matter, action equations, and macro successors simultaneously. In the molecular instantiation (Supplement S5.6), 37 fixed primitive inputs $I$ execute translation, repair, resource response, and chemical exchange to form a `Type 0` body $B$ : InputBodyAt$(I)$ retaining DNA, coding, protein, nuclear/electronic fields, resource account, open responsibilities,

and history with $\mathrm{next}(\mathrm{phase}, I, B) = (\mathrm{phase}+1, I, \mathrm{bodyNext}_I(B))$. Smoothness of the full nuclear–electronic quartic energy at separated poses yields $\pi(0) = 0$ and $\lim_{t\to 0} \pi(t) = 0$ under half-step Cayley updates: for reserve $A > 0$, dyadic halving $t_k = t_0 2^{-k}$ selects via `Nat.find` the first ready $k_*$ with $\pi(t_{k_*}) \le A$, generating a charge- and $(E + A)$-conserving finite paid history; for $A = 0$, the first ready $k_*$ emits paid if $\pi(t_{k_*}) \le 0$ and explicit energy-shortage disposition if $\pi(t_{k_*}) > 0$, never fabricating payment from zero reserve (the clinical label indexes the source row only and claims no clinical efficacy).

## 10.5 Four-Axis Literature Dialogue: Incremental/Reversible Computation, Coalgebraic Refinement, Resource Type Theory, and Categorical Revision

Placing C1–C54 beside the primary literature turns on one sharp question: **when a process evolves, changes representation, discharges obligations, or revises its language, which verifiable invariants travel with the answer?** Table 5 summarizes the theorem-level comparison across twenty-four primary works along four axes.

**Table 5 Mechanism-by-mechanism comparison between source-native process algebra (v3) and twenty-four primary works.**

| Axis & representative works | Established classical mechanism | Inherited baseline & non-overclaim scope | New invariants & theorems locked in v3 |
|---|---|---|---|
| **Incremental, adaptive & change calculi** DBSP [1]; ILC [2]; Adaptive FP [22]; Tag 00CX [5] | Z-set $\mathbb{Z}[A]$ stream IVM $D \circ Q \circ I$ & 3-term join [1]; $\lambda$-differentiation [2]; modifiable-ref dynamic dependency graphs & memo tables [22] | Credits $\mathbb{Z}[S]$ Z-sets & 3-term update; no recursive SQL library, closure derivatives, or mutable-pointer DDG | Structureless state lift (Prop. 2.5, Def. 2.6, Prop. 2.7); `Type` exact kernel $\mathrm{range}(\mathrm{rel}_\rho) = \ker(\mathrm{ev}_\rho)$ (Thm. 4.4); completion evolution (Thm. 4.7); memo-free double-negation retainedCode & prefix-locked tail continuation (Thms. 8.12, 10.3, 10.6) |
| **Provenance, tree lenses & logical reversibility** Provenance [3]; Lenses [23]; Bennett [4] | $\mathbb{N}[X]$ polynomials via $\mathrm{Eval}_v$ [3]; lossy-view get and put$(V, S)$ tree lenses [23]; compute–copy–uncompute erasure [4] | Shares symbolic-provenance-before-evaluation; no relational query functor or thermodynamic heat bound | All-zero-resilient ($\kappa \equiv 0$) one-sided syntactic left inverse needing no old tree (Thm. 3.9); uncomputation-free step–request slice inversion $r = a_i + s$ & gates (Thm. 10.5); total writeback $V = p + n$ (Thm. 8.15) |

| Axis & representative works | Established classical mechanism | Inherited baseline & non-overclaim scope | New invariants & theorems locked in v3 |
|---|---|---|---|
| **Process calculi, refinement & observability** CCS [20]; ACP [21]; TLA [6][7]; Events [8]; Coalgebra [9]; Kalman [10] | Handshake sync, choice + & right-distributive / bisimulation quotients [20][21]; history/prophecy refinement [7]; conflict posets [8]; final-coalgebra / observable quotients [9][10] | Deterministic single-root processes (no handshake channels or nondeterministic conflict); $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$ realizes observable quotients | Single constructor retains exact branching trees with $+, \times$ as cardinal shadows (Thm. 2.2); **zero prophecy variables**: shared occurrence forces unique comparison (Thm. 5.3); **source identity strictly stronger than bisimulation** (Thms. 3.9, 9.9, 9.12, 10.1) |
| **Resource logics, PCC & type foundations** Linear [11]; Sep. Logic [12][13]; PCC [14]; ITT [15]; HoTT [16]; Lean 4 [17] | Linear formula consumption [11]; disjoint-heap frame & static amortised potentials [12][13]; load-time safety proofs [14]; $\Sigma$-witnesses [15] & Univalence [16] | Built on Lean 4 kernel [17] & $\Sigma$-witnesses [15]; does not claim pointer-aliasing separation logic | Explicit **T0** `[]` vs. **T1** split; **rejects Univalence equating isomorphic carriers as one occurrence** (Thm. 5.3, Prop. 8.3 sealed envelopes); law-free $\leq$; $<$; $\leq$ well-foundedness (Thm. 6.7), step debit & `Type` receipts $d_i = B_i$ (Thms. 6.10, 6.12) |
| **Belief revision, simplicial cofaces & institutions** AGM [18]; Simplicial [24]; Institutions [19] | Fixed-language partial meet contraction [18]; simplicial coface $\delta_i^n$ & codegeneracy $\sigma_i^n$ [24]; external $\phi : \Sigma \to \Sigma'$, Satisfaction Condition & conservative extensions [19] | Shares coface/codegeneracy retraction geometry & conservative non-intrusion; no AGM selection translation | For **rooted double-empty bankruptcy** $\eta$ (not inconsistency), constructs in zero axioms the initial coface $C_\eta = L_+ \sqcup \{\star_\eta\}$: zero old-face leakage, full old-theorem survival, unique injective budget-non-increasing factorization into the 1st write & tick-2 direct answer (Thms. 8.4, 8.6–8.7, Prop. 8.9) |

Four deep methodological contrasts stand out across Table 5:

**First, from stream deltas, external memo tables, and lossy lenses to one-sided syntactic left inversion, in-place AST double negation, and bidirectional slice addressing.** DBSP[1] and higher-order change calculi[2] compute updated output values, adaptive functional programming[22] replays dynamic dependency graphs via mutable references and external memo tables, provenance semirings[3] record combinations via commutative polynomials, bidirectional tree lenses[23] take the old source tree $S$ alongside the updated view $V$ in $\mathrm{put}(V, S)$, allowing the update to preserve source information absent from the view, and Bennett's reversible Turing machine[4] runs inverse transitions to erase its history tape. Our algebra locks in information conservation at three levels needing neither external memo tables, old-source complements, nor reverse uncomputation: Theorem 3.9 recovers the full generative tree via a one-sided syntactic left inverse even when $\kappa \equiv 0$ collapses every semantic output to zero; Theorem 10.3 applies ring-

free double negation $(-\mathrm{id})((-\mathrm{id})(e'))$ to recover paid constant ASTs in place and execute $f^{(n+1)}$ across cofinal horizons; and Theorems 10.5–10.6 establish $r = a_i + s$ bidirectional slice inversion and prefix-locked tail continuation so omitting or duplicating any nonempty step group is caught directly on the native trace.

**Second, from classical process-calculus behavioral quotients and prophecy variables to exact branching trees, zero-lookahead comparison, and whole-word-first recovery.** Milner's CCS[20] and Bergstra–Klop's ACP[21] quotient branching differences via bisimulation or right distributivity, Rutten's final coalgebra[9] identifies all behaviorally indistinguishable states, and Abadi and Lamport[7] show how auxiliary history and prophecy variables can be needed to obtain state-space refinement mappings. Theorems 9.3 and 9.12 construct the exact module-theoretic observability quotient $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$, while drawing a sharp boundary: **behavioral bisimulation lives in the downstream observation layer and can never substitute for upstream source identity**. In T0, single-constructor unfolding preserves exact branching trees and stores zero completed futures (zero prophecy variables), so two faithful realizations determine their unique comparison solely via their shared root occurrence (Theorem 5.3); across horizons, Theorem 9.9 (and Example 9.13) recovers the unquotiented formal word $\mathrm{rec}(q(w)) = w$ and injects trace positions strictly monotonically ($\iota(j) = j + L_{\mathrm{old}} + L_{\mathrm{phys}} + 3$) *before* projecting to frames.

**Third, from static resource potentials and univalent isomorphism to T0 non-univalent occurrence rigidity and `Type` live-ledger receipts.** Linear logic[11], separation logic[12], and amortised resource separation logic[13] bound resource potentials inside static derivations or Hoare triples, proof-carrying code[14] checks safety predicates once at load time, and Homotopy Type Theory[16] elevates type equivalence to equality via Univalence ($(A \simeq B) \simeq (A = B)$). Our T0 zero-axiom kernel highlights why causal accounting deliberately rejects identifying isomorphic carriers as the same occurrence: an old disposition receipt transported onto the new coface support $\star_\eta$ must be explicitly sealed inside $\mathrm{Seal}_{\eta,k}$ rather than silently reused across isomorphic types (Proposition 8.3), and every reduction step under $\Lambda_{\rho,e}$ produces in `Type` (following Martin-Löf's $\Sigma$-witness discipline[15]) an exact-distance receipt $d_i = B_i$ and live-ledger debit even when the semantic effect vanishes (Theorems 6.10, 6.12, and 10.2).

**Fourth, from fixed-language belief contraction and external signature morphisms to endogenous simplicial initial cofaces entering the first whole-ledger write.** AGM theory change[18] defines contraction by a designated sentence and revision by new information within a fixed language, while Goguen and Burstall's institutions[19] formalize vocabulary change via external signature morphisms $\phi : \Sigma \to \Sigma'$, the Satisfaction Condition, and conservative extensions. Sections 7–8 solve endogenous vocabulary ascent when an evolving process hits expressive bankruptcy: the adjoined vertex $D_\eta = \{\star_\eta\}$ is forced in zero axioms by a rooted double-empty failure $\eta$ bound to an exact causal visit and live debt row, making $\mathrm{inl} : L_+ \hookrightarrow C_\eta$ and its deterministic $e_+$-retraction a cosimplicial coface–codegeneracy retraction pair[24]; Proposition 8.3 and Theorem 8.4 prove zero old-face leakage, full survival of old theorems, and unique injective budget-non-increasing factorization across three coordinates; and Theorems 8.6–8.7 with Proposition 8.9 deliver that exact direction into the live row consumed by the new root's first whole-ledger write so the new root answers directly at tick 2.

## 11 Conclusion

The opening three-tick readings $0, 0, 1$ agree numerically at their junction, yet admit no common source word across the full history. That elementary counterexample exposes a foundational principle across computation, logic, and dynamics: **equal readings are the weakest instantaneous shadow; for an answer to belong to an evolving run, each tier must lock in the reading's generating source object, that source object's concrete occurrence, and the live obligations and expressive boundaries left by that occurrence**.

Working across an explicitly separated **T0 zero-axiom constructive kernel** (`#print axioms = []`) and **T1 free-module completion layer**, this paper closes both branches of this four-tier process algebra (Table 6):

1. **Inside the T0 zero-axiom kernel**, a single constructor $\mathrm{occur}(a, \vec{T})$ derives leaf frontiers, associative substitution, the joint cardinality shadow, and a universally unique fold. The parallel cardinality formula uses `propext`. Root-read identity $\mathrm{rootRead}(o) = o$ excludes hidden-null witnesses for every downstream disposition; any two faithful realizations of a fixed root yield an unconditionally pointwise unique event comparison without prophecy variables or preset simulation relations; law-free $\leq$; $<$; $\leq$ payment macros are constructively well-founded; and whenever an old theory hits rooted double-empty bankruptcy $\eta$, $\eta$ constructs in zero axioms the **categorical initial coface** $C_\eta = L_+ \sqcup \{\star_\eta\}$, preserving every old theorem with zero old-face leakage and factoring uniquely, injectively, and without budget refill into the exact live row consumed by a typed revision's first whole-ledger write so the new root answers directly at tick 2.
2. **Inside the T1 module and reversible-continuation layer**, structureless native states lift freely into $\mathbb{Z}[S]$ under the three-term update law; whole-tree programs retain evaluation-free syntactic left inversion even when $\kappa \equiv 0$ collapses all semantic outputs to zero; derivation differences and module laws span the exact evaluation kernel $\mathrm{range}(\mathrm{rel}_\rho) = \ker(\mathrm{ev}_\rho)$ over any commutative ring, and inverse-limit completions $\widehat{C}(s)$ obey elementwise evolution; environment–expression pairs birth bearer-free obligations returning `Type`-valued completion receipts of exact distance $d_i = B_i$. After settlement, $e \ominus e'$ births the next inquiry, writes back $V = p + n$, and generates six compatible faces alongside $\mathsf{M}_{\mathrm{fut}} = \mathcal{W}/K_{\mathrm{fut}}$; universal semantic-zero congruence equates $e \ominus e'$ with $\delta_e - \delta_{e'}$ in $\mathsf{M}_{\mathrm{fut}}$ and transports it directly into canonical next; ring-free syntactic double negation retainedCode recovers paid ASTs to execute any pre-fixed homomorphism $f^{(n+1)}$ across cofinal stops while whole-word primitive replay keeps substitution $\tau_1$, relation boundaries, and cochain differentials $d^{0,1}$ in lockstep; and bidirectional slice inversion (stepInterval / readRequestOrigin, $r = a_i + s$) together with extendTo prefix-locked tail continuation renders cross-horizon incremental computation lossless and reversible across syntax, ledgers, and execution traces simultaneously.

**Table 6 Summary of the four-tier process algebra and third-edition bifurcated capstone (C1–C54).** Unassumed native structures, explicit inputs, generated objects, paired counterexample gates, and cross-domain instantiations side by side.

| Tier & axiom kernel | Native assumptions & input boundary | Endogenously generated objects | Core theorems & counterexample gates | Cross-domain instantiations |
|---|---|---|---|---|
| **T0 unfolding & source process** (`[]`, §2.1–§2.2) | Arbitrary labels $A$, structureless states $S$; zero arithmetic axioms or future tables | Single-constructor trees, associative advance, the joint cardinality shadow, root identity, on-demand histories | Thm. 2.2, Prop. 2.4; projection hidden-null blind-spot gate | Universal occurrence trees & single-root admission |
| **Tier 2: Source object & exact kernel** (T1, §2.3–§4) | States & binary maps need no addition or linearity; ring $R$ & $\kappa$ arbitrary | Free carrier $\mathbb{Z}[S]$, mixed lists, whole-tree programs & syntactic left inverse, exact kernel, $\widehat{C}(s)$ | Props. 2.5, 2.7, 3.5, 3.7, 4.8; Thms. 3.9, 4.4, 4.6, 4.7; Exs. 4.1–4.2 & $\kappa \equiv 0$ control | Fock native field words & unit-history tree charging |
| **Tier 3: Same occurrence** (T0 `[]`, §5) | Fixed causal world $W$ & two faithful realizations; zero simulation or prophecy inputs | Event-wise recovery, unique emitter- & compiler-preserving event comparison | Prop. 5.2, Thm. 5.3; multi-point fibre non-uniqueness gate | Spin×SU(7) unified-field representation alignment |
| **Tier 4: Same obligation** (T0/T1, §6) | Event dispositions; general discharge takes master law, computation needs no bearer or law | Three lifecycle laws, extended ledger, law-free well-foundedness, execution ledger & `Type` receipt $d_i = B_i$ | Props. 6.1, 6.4–6.5, 6.11; Thms. 6.7, 6.10, 6.12; Note 6.8 empty-terminal counterexample | Target-6 partition debt; cofinal birth stops |
| **Branch I: Initial-coface revision** (T0 `[]`, §7–§8.4) | Rooted double-empty failure $\eta$; dynamic run takes typed revision & semantic anchoring | Same-support bankruptcy, initial coface $C_\eta = L_+ \sqcup \{\star_\eta\}$, unique factorization, 2-row 1st write & tick-2 answer | Props. 7.1, 7.3–7.6, 8.2–8.3, 8.8–8.10; Thms. 7.7, 8.4, 8.6–8.7; empty-fibre gate | Causal audit transition & fixed two-tick $\mathbb{N}$ revision |
| **Branch II: Complete execution & future kernel** (T1, §8.5–§9) | Initial frame & source-program config (variables & reader only); zero future query tables | Residual request $e \ominus e'$, boundary differences, total $V = p + n$, all-actor model, stock injection, 6 faces, $\mathsf{M}_{\mathrm{fut}}$ | Thms. 8.12, 8.14–8.15, 9.2–9.3, 9.5–9.12; Exs. 8.13, 9.13 & constant zero-effect control | Full $R^m q_0$ comb & $Pu + (1-P)u$ Mellin pricing; 37-primitive CPS1 body |
| **Reversible continuation capstone** (T1, §10.1–§10.3) | Same source-program family plus pre-fixed homomorphism $f$; zero external index tables | $\mathcal{Z}_{\mathrm{sem}}$ request congruence, regrouped jointCode, double-negation retainedCode, step cochain replay, stepInterval and readRequestOrigin, extendTo & storedTimeline | Thms. 10.1–10.6, Ex. 10.7; anti-omission & anti-duplication gate on nonempty step slices | Finite-window 4-trace stock & inverse-residual correction; $\Gamma$ grouped write; fixed-target stored timeline |

When the language suffices, a settled answer turns its own updated residual into the next question and advances carrying its paid syntax tree, cochain boundary, and bidirectionally invertible step slices; when the language goes bankrupt, a rooted double-empty obstruction grows in zero axioms the categorical initial coface, sending its unique new direction into the new root's first write to answer directly at the next tick. Through execution, settlement, and bankruptcy alike, the source carries the complete credentials for the next answer.

## Appendix A Finite-Language Technical Details

**A.1 Joining native state points to runtime stages.** Let $u_i$ be runtime $u$ after $i$ steps, with native state $s_i$, living current $c_i$, and generated material stage $g_i$. Definition 2.3 and §2.3 give $\iota(s_i) = c_i$ and $A\delta_{s_i} = \delta_{s_{i+1}}$, while the stage generator locks three runtime equations: $g_i$'s occurrence equals the canonical emission of $c_i$'s root at that visit; $g_i$'s whole-ledger write-back equals heterogeneously that root's compilation; and $g_i$'s next current equals the next current recorded by $u_i$. Writing $w_{\mathrm{src}} = \mathrm{var}$ and $w_A = \mathrm{linear}_A(\mathrm{var})$, stage-$i$ evaluations read $(\delta_{s_i}, \delta_{s_{i+1}} - \delta_{s_i})$ and $(\delta_{s_{i+1}}, \delta_{s_{i+2}} - \delta_{s_{i+1}})$. The first coordinate of $w_A$ is witnessed directly by $u_{i+1}$; any native readout $h : S \to B$ (requiring no addition on $B$) evaluates $h(s_{i+1})$ forward on that witness and checks its basis image under the free observation, never inverting states from linear combinations.

**A.2 Why the paired language yields the complete stage inventory.** Set $\widetilde{V}(\kappa) := V(\kappa) \times V(\kappa)$. In paired environment $(\rho, \varepsilon)$, variables take $(\rho(x), \varepsilon(x))$, constants $a$ lift to $(a, 0)$, addition is coordinatewise, additive $h$ lifts to $\tilde{h}(a, da) := (h(a), h(da))$, and biadditive $\mu$ lifts to

$$\tilde{\mu}((a, da), (c, dc)) := (\mu(a, c),\ \mu(a, dc) + \mu(da, c) + \mu(da, dc)).$$

Replacing each node of $e$ along the same syntax tree defines $\tilde{e}$. By structural induction, the second coordinate of a biadditive node reproduces the three terms of (3.1), so $\mathrm{ev}_{(\rho,\varepsilon)}(\tilde{e}) = (\mathrm{ev}_\rho(e), \mathrm{eff}_{\rho,\varepsilon}(e))$. Extending $R$-linearly to formal words defines $w \mapsto \widetilde{w}$ in Theorem 4.6, reducing tick-wise completion fibres to Theorem 4.4 on the paired model.

**A.3 Explicit two-certificate assembly for** $\ker(\mathrm{ev}_\rho) \subseteq \mathrm{range}(\mathrm{rel}_\rho)$**.** Let $w = \sum_i a_i \delta_{e_i} \in \mathrm{Fm}_R(\kappa)$ satisfy $\mathrm{ev}_\rho(w) = 0$. Put $v_i = \mathrm{ev}_\rho(e_i)$, $q_0 = \sum_i a_i \delta_{v_i} \in R[V(\kappa)]$, $j(\delta_v) := \delta_{\mathrm{const}(v)}$, and $q := j(q_0)$. Each endogenous normalization $\mathrm{norm}_\rho(e_i) : e_i \vdash_\rho \mathrm{const}(v_i)$ supplies a left relation index; weighting by $a_i$ forms $c_{\mathrm{der}}$ with $\mathrm{rel}_\rho(c_{\mathrm{der}}) = w - q$. The remainder $q_0$ satisfies $\sum_i a_i v_i = \mathrm{ev}_\rho(w) = 0$. Using the scalar module-law generators $\delta_x + \delta_y - \delta_{x+y}$ and $\delta_{av} - a\delta_v$, the quotient $R[V(\kappa)]/\mathrm{range}(\mathrm{rel}_{\mathrm{scl}})$ is $R$-linearly isomorphic to $V(\kappa)$ via $[x] \mapsto \mathrm{eval}(x)$ and $v \mapsto [\delta_v]$; since $q_0$ evaluates to 0, its quotient class vanishes, yielding $c_0$ with $\mathrm{rel}_{\mathrm{scl}}(c_0) = q_0$. Injecting $c_0$ into the right summand as $c_{\mathrm{scl}}$ gives $\mathrm{rel}_\rho(c_{\mathrm{der}} + c_{\mathrm{scl}}) = (w - q) + q = w$.

## Appendix B Completion-Carrier Details and Counterexample Proofs

**B.1 Stage quotients and inverse-limit successor.** At start $s$ and horizon $b$, $Q_b(s) = \mathrm{Fm}_R(\kappa)/\ker(\mathrm{pre}_b^s)$; $r_b^s : Q_{b+1}(s) \to Q_b(s)$ drops $U_{b+1}^s$, while $d_b^s : Q_{b+1}(s) \to Q_b(\sigma(s))$ drops $U_0^s$ (since $U_i^{\sigma(s)} = U_{i+1}^s$). For any $z = (z_b)_{b\in\mathbb{N}} \in \widehat{C}(s)$ with $r_b^s(z_{b+1}) = z_b$, set $(Tz)_b := d_b^s(z_{b+1})$. Because dropping first then last commutes with dropping last then first on windows of length $b+2$ ($r_b^{\sigma(s)} \circ d_{b+1}^s = d_b^s \circ r_{b+1}^s$), $r_b^{\sigma(s)}((Tz)_{b+1}) = d_b^s(r_{b+1}^s(z_{b+2})) = d_b^s(z_{b+1}) = (Tz)_b$, so $Tz \in \widehat{C}(\sigma(s))$. For any fixed tick $i$, surjectivity at finite stage $b = i+1$ supplies $w \in \mathrm{Fm}_R(\kappa)$ with $[w]_{i+1} = z_{i+1}$, whence $\mathrm{old}_i(Tz) = \mathrm{old}_i(d_i^s([w]_{i+1})) = \mathrm{ev}_{\rho_{i+1}}(w) = \mathrm{ev}_{\rho_i}(w) + \mathrm{eff}_{\rho_i,\varepsilon_i}(w) = \mathrm{old}_i(z) + \mathrm{eff}_i(z)$ without assuming a single global representing word.

**B.2 Complete counterexample proofs.** Example 4.1(4): $\mathrm{ev}_1(w) = 0$ and $\mathrm{ev}_1(w) = 1$ force $0 = 1$ in $\mathbb{Z}$ by symmetry and transitivity. Example 4.2: $w = x \cdot x + (-1) \cdot x$ evaluates at $\rho(x) = 1$ to $1 + (-1) = 0 \in \ker(\mathrm{ev}_1)$, while under $\rho'(x) = 2$ ($\varepsilon = +1$) Proposition 3.3 gives effect $1 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 + (-1) = 2 \neq 0$, so $\mathrm{ev}_2(w) = 2 \notin \ker(\mathrm{ev}_2)$.

## Appendix C Fibre Definitions and Derivations from Rooted Failure to First Write

**C.1 The four dependent fibres of the initial-coface world.** Fix rooted failure $\eta$ with old target support $s_+$ and $D_\eta = \{\star\}$. Open-responsibility fibres $\mathrm{Open}'$ of $N_\eta$ are defined piecewise by

$$\begin{aligned} \mathrm{Open}'(\mathrm{inl}\, s, \mathrm{inl}\, r) &:= \mathrm{Open}(s, r), \qquad \mathrm{Open}'(\mathrm{inl}\, s, \mathrm{inr}\, \star) := \emptyset, \\ \mathrm{Open}'(\mathrm{inr}\, \star, \mathrm{inl}\, r) &:= \mathrm{Open}(s_+, r), \quad \mathrm{Open}'(\mathrm{inr}\, \star, \mathrm{inr}\, \star) := \mathrm{Aligned}_\eta(\star, \star), \end{aligned}$$

where $\mathrm{Aligned}_\eta$ has unique constructor $\mathrm{same}(\star)$; $\mathrm{Holds}'$ is defined identically with claims $c$ and Holds. Thus the new coface row is $e_{\mathrm{obs}} = \langle \mathrm{inr}\, \star, \mathrm{same}(\star) \rangle$ (claim $\mathrm{inr}\, \star$, budget 0), while $\mathrm{Open}'(\mathrm{inl}\, s, \mathrm{inr}\, \star) = \emptyset$ enforces zero leakage onto old supports. Dispositions of kind $k$ at $\mathrm{inr}\, \star$ are wrapped in the private single-field seal $\mathrm{Seal}_{\eta,k}(d)$ ($d : \mathrm{Disposition}(s_+, k)$), and the sole added semantic-change constructor is $\mathrm{reveal}_\eta : \mathrm{SemanticChange}'(\mathrm{inr}\, \star, \mathrm{inl}\, \mathrm{claim}(o), \mathrm{inr}\, \star)$.

**C.2 Eight conditions of exact semantic anchoring and split-refined anchoring.** For a typed revision $\Xi$, exact semantic anchoring comprises one world equality ($\Xi$'s world equals $N_\eta$) plus seven dependent heterogeneous equalities matching root law surface, translation, initial support, old-ledger retraction, new row, new fact witness, and semantic change to $N_\eta$'s root semantics and canonical objects—none reducible to cardinality equality. Split-refined anchoring drops world equality while requiring root semantics on $\Xi$'s world and a genuine split in the first compilation into child rows $l \neq r$ with $\mathrm{origin}(l) = \mathrm{origin}(r) = e$, strictly smaller budgets, and full origin/initial coverage by $L_+$ and $e_{\mathrm{new}}$.

**C.3 From first-write generated rows to two-row answer–next.** Let $g_0$ be the generated initial evolution of $\Xi$'s new root at $v_0$. Non-terminality of the first write yields successor visit $v_1$ and whole-ledger evolution. Any generated row $e$ of $g_0$ acquires initial authority $\mathrm{au}_0(e)$, advances to destination $e_1$ at $v_1$ with authority $\mathrm{au}_1(e_1)$, and invokes $\Xi$'s own compiler $\mathrm{ans}_\Xi(v_1, e_1, \mathrm{au}_1(e_1))$. Instantiating at the migrated failure row and $e_{\mathrm{new}} = F_\Xi(\mathrm{inr}\, \star)$ yields Theorem 8.7.

**C.4 Capacity composition associativity.** Given injective embeddings $k_1 : O \sqcup D_1 \to M$, $k_2 : M \sqcup D_2 \to F$, and $k_3 : F \sqcup D_3 \to G$, canonical coproduct rebracketing $\alpha : O \sqcup (D_1 \sqcup (D_2 \sqcup D_3)) \xrightarrow{\sim} O \sqcup ((D_1 \sqcup D_2) \sqcup D_3)$ evaluates identically on old elements $o$ and directions $d_1, d_2, d_3$ on both sides, proving $k_{(12)3} \circ \alpha = k_{1(23)}$.

**C.5 Extended-world readback and event-indexed compilation (Propositions 6.4 and 6.5).** Proposition 6.4 splits by coproduct constructors: inactive supports have empty right fibres, active opening witnesses force label $\mathrm{id}_0$, and injectivity combines old-row injectivity with empty old preimages. Proposition 6.5 dispatches on $\delta = \mathrm{dispositionAt}(\mathrm{src.gen})$—compiling payment steps to target whole-ledger evolutions, phase terminals to whole-support terminals, and obstructions to same-row theory audits while storing the branch equation inside the compiled image.

**C.6 Causal audit field construction (Proposition 7.5).** Setting the audit root's source payload to $(\mathrm{occ}(a), \mathrm{occ}^+(r))$ lets both initial and target occurrences project out both endpoints without guessing from support labels. Embedded old rows remain identity remainders returning "not newly generated" on the finite patch, while the residual row alone carries the generated-row witness, acquiring initial live temporal causal authority to assemble Definition 8.1's $\eta$ and feed the initial coface directly.

## Appendix D Formalization Entry Points and Axiom Boundaries

Every definition, construction, positive theorem, and counterexample gate in this paper is machine-verified in Lean 4[17]. Pinned source commits are: **H** = `e60a86058abcb74b7f2225f2383c182be8343785` for the foundational layer (with two private list-membership tactic alignments for Proposition 6.1's regression at base `8e29b8e`); **AB** = `fde5c14df36b6064c87064acd20830d3df0500bb`, **AC** = `b67aa48c75b3cd7738766ed40093bb3f7c1d7a1a`, **AD** = `aef0ad2a26199d9fd6da9bc9e7ec1a65636ef63f`, and **AE** = `26b91feb6b507fdc3f43be9c429edb63fb9395ef` for the endogenous computational and complete-execution chain (three-line upstream interface alignment at `089a1727993bec51c27ee72a0311022fec8111ab`); **PR** = `e05558097256e86b537c019c8ca63cd449d6b2d2`, **MF** = `c62f3d25c42d4bf2b77ac009b49fc71cfd752f46`, **MV2** = `71e94e8261d67cf4849a05eb14ac95aa2f7dcbb9`, and **MV3** = `9c73a630ce05bea062ec889b188f5105c0afb796` (public environment supplements at H0mework **H0-S** = `a1adcc9f4a06b872c381b8ad49513d6bf1cd621a`) for completion receipts, total-value writeback, and the maximal future model; and **MV4** = `ba7e274beb1de38900a6fc2c8c8dfe6cfad9d9aa` (C46–C50), **MV5** = `df421ff600085cbc290a7ae946193961bed7c83b` (C51–C52), and `7e4da746f300b3796b8b18afb26cb0765c29afcb` (C53–C54) for the third-edition reversible-continuation capstone.

**T0 vs. T1 axiom-closure boundary.** Declarations marked `[T0]` in D.1 (collected mainly under `Tracks/LivingLawAxiomFreeCore.lean`, with standalone comparison and regression modules verified via `#print axioms`) depend on **zero axioms** (`#print axioms` outputs `[]`). Modules involving free-module quotients, inverse limits, nonconstructive six-way step classification (`noncomputable`), and operator or smooth dynamical instantiations have transitive axiom closures strictly confined to `{propext, Classical.choice, Quot.sound}` (**T1**), with zero research axioms, zero `sorry`, and zero logical `unsafe`/`partial` dependencies.

## D.1 T0 Zero-Axiom Kernel and Foundational Layer Entry Points (Commit H Kernels; Byte-Identical at MV4 Except `eec031c489` C17 Control Lemma)

Paths relative to `Lean/SaturationMonoid/`:

| Text result & structure | Lean 4 modules & declarations (`[T0]` denotes empty axiom list `[]`) |
|---|---|
| Def. 2.1, Thm. 2.2(1)(2) single-constructor unfolding & universal fold `[T0]` | `LivingLawRootedAccountedUnfoldingKernel.lean`: `RootedAccountedUnfolding.occur`, `frontier`, `frontier_ne_nil`, `trace`, `advance`, `advance_assoc`, `parallelAt`, `jointAt`, `cardinalShadow_parallelAt` (propext), `cardinalShadow_jointAt`, `fold`, `fold_unique` |
| Thm. 2.2(3) root-read identity, zero hidden-null & projection factorization `[T0]` | `LivingLawRootCoreRepresentationKernel.lean`: `rootCofaceRead`, `rootCofaceRead_consumerSafe`, `no_hiddenNull_at_rootCofaceRead`, `installedSubsystemAuthority_factorizes`, `installedJointSubsystemAuthority_factorizes` |
| Def. 2.3, Prop. 2.4 source processes & fixed-law successor rigidity `[T0]` | `LivingLawRelationNetworkKernel.lean`: `WorldRelationNetwork`; `LivingLawRootAnswerNextHistoryKernel.lean`: `SourceNativeLivingRootCurrentAt`, `erase`, `SourceNativeLivingRootProcess`, `successor_difference_reflects_current`, `generateHistory` |
| Prop. 2.5, Def. 2.6, Prop. 2.7, App. A, Thm. 4.4 free lifts & exact kernel | `GenericFoundation/Operations/Native/{Core,Field}.lean`: `point_factorizes`, `nextWitness`, `nativeReadWitness`; `Scalar/InventoryLift.lean`: `eval_liftExpr`, `evaluation_liftMap`; `Scalar/Presentation.lean`: `source_reduction`, `relation_range_eq_kernel` |
| Thms. 4.6–4.7, Prop. 4.8 completion fibres & elementwise evolution | `GenericFoundation/Operations/Runtime/{Relations,Successor,Evolution}.lean`: `completion_fibre_generated`, `successor_fibre_generated`; Fock extension in `ParticleWave/Fock/Runtime/SourceHistory/...` |
| Prop. 5.2, Thm. 5.3 faithful recovery & unique comparison `[T0]` | `LivingLawCausalRealizationConstructiveKernel.lean`: `World`, `NoSuspendedCausalMagic`, `commutingPresentation`; `LivingLawRootGroundedRealizationKernel.lean`: `recover_eq_generated`, `event_eq_emitted`, `comparison_unique` |
| Props. 6.1, 6.4–6.5, Thm. 6.7 lifecycle & well-founded settlement `[T0]` kernel | `ResponsibilityLifecycleKernel.lean`: `Edge.freshlyAllocated_implies_admitted`, `newlyLive_implies_admitted_or_reopened`, `disappears_implies_obligationTerminal`, `Run.history_eq`; `GenericFoundation/Responsibility/Debt/Activation/World/LivingLawRootGeneratedDebtActivationWorldReadback.lean [T0]`: `inactiveInventoryPresentation`, `activeInventoryPresentation`, `active_debt_row_unique`; `.../Root/Direct/LivingLawRootGeneratedDebtActivationDirectCompilerKernel.lean [T0]`: `CompiledResultAt`, `compile`; `LivingLawRootNoetherianDebtClosureKernel.lean [T0]`: `generatedHistory`, `terminalSettlement`, `generatedSettlement`, `generatedReceipt`; Table 2 in `...TargetSixActionDebt.lean` |

| Text result & structure | Lean 4 modules & declarations ([TO] denotes empty axiom list []) |
|---|---|
| Props. 7.3–7.5 same-support bankruptcy & causal audit [TO] bankruptcy kernel | GenericFoundation/Occurrence/Residual/Admission/LivingLawRootGeneratedResidualWorldCofaceKernel.lean [TO]: liftTheory, residualFailure; ...DependentJointPassiveResidualAuditTransition.lean: causal_transition_mouth |
| Props. 8.2–8.3, Thms. 8.4, 8.6–8.7, Prop. 8.8 initial coface [TO] | LivingLawObstructionGeneratedMinimalCofaceKernel.lean [TO]: GeneratedMinimalCofaceAt, oldTheoremSurvival, no_newResponsibility_on_oldFace, no_newClaim_on_oldFace, FaithfulCofaceMorphism, minimal_coface_initial, minimal_coface_factorization_unique, compose_embed_associates; LivingLawTypedSemanticWorldNetworkU8{,Recovery}Kernel.lean [TO]: factorizesThroughInitialCoface, factorizesThroughInitialCofaceAnswerAndNext |
| Table 4, Props. 8.9–8.10 two-tick revision & one-shot impossibility | LivingLawRootInquiryU8CompletionRegression.lean: generatedTypedRevision, public_old_new_rows_generate_same_next; LivingLawCanonicalInquiryRuntimeRegression.lean: firstTick_resolution_is_revised, secondTick_is_revised_root_answer, oneShot_has_no_autonomous_runtime [TO] |

## D.2 Endogenous Computational Chain and Complete Execution Entry Points (Commits AB, AC, AD, AE)

Paths relative to Lean/SaturationMonoid/GenericFoundation/ (JS/ = Responsibility/Debt/Activation/Root/Direct/JointSource/, DJ/ = Operations/Native/Tree/Fold/Dependent/, ACT/ = Operations/Inquiry/Context/Faces/Execution/Activation/, AW/ = DJ/Joint/ActionWords/Inventory/):

| Text result (pinned commit) | Lean 4 modules & declarations |
|---|---|
| Prop. 3.7, Thm. 3.9 step-charged traces & syntactic left inverse (AB) | Operations/Execution/{Step,Run,Trace,Relations}.lean: remaining, Step.remaining_eq, step_wellFounded, execution_length, Trace.toDerivation, Trace.relation_boundary; Operations/Native/Tree/Fold/{Source,Inverse}.lean: program, program_value, program_budget, readTree, read_program, program_injective |
| Def. 6.9, Thm. 6.10, Prop. 6.11 bearer-free ledger & whole-tree inquiry (AB) | Operations/Execution/Debt/Source.lean: State, law, completed_value; JS/OwnerFree/{Source,Compiler,Authority,Runtime,Completion,Payment,Consumer}.lean: mathEntry, patch_fold, budget, activePayment, no_refill, endpoint_no_paid, relation_boundary; Operations/Native/Tree/Fold/{Installation,Consumer,Recovery}.lean |
| Prop. 7.6 (AB), Thm. 7.7 (AC) dependent feed & successor activation | DJ/{Source,Installation}.lean: sourceFeed, generatedFeed, actual_effect_value; DJ/Input/{Source,Consumer,Runtime/Source,Runtime/Consumer}.lean: inputState, generatedRuntime; DJ/Next/{Source,Consumer}.lean: nextVisit, generatedNext, current_whole_next |

| Text result (pinned commit) | Lean 4 modules & declarations |
|---|---|
| Def. 8.11, Thm. 8.12 residual request & recursive frame runtime (AB) | JS/Native/ResidualRequest.lean: expression_eval, budget, updated_value, residual_zero_iff; JS/Native/Restructuring/Inquiry/Continuation/{Frame,Source,Policy,Inventory,Occurrence,Runtime,Payment,Residual}.lean: frames_erase_injective, actual_node, macro_next_preserves, actual_paid_receipt |
| Thms. 9.2, 9.3, 9.5 joint temporal program, all actors & admission (AD) | Occurrence/Temporal/{Source,Action/Source,Action/Inverse,History/Common/...}.lean; DJ/Joint/{Source,Laws,Consumer,TargetDifference,TargetProgramme,TargetConsumer}.lean; AW/{Source,Complete/Source,EffectHistory/...,NodeHilbert/...}.lean; ACT/{Programme,Target,Receipt/Source,Receipt/Execution,Receipt/Compiler,Receipt/Consumer,Receipt/Runtime}.lean |
| Thms. 9.6, 9.7, 9.8 stock crossing, covariance & default run (AE) | ACT/Receipt/Inventory/{Source,Laws,Installation,Born/...}.lean: binding_value, migrated_charge, actual_injective; Operations/Inquiry/Context/Native/Pairing/{Source,Orbit/...}.lean: point_covariance, complete_charge; DJ/{Joint,Branch}/Primary/Consumer.lean; DJ/Branch/JointQuery/Actor/.../Consumer.lean |

## D.3 Type Completion Receipts, Total-Value Feedback, Six Faces, and Maximal Future Model (Commits PR, MF, MV2, MV3, H0-S)

With W/ = ACT/Receipt/Configured/Writeback/, I/ = Operations/Inquiry/Cofinal/Native/, F/ = DJ/Branch/JointQuery/Actor/Operation/Query/Inventory/Observer/CurrentChild/Dynamic/Live/Native/Action/Coupled/Registry/Calculation/Common/Feedback/Continuation/, and Cx/ = F/SourceFamily/Foresight/Contextual/:

| Text result (pinned commit) | Lean 4 modules & declarations |
|---|---|
| Thm. 6.12 Type completion receipts & cofinal births (MV2/MV3) | ACT/Completion.lean: Receipt, receipt, receipt_next, receipt_compiles; ACT/Completion/{Prefix,Births}.lean: receipt_distance_exact, receipt_paid_prefix, birthIndex, cover |
| Thm. 8.14 boundary differences & environment square (PR, MF, H0-S) | Operations/Execution/Coefficients/{Source,Words}.lean: expression_eval, expression_remaining; Cx/Profile/Finite/Reader/Difference/{Source,Inquiry/...,Actual}.lean: difference_in_actual_history, own_fee; Cx/Mixed/{Effect,Receipt,Consumer}.lean; H0-S FiniteIncoming.lean, ActualNativeEnvironment.lean |
| Thm. 8.15 total writeback & regenerated decoders (MV3) | W/{Source,Consumer}.lean: old_paid_value, native_value, updated_total, born_environment; W/Successor/{Source,Consumer}.lean: successorPacket, second_decoder, second_writeback |

| Text result (pinned commit) | Lean 4 modules & declarations |
|---|---|
| Thms. 9.9–9.12 stock injection, cofinal stops, 6 faces & $M_{fut}$ (MV2/MV3) | Operations/Inquiry/Context/Native/Frame/{Source,Stock,Stock/Return,Stock/Coordinates,Stock/Birth}.lean: fullword_read, intoBorn_strictMono; F/SourceFamily/Activation/{Source,Stage,Admissions}.lean: nextAdmissionAt, actual_segment, cofinal; W/Successor/FourFace/...: actualRelation, generated_differential; I/Installed/Cofinal/Tower/...: actual_receiver_square, model_fibre, model_action_native; Operations/Observed/Words/{Kernel,Model}.lean |

## D.4 Third-Edition Reversible Continuation and Windowed Instantiation Entry Points (Commits MV4, MV5, 7e4da746f3)

With SF/ = F/SourceFamily/Calculation/, Q/ = ACT/Receipt/Inventory/ActualQuery/, RC/ = I/Installed/Cofinal/Action/Common/RootConsumer/, A_src/ = I/Installed/Cofinal/Action/Source/, NM/ = NoIslandNoMagic/CanonicalArithmeticState/ParticleWave/Fock/Runtime/SourceHistory/Operation/Calculation/Admission/Inquiry/Continuation/Residual/PaidNativeMethod/, and J/ = NoIslandNoMagic/CanonicalRiemann/GeneralizedDual/CoPoissonEnergy/Muntz/Cokernel/CenteredGram/JointAction/Runtime/Feedback/:

| Text theorem & claim (pinned commit) | Lean 4 production modules & verification declarations |
|---|---|
| Thm. 10.1 universal semantic zero & next request (C46, MV4) | RC/Semantics.lean: semanticZero, universal_inventory_zero, lift_semantic_zero, residual_semantic_zero, semantic_model_equiv; RC/{Source,Consumer}.lean: rootedMaterial, rootedRequest, rooted_request_model, actual_generated_current, originalOldRow, originalBornRow, actual_root_future (CP121) |
| Thm. 10.2(1) regrouped residual evaluation (C47, MV4) | SF/Factory/{Source,Consumer}.lean: withCalculation, codeAt, cofinal_actual_value; SF/ActualInput/{Source,Consumer}.lean: withActualInput, actual_environment; SF/FullResidual/{Source,Consumer,Dynamics/Source,Dynamics/Consumer,Dynamics/Initial}.lean: regroup, jointCode, joint_eval, joint_complete, cofinal_collapsed_value (CP122–126) |
| Thm. 10.2(2) shared Raw & linear receiver provenance (C48, MV4) | Q/{Source,Consumer,Next/Source,Next/Consumer}.lean: commonMaterial, query_action_raw, commonAtNext; Q/Next/EnvironmentUpdate/{Source,Consumer}.lean: source_env_update, source_sum_actual; Q/Next/Linear/Receiver/{Source,Consumer,Action}.lean: actual_updated_value, read_material_update (CP128–131, 135–136) |
| Thm. 10.2(3) same-debt syntax debit & settlement (C49, MV4) | SF/ActualInput/EnvironmentUpdate/CurrentMaterial/{Source,Consumer}.lean: nativeMaterial, consume_actual; .../RootedEffect/{Source,Consumer,Completion/Source,Completion/Consumer}.lean: native_residual_effect, born_actual_payment, bornWholeWrite, completed_native_effect, consume_rooted_completed_answer (CP132–134) |

| Text theorem & claim (pinned commit) | Lean 4 production modules & verification declarations |
|---|---|
| §10.4 finite $P_a$-window 4-trace & residual correction (C50, MV4) | J/ClozelOriginalKFinitePaWindowOperatorSourceAction/{Word,Inverse,Cochain,Emitter,Born,Received,Consumer,Root,Complete}.lean: finite_stock_generated_source_delta, actual_finite_window_whole_next_future (CP217); J/ClozelOriginalKWindowInverseResidualSourceAction/{Source,Effect,Emitter,Born,Received,Observation,Physical,Consumer}.lean: received_source_recovery_action, received_actual_source_window_price (CP219) |
| Thm. 10.3 double-negation AST & homomorphic continuation (C51, MV5) | SF/ActualInput/PaidMethod/Source.lean: pairAction, actionAt, retainedCode, raw, raw_value, factory, templateFourFaces; PaidMethod/Recovery.lean: negate_paid, after_native_retained, after_native_value, after_native_whole_action, actual_complete_query_written, actual_native_faces; PaidMethod/Admissions.lean: cofinal_paid_method_faces, finite_paid_method_faces (CP483) |
| Thm. 10.4 primitive replay & cochain lockstep (C52, MV5) | Operations/Native/Context/WordExecution.lean: observerEnvironment, embed_eval, embeddedStep, embeddedTrace, pairedForward, forward_charge; NM/RelationCochain/{Source,Consumer}.lean: replayStock, replay_written, replay_next_query, actual_boundary_from_source, actual_complex_boundary, actual_complex_differential, actual_coimage_boundary, actual_relation_cochain_method (CP485) |
| Thm. 10.5 bidirectional step–request slice inversion (C53, 7e4da746f3) | A_src/Receipt.lean: atBirth; A_src/Receipt/Block/Position.lean: readExecutedStep; A_src/Receipt/Origin.lean: RequestOriginAt, readRequestOrigin; A_src/Receipt/Block/StepInterval.lean: StepInterval, stepInterval, sourceRequestAt_interval_address; StepInterval/Whole.lean: grouped_requests, grouped_executed_trace, grouped_request_boundary; anti-tamper gate in Verification/no-island/riemann-r-cofinal-complete-grouped-write/Gate.lean (CP514, 521, 528–532) |
| Thm. 10.6 prefix-locked continuation & timeline (C54, 7e4da746f3) | Operations/Inquiry/NativeHistoryContinuation.lean: HistoryAt.eq_runFrom, storedOccurrenceAt, stored_occurrence_eq, continueFor, extendTo, extendTo_prefix; Operations/Runtime/StoredHistory.lean: stored_pair_actual, stored_inventory_actual; Operations/Runtime/StoredTimeline.lean: storedTimeline, stored_timeline_adjacent, timeline_inventory_actual (CP596–598) |

# Appendix E Quick Reference of Core Terms and Symbols

| Term / symbol | One-line structural definition | First formal entry |
|---|---|---|
| T0 / T1 axiom split | T0: zero-axiom constructive kernel (`#print axioms = []`); T1: module/completion layer using only `propext`, `Classical.choice`, `Quot.sound` | Frontmatter, §1.1 |
| Rooted unfolding $\mathrm{occur}(a, \vec{T})$ | Sole inductive constructor deriving frontier fr, associative advance adv, $\oplus_r, \otimes_r$, and unique fold, with `propext` for the parallel cardinality formula | Def. 2.1, Thm. 2.2 |
| Root-read identity $\mathrm{rootRead}(o) = o$ | Identity readout on exact temporal root events, consumer-safe with empty hidden-null witness type | Def. 2.1, Thm. 2.2(3) |
| Source process $\mathcal{P} = (S, \iota, s_0, \sigma)$ | Structureless state set + injective living-current registration + initial state + law-preserving successor | Def. 2.3, Prop. 2.4 |
| Free carrier $\mathbb{Z}[S]$ & joint lift $\mathrm{lift}(f)$ | Finitely supported integer formal sums (Z-sets) and biadditive binary lift $\mathrm{push}_f \circ \mathrm{joint}$ | §2.3, Def. 2.6 |
| Structural effect $\mathrm{eff}_{\rho,\varepsilon}(e)$ & mixed list | Update difference $\mathrm{ev}_{\rho+\varepsilon}(e) - \mathrm{ev}_{\rho}(e)$ (three bilinear terms) and order/multiplicity-preserving list | Def. 3.2, Def. 3.4 |
| Whole-tree program $\mathrm{prog}_{\kappa} T$ & parser read | Node-wise bilinear lift of $\kappa$ (budget $\beta(T)$) and evaluation-free syntax-only left inverse | Def. 3.8, Thm. 3.9 |
| Relation map $\mathrm{rel}_{\rho}$ & completion $\widehat{C}(s)$ | Linear extension of derivation differences $\sqcup$ module laws (range $= \ker \mathrm{ev}_{\rho}$) and prefix-quotient inverse limit | Def. 4.3, Def. 4.5 |
| Faithful realization & unique comparison $\Phi_{12}$ | Two-sided occurrence–event representation + emitter compatibility + lawful compilation forcing unique comparison | Def. 5.1, Thm. 5.3 |
| Whole live ledger $L_s$ & payment macro | Dependent sum $\sum_r \mathrm{OpenAt}(s, r)$ and $\leq$; $<$; $\leq$ sandwiched well-founded debit chain | §6, Def. 6.6, Thm. 6.7 |
| Execution debt law $\Lambda_{\rho,e}$ & `Type` receipt | Paid-prefix debt states with budget $r(e')$ and dependent-sum receipt returning exact distance $d_i = B_i$ | Def. 6.9, Thm. 6.12 |
| Rooted failure $\eta$ & initial coface $C_{\eta}$ | Visit- and row-anchored double-empty bankruptcy generating $L_+ \sqcup \{\star_{\eta}\}$ with unique faithful factorization | Def. 8.1, Prop. 8.2, Thm. 8.4 |
| Semantic anchoring (exact / split-refined) | Identifying a typed revision $\Xi$'s fields with $N_{\eta}$ or a genuine two-child split to recover two-row answer–next | Thm. 8.7, App. C.2 |

| Term / symbol | One-line structural definition | First formal entry |
|---|---|---|
| Residual request $e \ominus e'$ & total $V = p + n$ | Updated difference $e + \mathrm{linear}_{-\mathrm{id}}(e')$ (budget $\geq 2$) and writeback of old paid value plus native effect | Def. 8.11, Thm. 8.15 |
| Maximal future kernel $K_{\mathrm{fut}}$ & $\mathsf{M}_{\mathrm{fut}}$ | Largest present-invisible $S$-invariant submodule of $\bigoplus_j \mathcal{W}_j$ across all $a \in \{*\}^{<\omega}$ and sites $j$, with quotient | Thm. 9.12 |
| Universal semantic-zero submodule $\mathcal{Z}_{\mathrm{sem}}$ | Formal words vanishing in every environment, closed under pair lifts and equating $\delta_{e \ominus e'}$ with $\delta_e - \delta_{e'}$ in $\mathsf{M}_{\mathrm{fut}}$ | §10.1, Thm. 10.1 |
| Retained-code extractor `retainedCode` | In-place ring-free double negation $(-\mathrm{id})((-\mathrm{id})(e'))$ recovering paid ASTs to execute $f^{(n+1)}$ across horizons | Thm. 10.3 |
| Slice inversion `stepInterval` / `readRequestOrigin` | Forward mapping of step $i$ to contiguous slice $[a_i, a_i + c_i)$ and backward origin inversion $r = a_i + s$ | Thm. 10.5 |
| Tail continuation `extendTo` & `storedTimeline` | Prefix-locked suffix execution of `total-fuel` steps from the tail state and single-pass length-`fuel+1` table | Thm. 10.6 |

## Code and Data Availability

Inherited first-edition proofs and verification modules remain pinned to public repository H0mework commit `234817b3c3f7a1023226e7a4fdb8660ddcbc9d0a`, with their first-release map and reproduction guide.

Second-edition proofs, source-generated action-environment squares, and finite incoming wiring remain pinned to public commit `b9d66048342b1a5aeac3e112d920ed85401bb20a`, with source and public-package mappings, readiness records, published binding index, and published archive (DOI: 10.5281/zenodo.23297175). Per the edition reproduction guide, run `python3 tools/edition_release.py --edition second verify-map` to verify mappings and `python3 tools/edition_release.py --edition second --paper source-process-core build --output .local/v2-build` (or `trust` in place of `build`) for strict compilation and declaration-closure verification.

The public proofs for C46–C54 are fixed at H0mework commit `5f94af7e988c9831c92e54fe89747699fa29cd85`. The claim map binds sources, direct consumers, complete declaration types, and original certifications. The C47–C48 entry is `H0mework.Papers.SourceProcessCoreV3.MV4`. C46, C49, and C50 use WA38–WA40. C51–C54 use WA41–WA42 and WA44–WA45. The research sources are MV4 `ba7e274beb1de38900a6fc2c8c8dfe6cfad9d9aa`, MV5 `df421ff600085cbc290a7ae946193961bed7c83b`, and `7e4da746f300b3796b8b18afb26cb0765c29afcb`. Each historical certification retains its own execution revision.

Run `python3 tools/source_process_core_v3.py entry` at the H0mework repository root. The reproduction guide gives the build, trust, complete-type, and source-recovery commands. The local acceptance record includes the observed axiom dependencies of the unfolding and root-read declarations. The editable archive includes both languages, figure sources, bibliography, and `publi`

`c-source-binding.json`. Code is Apache-2.0. Paper text, supplements, and original figures are CC BY 4.0.

## References


1. M. Budiu, T. Chajed, F. McSherry, L. Ryzhyk, V. Tannen. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proceedings of the VLDB Endowment*, 16(7):1601–1614, 2023. doi:10.14778/3587136.3587137.
2. Y. Cai, P. G. Giarrusso, T. Rendel, K. Ostermann. A Theory of Changes for Higher-Order Languages: Incrementalizing λ-Calculi by Static Differentiation. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14)*, 145–155, 2014. doi:10.1145/2594291.2594304.
3. T. J. Green, G. Karvounarakis, V. Tannen. Provenance Semirings. In *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS '07)*, 31–40, 2007. doi:10.1145/1265530.1265535.
4. C. H. Bennett. Logical Reversibility of Computation. *IBM Journal of Research and Development*, 17(6):525–532, 1973. doi:10.1147/rd.176.0525.
5. The Stacks Project Authors. Lemma 10.12.2: Tensor Products, Tag 00CX. *The Stacks Project*, https://stacks.math.columbia.edu/tag/00CX.
6. L. Lamport. The Temporal Logic of Actions. *ACM Transactions on Programming Languages and Systems*, 16(3):872–923, 1994. doi:10.1145/177492.177726.
7. M. Abadi, L. Lamport. The Existence of Refinement Mappings. *Theoretical Computer Science*, 82(2):253–284, 1991. doi:10.1016/0304-3975(91)90224-P.
8. G. Winskel. Event Structures: Lecture Notes for the Advanced Course on Petri Nets. Technical Report UCAM-CL-TR-95, University of Cambridge Computer Laboratory, 1986. doi:10.48456/tr-95.
9. J. J. M. M. Rutten. Universal Coalgebra: A Theory of Systems. *Theoretical Computer Science*, 249(1):3–80, 2000. doi:10.1016/S0304-3975(00)00056-6.
10. R. E. Kalman. Mathematical Description of Linear Dynamical Systems. *Journal of the Society for Industrial and Applied Mathematics, Series A: Control*, 1(2):152–192, 1963. doi:10.1137/0301010.
11. J.-Y. Girard. Linear Logic. *Theoretical Computer Science*, 50(1):1–101, 1987. doi:10.1016/0304-3975(87)90045-4.
12. J. C. Reynolds. Separation Logic: A Logic for Shared Mutable Data Structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS 2002)*, 55–74, 2002. doi:10.1109/LICS.2002.1029817.
13. R. Atkey. Amortised Resource Analysis with Separation Logic. *Logical Methods in Computer Science*, 7(2:17):1–33, 2011. doi:10.2168/LMCS-7(2:17)2011.
14. G. C. Necula, P. Lee. Safe Kernel Extensions Without Run-Time Checking. In *Proceedings of the Second USENIX Symposium on Operating Systems Design and Implementation (OSDI '96)*, 229–243, 1996. doi:10.1145/238721.238781.
15. P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, Naples, 1984.
16. The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, 2013.
17. L. de Moura, S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, vol. 12699, 625–635, 2021. doi:10.1007/978-3-030-79876-5_37.
18. C. E. Alchourrón, P. Gärdenfors, D. Makinson. On the Logic of Theory Change: Partial Meet Contraction and Revision Functions. *Journal of Symbolic Logic*, 50(2):510–530, 1985. doi:10.2307/2274239.
19. J. A. Goguen, R. M. Burstall. Institutions: Abstract Model Theory for Specification and Programming. *Journal of the ACM*, 39(1):95–146, 1992. doi:10.1145/147508.147524.
20. R. Milner. *A Calculus of Communicating Systems*. Lecture Notes in Computer Science, vol. 92, Springer-Verlag, Berlin, 1980. doi:10.1007/3-540-10235-3.
21. J. A. Bergstra, J. W. Klop. Process Algebra for Synchronous Communication. *Information and Control*, 60(1–3):109–137, 1984. doi:10.1016/S0019-9958(84)80025-X.
22. U. A. Acar, G. E. Blelloch, R. Harper. Adaptive Functional Programming. *ACM Transactions on Programming Languages and Systems*, 28(6):990–1034, 2006. doi:10.1145/1186632.1186634.
23. J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, A. Schmitt. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View-Update Problem. *ACM Transactions on Programming Languages and Systems*, 29(3):Article 17, 2007. doi:10.1145/1232420.1232424.

24. S. Mac Lane. *Categories for the Working Mathematician*. 2nd ed., Graduate Texts in Mathematics, vol. 5, Springer, New York, 1998. doi:10.1007/978-1-4757-4721-8.